



저작자표시-비영리-변경금지 2.0 대한민국

이용자는 아래의 조건을 따르는 경우에 한하여 자유롭게

- 이 저작물을 복제, 배포, 전송, 전시, 공연 및 방송할 수 있습니다.

다음과 같은 조건을 따라야 합니다:



저작자표시. 귀하는 원저작자를 표시하여야 합니다.



비영리. 귀하는 이 저작물을 영리 목적으로 이용할 수 없습니다.



변경금지. 귀하는 이 저작물을 개작, 변형 또는 가공할 수 없습니다.

- 귀하는, 이 저작물의 재이용이나 배포의 경우, 이 저작물에 적용된 이용허락조건을 명확하게 나타내어야 합니다.
- 저작권자로부터 별도의 허가를 받으면 이러한 조건들은 적용되지 않습니다.

저작권법에 따른 이용자의 권리는 위의 내용에 의하여 영향을 받지 않습니다.

이것은 [이용허락규약\(Legal Code\)](#)을 이해하기 쉽게 요약한 것입니다.

[Disclaimer](#)

철도시스템공학박사 학위논문

도시철도 안전관리를 위한
딥러닝 기반 이상행동 감지



2023년 2월

경성대학교 대학원

철도시스템공학과

임 승 진

철도시스템공학박사 학위논문

도시철도 안전관리를 위한
딥러닝 기반 이상행동 감지

지도교수 박 장 식

이 논문을 철도시스템공학박사 학위논문으로 제출함

2022년 12월

경성대학교 대학원

철도시스템공학과

임 승 진

임승진의 철도시스템공학박사 학위논문을 인준함

위 원 장 이 동 희 (인)

위 원 김 현 태 (인)

위 원 김 석 찬 (인)

위 원 송 종 관 (인)

위 원 박 장 식 (인)

2022년 12월

경성대학교 대학원

목 차

표 목차	iv
그림 목차	v
I. 서론	1
1. 연구 배경	1
2. 연구 목표	3
II. 객체 검출 및 이상행동 감지 딥러닝	5
1. 선행연구 분석	5
2. 딥러닝 개념	6
3. 딥러닝 모델	7
3.1 YOLO 계열	7
3.2 SSD	19
3.3 EfficientNet	21
III. 제안하는 이상행동 감지 딥러닝	22
1. 인공지능 개요	22
2. AI의 국내 환경	23
3. AI 학습용 데이터 구축	24
3.1 AI 학습용 데이터 구축과 서비스 출시 절차	24
3.2 AI 학습용 데이터 구축 방법	26
4. 팽창 컨벌루션을 적용한 YOLOv5	28

IV. 실험 및 결과	30
1. 실험 환경	30
1.1 학습데이터 생성	30
1.2 이상행동 학습데이터	31
1.3 YOLOv5 구조와 모델	35
2. YOLOv5 2 클래스 이상행동 감지 시뮬레이션 결과 및 고찰	37
2.1 YOLOv5 n버전 2 클래스 분류 학습 결과	37
2.2 YOLOv5 s버전 2 클래스 분류 학습 결과	43
2.3 YOLOv5 m버전 2 클래스 분류 학습 결과	46
2.4 YOLOv5 l버전 2 클래스 분류 학습 결과	49
2.5 YOLOv5 x버전 2 클래스 분류 학습 결과	52
2.6 팽창(dilated) 컨벌루션 YOLOv5 2 클래스 학습 및 추론 결과	58
3. YOLOv5 4 클래스 이상행동 감지 시뮬레이션 결과 및 고찰	61
3.1 YOLOv5 n버전 4 클래스 분류 학습 결과	61
3.2 YOLOv5 s버전 4 클래스 분류 학습 결과	64
3.3 YOLOv5 m버전 4 클래스 분류 학습 결과	67
3.4 YOLOv5 l버전 4 클래스 분류 학습 결과	70
3.5 YOLOv5 x버전 4 클래스 분류 학습 결과	73
3.6 YOLOv5 n버전 팽창(dilated) 컨벌루션 4 클래스 분류 학습 결과	79
3.7 YOLOv5 s버전 팽창(dilated) 컨벌루션 4 클래스 분류 학습 결과	82
3.8 YOLOv5 m버전 팽창(dilated) 컨벌루션 4 클래스 분류 학습 결과	85

3.9 YOLOv5 1버전 팽창(dilated) 컨벌루션 4 클래스 분류 학습 결과 ..	88
3.10 YOLOv5 x버전 팽창(dilated) 컨벌루션 4 클래스 분류 학습 결과 ·	91
4. 안드로이드 앱 구현 결과	97
V. 결론	99
참고문헌	101
ABSTRACT	106



표 목차

표 1. YOLOv5 학습결과 성능 비교표	56
표 2. YOLOv5 4 클래스 학습결과 성능 비교표	77



그림 목차

그림 1. 신경회로망 기본 구성	6
그림 2. YOLOv1 모델	9
그림 3. YOLOv1의 구조	9
그림 4. VOC와 COCO 데이터셋에 대한 클러스터링 박스	11
그림 5. 바운딩 박스와 검출 위치 추정	12
그림 6. YOLOv3의 구조	13
그림 7. YOLOv4의 구조	15
그림 8. YOLOv5의 기본적인 구조	17
그림 9. MS COCO Object Detection	18
그림 10. SSD 모델의 레이어 구조	20
그림 11. SSD의 모델 구조	20
그림 12. 레이어 깊이에 따른 성능 변화	21
그림 13. 목적에 따른 AI 학습용 데이터 활용 체계	24
그림 14. AI 학습용 데이터 수집 및 서비스 출시 단계	25
그림 15. AI 학습용 데이터 구축 방법	26
그림 16. 3x3 컨벌루션과 5x5 팽창 컨벌루션	28
그림 17. 팽창 컨벌루션을 적용한 결과 예	29
그림 18. AI Hub 구축 플랫폼	31
그림 19. AI Hub 지하철 이상행동 데이터 예	31
그림 20. Roboflow 플랫폼 및 가공 현황	33
그림 21. 정상상태 영상 라벨링	33
그림 22. 쓰러짐 영상 라벨링	34

그림 23. 쓰러짐과 정상상태 영상 라벨링 결과	34
그림 24. Proposed YOLOv5 Structure	35
그림 25. YOLOv5 5가지 모델	36
그림 26. YOLOv5 학습 그래프	37
그림 27. YOLOv5 학습결과 혼란 행렬(confusion matrix)	38
그림 28. YOLOv5 n버전 학습결과 Precision-Confidence 그래프	40
그림 29. YOLOv5 n버전 학습결과 Precision-Recall 그래프	40
그림 30. YOLOv5 n버전 학습결과 Precision-Recall Confidence 그래프 ..	41
그림 31. YOLOv5 n버전 학습결과 F1-Confidence 그래프	41
그림 32. YOLOv5 GT 영상 및 라벨	42
그림 33. YOLOv5 n버전 학습 결과	42
그림 34. YOLOv5 s버전 학습결과 Precision-Confidence 그래프	43
그림 35. YOLOv5 s버전 학습결과 Precision-Recall 그래프	44
그림 36. YOLOv5 s버전 학습결과 Precision-Recall Confidence 그래프 ..	44
그림 37. YOLOv5 s버전 학습결과 F1-Confidence 그래프	45
그림 38. YOLOv5 s버전 학습 결과	45
그림 39. YOLOv5 m버전 학습결과 Precision-Confidence 그래프	46
그림 40. YOLOv5 m버전 학습결과 Precision-Recall 그래프	47
그림 41. YOLOv5 m버전 학습결과 Precision-Recall Confidence 그래프 ..	47
그림 42. YOLOv5 m버전 학습결과 F1-Confidence 그래프	48
그림 43. YOLOv5 m버전 학습 결과	48
그림 44. YOLOv5 l버전 학습결과 Precision-Confidence 그래프	49
그림 45. YOLOv5 l버전 학습결과 Precision-Recall 그래프	50

그림 46. YOLOv5 1버전 학습결과 Precision-Recall Confidence 그래프	50
그림 47. YOLOv5 1버전 학습결과 F1-Confidence 그래프	51
그림 48. YOLOv5 1버전 학습 결과	51
그림 49. YOLOv5 x버전 학습결과 Precision-Confidence 그래프	52
그림 50. YOLOv5 x버전 학습결과 Precision-Recall 그래프	53
그림 51. YOLOv5 x버전 학습결과 Precision-Recall Confidence 그래프	53
그림 52. YOLOv5 x버전 학습결과 F1-Confidence 그래프	54
그림 53. YOLOv5 x버전 학습 결과	54
그림 54. YOLOv5 2 클래스 학습결과 성능 비교	57
그림 55. YOLOv5 2 클래스 학습결과 시간 비교	57
그림 56. 3x3 컨벌루션의 학습 결과 혼란 행렬(confusion matrix)	59
그림 57. 5x5 팡창 컨벌루션 적용한 혼란 행렬(confusion matrix)	59
그림 58. 3x3 컨벌루션 YOLOv5 n버전 F1-Confidence 그래프	60
그림 59. 팡창 컨벌루션 YOLOv5 n버전 F1-Confidence 그래프	60
그림 60. YOLOv5 n버전 학습결과 Precision-Confidence 그래프	61
그림 61. YOLOv5 n버전 학습결과 Precision-Recall 그래프	62
그림 62. YOLOv5 n버전 학습결과 Precision-Recall Confidence 그래프	62
그림 63. YOLOv5 n버전 학습결과 F1-Confidence 그래프	63
그림 64. YOLOv5 n버전 학습 결과	63
그림 65. YOLOv5 s버전 학습결과 Precision-Confidence 그래프	64
그림 66. YOLOv5 s버전 학습결과 Precision-Recall 그래프	65
그림 67. YOLOv5 s버전 학습결과 Precision-Recall Confidence 그래프	65
그림 68. YOLOv5 s버전 학습결과 F1-Confidence 그래프	66

그림 69. YOLOv5 s버전 감지 결과	66
그림 70. YOLOv5 m버전 학습결과 Precision-Confidence 그래프	67
그림 71. YOLOv5 m버전 학습결과 Precision-Recall 그래프	68
그림 72. YOLOv5 m버전 학습결과 Precision-Recall Confidence 그래프	68
그림 73. YOLOv5 m버전 학습결과 F1-Confidence 그래프	69
그림 74. YOLOv5 m버전 학습 결과	69
그림 75. YOLOv5 l버전 학습결과 Precision-Confidence 그래프	70
그림 76. YOLOv5 l버전 학습결과 Precision-Recall 그래프	71
그림 77. YOLOv5 l버전 학습결과 Precision-Recall Confidence 그래프 ..	71
그림 78. YOLOv5 l버전 학습결과 F1-Confidence 그래프	72
그림 79. YOLOv5 l버전 학습 결과	72
그림 80. YOLOv5 x버전 학습결과 Precision-Confidence 그래프	73
그림 81. YOLOv5 x버전 학습결과 Precision-Recall 그래프	74
그림 82. YOLOv5 x버전 학습결과 Precision-Recall Confidence 그래프 ·	74
그림 83. YOLOv5 x버전 학습결과 F1-Confidence 그래프	75
그림 84. YOLOv5 x버전 학습 결과	75
그림 85. YOLOv5 4 클래스 학습결과 성능 비교	78
그림 86. YOLOv5 4 클래스 학습결과 시간 비교	78
그림 87. 팽창 컨벌루션 YOLOv5 n버전 학습결과 Precision-Confidence 그래프	79
그림 88. 팽창 컨벌루션 YOLOv5 n버전 학습결과 Recall-Confidence 그래프	80

그림 89. 팽창 컨벌루션 YOLOv5 n버전 학습결과	
Precision-Recall Confidence 그래프	80
그림 90. 팽창 컨벌루션 YOLOv5 n버전 학습결과	
F1-Confidence 그래프	81
그림 91. 팽창 컨벌루션 YOLOv5 n버전 학습결과	81
그림 92. 팽창 컨벌루션 YOLOv5 s버전 학습결과	
Precision-Confidence 그래프	82
그림 93. 팽창 컨벌루션 YOLOv5 s버전 학습결과	
Recall-Confidence 그래프	83
그림 94. 팽창 컨벌루션 YOLOv5 s버전 학습결과	
Precision-Recall Confidence 그래프	83
그림 95. 팽창 컨벌루션 YOLOv5 s버전 학습결과	
F1-Confidence 그래프	84
그림 96. 팽창 컨벌루션 YOLOv5 s버전 학습결과	84
그림 97. 팽창 컨벌루션 YOLOv5 m버전 학습결과	
Precision-Confidence 그래프	85
그림 98. 팽창 컨벌루션 YOLOv5 m버전 학습결과	
Recall-Confidence 그래프	86
그림 99. 팽창 컨벌루션 YOLOv5 m버전 학습결과	
Precision-Recall Confidence 그래프	86
그림 100. 팽창 컨벌루션 YOLOv5 m버전 학습결과	
F1-Confidence 그래프	87
그림 101. 팽창 컨벌루션 YOLOv5 m버전 학습결과	87

그림 102. 팽창 컨벌루션 YOLOv5 1버전 학습결과	
Precision-Confidence 그래프	88
그림 103. 팽창 컨벌루션 YOLOv5 1버전 학습결과	
Recall-Confidence 그래프	89
그림 104. 팽창 컨벌루션 YOLOv5 1버전 학습결과	
Precision-Recall Confidenc 그래프	89
그림 105. 팽창 컨벌루션 YOLOv5 1버전 학습결과	
F1-Confidence 그래프	90
그림 106. 팽창 컨벌루션 YOLOv5 1버전 학습결과	90
그림 107. 팽창 컨벌루션 YOLOv5 x버전 학습결과	
Precision-Confidence 그래프	91
그림 108. 팽창 컨벌루션 YOLOv5 x버전 학습결과	
Recall-Confidence 그래프	92
그림 109. 팽창 컨벌루션 YOLOv5 x버전 학습결과	
Precision-Recall Confidenc 그래프	92
그림 110. 팽창 컨벌루션 YOLOv5 x버전 학습결과	
F1-Confidence 그래프	93
그림 111. 팽창 컨벌루션 YOLOv5 x버전 학습결과	93
그림 112. YOLOv5 n버전 학습결과	94
그림 113. 팽창 컨벌루션 YOLOv5 n버전 학습결과	94
그림 114. 3x3 컨벌루션 결과와 팽창 컨벌루션 n버전 비교	95
그림 115. 안드로이드 앱 구현 과정	96
그림 116. 안드로이드 앱 구현 결과	97

I. 서론

1. 연구 배경

우리나라는 1970년대 초반에 들어서면서 산업의 발전과 함께 늘어난 교통수요를 처리하기 위하여 도시철도를 건설하기 시작하였다. 1974년 8월 15일 서울의 도시철도 1호선이 처음 개통되었고, 부산은 1985년 7월 19일 1호선을 개통하여 총 4개 호선을 운영 중이며 현재 1일 평균 90만여명을 수송하고 있다. 수송 인원은 과거에 비해 폭발적으로 늘어난 반면 역사에 근무하는 직원은 매표, 게이트 관리 등 시스템의 자동화에 따라 관리 직원이 감소하는 실정이다[1]. 한 개 역사 근무 직원이 감소하여 발생하는 업무의 공백을 감시카메라(CCTV)로 대체됨에 따라 적은 인원으로 역사의 안전을 확보하고자 한다. 도시철도 직원은 역무실 감시카메라(CCTV)로 역사 및 승강장을 감시해야 하지만 제한된 인원으로서 각 지역 없이 지속적해서 모니터를 감시할 수는 없다.

한편, 최근 도시철도 역사 및 승강장에서는 다양한 안전사고와 범죄가 발생하고 있으며, 부산교통공사의 경우 최근 5년간 안전사고 5,984건, 각종 범죄 관련 사건이 1,215건이 발생하였고 안전사고와 범죄로 인한 인적·물적 피해가 지속 증가하고 있으며, 이를 예방하기 위한 노력과 비용 또한 꾸준히 증가하는 추세다. 또한, 최근 코로나19 확산을 막기 위한 감염자 동선 파악, 정신 지체장애인 실종자 및 시각 장애인 이동상황 확인 등 최근 발생하는 사회적 이슈에 따라 활용 수요가 지속 증가하고 있다[2].

대표적인 사고로는 2003년 2월 18일 발생한 대구 도시철도 화재 사고를 들 수 있다. 이 사건은 운동복 차림의 한 남성이 약수통을 들고 다른 한 손에는 라이터를 들고 라이터를 켜다 켜는 행동을 반복하고 있었다[3]. 이러한 행동들은 역무원이 직접 확인하거나 주변 승객의 신고가 없이는 현장 확인이 어렵다. 그리고 CCTV 모니터를 지속 감시할 수 없는 여건이 있어 현장 조치가 제한된다.

한편, 최근 안전의 중요성 증가와 중대재해처벌 등에 관한 법률 시행으로 도시철도 역사 또는 승강장에 감시설비를 신규 및 증설이 계속 이루어지고 있으며, 감시설비의 증설과 신규 장비 교체를 통해 고화질 획득 및 최신영상분석 플랫폼을 활용할 수 있는 바탕이 되고 있다. 이는 근무 직원 감소에 대응한 대안으로 추진 할 수 있으며, 각종 사고를 사전에 방지하거나 사고가 발생하였을 때 신속한 확인과 조치로 골든타임을 확보하고 소방, 경찰 등 관계기관과 협업하여 사고를 조치를 할 수 있는 바탕이 될 수 있다. 또한 철도안전법 제39조3 영상기록장치의 설치·운영 등에 따라 철도차량의 운전실 전면과 운전실의 영상기록, 사고 발생의 상황 파악, 다양한 안전사고 방지, 각종 범죄 예방을 위하여 영상기록 장치를 운영하도록 하고 있다. 따라서 철도안전법에 따라 의무화되어 있는 영상기록장치를 활용하여 각종 상황에 대한 객체인식, 추적과 감시 등을 적용한다면 안전 위해 요소의 사전 차단과 후속 조치가 가능할 것으로 판단된다.

최근 기계학습 딥러닝(Machine learning, deep learning) 기술 발전으로 CCTV 영상에 대하여 CNN 딥러닝(Convolutional Neural Networks)을 활용한 자동 객체 검출(detection) 및 분류(Classification), 추적(Tracking) 등의 다양한 응용 분야에 활용되고 있다. 객체 검출은 종래의 기계학습 DPM(Deformable part model) 을 근간으로 하는 R-CNN(Regions with Convolutional Neuron Networks Features), Fast R-CNN, Faster R-CNN 등으로 발전하였다. 그러나 언급한 딥러닝 모델들의 처리 속도는 각각 0.7, 0.5, 5, 그리고 7 fps로 실시간 모니터링에 다소 부적합하다. 2016년 발표된 YOLO는 455 fps의 고속 처리가 가능한 YOLO(You Only Look Once)의 모델이[12] 2016년도 제안되어 영상에서 객체를 검출하는 다양한 분야에 활용되고 있다.

도시철도 역사 내에서 쓰러짐, 폭행 등의 이상행동 등의 상황인식 기술도 딥러닝을 기술을 적용하여 개선되고 있으며, 이를 바탕으로 역사 내 다양한 감시 장비를 기반으로 안전 관리를 향상할 수 있을 것이다.

2. 연구 목표

본 논문에서는 도시철도 역사의 안전 관리를 위하여 CCTV의 효율적인 운영 방안을 제시하고자 한다. 최근 다양한 분야에 응용되고 있는 딥러닝을 활용해서 자동으로 역사 내에서 이상행동을 감지한다. 본 논문에서는 객체 검출에 널리 활용되고 있는 YOLO 딥러닝 모델에 대하여 AI Hub에서 제공하는 도시철도 이상행동 데이터를 활용하고 최적의 모델에 대하여 팽창(dilated) 컨벌루션을 적용하여 성능을 개선한다.

이상행동 CCTV 영상 데이터를 부산도시철도에서 확보하기에는 개인정보 보호법에 따라 제한되는 실정이다. AI hub에서 제공하는 학습용 이상행동 CCTV 영상 데이터셋을 이용하여 이상행동 감지 연구를 시행한다.

이상행동 CCTV 영상 데이터셋은 다양한 데이터를 바탕으로 AI 기술을 적용하여 새로운 제품생산과 혁신적 서비스를 제공하는 현시대에 AI와 데이터는 전체 산업에서의 혁신성장을 가속화 할 수 있는 중요한 요소이다. AI와 데이터는 4차 산업혁명 시대의 중요한 생산요소이며 기업이나 기관이 경쟁력을 갖추기 위해서는 민·관 협동으로 대규모 데이터를 구축하고 개방하는 것이 필요하며, 그 데이터를 바탕으로 현 상황에 맞는 연구를 통해 제품을 개발하는 것이 기본이 될 것이다. 미국, 유럽 등 AI 선도국가에서는 정부와 민간의 유기적인 협력과 투자로 빅 데이터를 구축 및 제공하고 있다. 국내 중소기업과 벤처기업들은 AI 학습용 데이터의 확보가 어렵고, 자체적으로 데이터의 구축 및 재가공에 큰 비용과 시간을 소요하고 있다. 이러한 제한사항을 극복하기 위해 한국지능정보사회진흥원에서는 AI Hub 라는 통합 플랫폼을 구축하여 AI 데이터 6종인 한국어, 영상 이미지, 헬스케어, 재난 안전 환경, 농축 수산, 교통물류를 국내외 기관과 기업에 AI 학습용 데이터로 제공하고 있다[4].

본 연구에서는 이상행동 인식을 다루는 여러 가지 논문을 분석하였고, 도시철도에서 많이 발생할 수 있는 폭행, 절도, 주취 행동, 싸움 등 13가지 시나리오를 분석

하여 이중 이상행동과 동작에 대한 주요 특징은 다음과 같이 정의하였다. 첫 번째는 동작으로 사람이 움직임 등을 구체적으로 정의하는 것이고 이상행동으로 구성하는 세부 동작을 20가지로 분류하였는데 그 종류는 휘두르기, 차기, 밀치기, 당기기, 던지기, 위협하기, 찌르기, 쓰러짐, 줍기, 운반하기, 비틀거림, 맴돌기, 가다 서다 반복, 담치기, 떨어뜨림, 빼앗기, 안기, 만지기, 주저앉음이며, 이상행동(Abnormal event)은 여러 동작이 모여서 이루어지는 행동 가운데 정상이 아닌 행동으로 12가지로 분류하였고 그 종류로는 폭행, 기물파손, 실신, 배회, 침입, 투기, 납치, 강도 등의 행동이다[5].

본 논문에서는 도시철도에서 가장 발생률이 높은 재난 안전 환경 데이터 영역의 데이터셋을 활용하며 실시간으로 도시철도 역사 내 이상행동을 감지하기 위하여 객체 검출 딥러닝 모델의 감지 성능을 비교 분석하고, 경량화를 통한 임베디드 에지컴퓨팅에 적합한 모델을 제안한다. 제안하는 모델은 YOLOv5 n 버전을 팽창 컨벌루션을 적용하여 성능을 개선할 것이다. 또한 이상행동 감지 정확도를 개선하기 위하여 학습데이터를 2가지 클래스에서 4가지 클래스로 확대하고 도시철도 역사 내에서 발생할 수 있는 이상행동 이벤트를 추가하여 개발한 이상행동 감지 시스템을 개선하고자 한다.

II. 객체 검출 및 이상행동 감지 딥러닝

1. 선행연구 분석

도시철도 역사 내 이상행동 상황의 효율적 안전관리를 위해 여러 논문과 관련 자료를 분석하였고, 사람 행동 인식(HAR, Human Activity Recognition)은 지난 10년 동안 비디오 감시, 가상현실, 지능형 인간, 컴퓨터 상호작용 등과 같이 여러 응용 프로그램으로 발전해 나가고 있다. HAR은 패턴 인식 문제로 취급될 수 있고, 이러한 것은 기계학습 분야에서 많은 부분이 성공적인 결과를 도출하였다. 특히, LSTM(Long Short Term Memory)은 RNN으로부터 개선된 모델 구조를 바탕으로 인간 활동 인식을 94% 이상의 정확도를 달성했다[6].

이러한 개선된 모델을 기반으로 도시철도 역사 감시를 위한 다양한 연구를 진행해 나가고 있다. 한편, 도시철도 분야에는 비용 절감 등으로 역사 내 관리 직원이 지속 감소 되고 있고, 이러한 안전관리 능력을 향상하기 위해 보안 감시용 CCTV 카메라(video surveillance) 설치가 확대되고 있으나 직원의 다양한 업무로 인해 지속적인 감시가 어려운 실정이다. 이러한 상황을 극복하기 위해 특정 상황이나 사람의 행동을 자동으로 인식하기 위한 기술들이 연구 개발되고 있으며, 객체 검출, 추적 및 행동인식 딥러닝(deep learning) 기술 적용을 통하여 성능이 현저히 개선되고 있다[7]. 이러한 성능 개선을 위해 이상행동에 대한 AI Hub의 학습용 데이터셋을 활용하여 역사 내 이상행동을 자동으로 감지하여 신속한 대응을 지원하기 위한 지능형 CCTV에 적합한 딥러닝 모델을 비교 분석한다. 도시철도 이상행동 중 위급상황인 쓰러짐, 폭행, 기물파손 데이터를 전처리하여 YOLOv5를 활용하여 학습하고 안드로이드 기반 어플리케이션 구현을 통해 실증하여 실시간으로 효율적인 역사 안전관리를 위하여 모델에 대한 경량화와 성능 개선에 관한 연구이다.

2. 딥러닝 개념

딥러닝은 인간의 뇌와 유사한 방식으로 사고하고 기능하도록 만들어진 알고리즘을 사용한 기계학습(Machine learning)의 한 유형이다. 딥러닝은 기계학습의 하위 집합이고, 기계학습과 딥러닝을 포함한 전체를 AI(Artificial Intelligence)라고 한다. AI는 인간의 지적 능력을 컴퓨터를 통해 나타내는 기술이며 사람처럼 자유로운 사고가 가능한 자아를 지닌 것을 말한다[8].

AI 기반 시스템은 인간의 행동과 의사 결정 구조를 모방하거나 모델링하기 위해 인공신경망(Artificial Neural Network, ANN) 또는 그 밖의 기계학습 변형 등 다양한 방법을 활용한다[8].

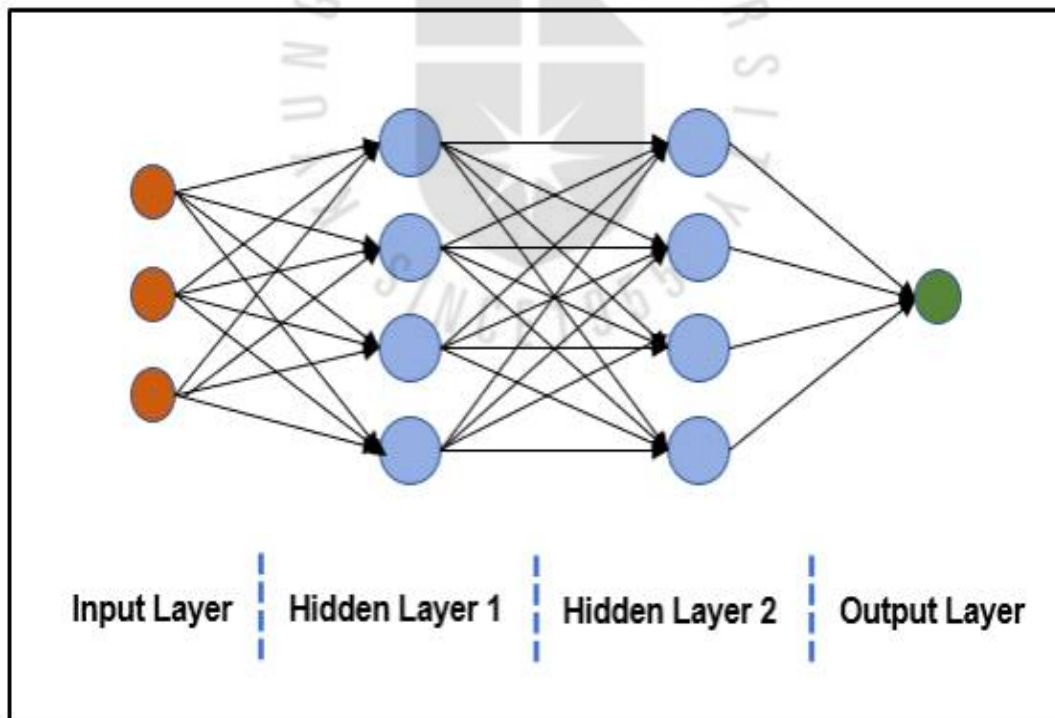


그림 1. 신경회로망 기본 구성

AI의 하위 분야인 기계학습은 여러 가지의 학습용 데이터에서 일반적인 규칙을 도출하기 위한 자동화된 절차로 구성된다. 즉, 학습용 데이터로부터 규칙이 학습되고 사전에 정의된 알고리즘에 적용할 규칙을 더해 수행하게 되고 이러한 모델의 대표적인 것이 인공신경망을 사용한 딥러닝이다. 기계학습은 지도학습과 비지도 학습으로 구분되며, 지도학습은 학습을 위한 전처리 데이터의 입력값과 예상 결과값 모두를 포함하는 반면, 비지도 학습에서는 시스템에서의 입력값 자체에서 그 결과를 식별한다.

딥러닝은 여러 비선형 변환기법의 조합을 통해 높은 수준의 추상화(abstractions)를 시도하는 기계학습 알고리즘의 집합으로 정의되며[9], 사람의 사고방식을 컴퓨터에 가르치는 기계학습 분야라고 이야기할 수 있다.

수많은 데이터를 기반으로 컴퓨터가 이해할 수 있는 형태로 표현하고, 학습하기 위한 연구가 많이 진행되고 있으며, 여러 노력을 통해 DNN(Deep Neural Networks), CNN(Convolutional Neural Network) 등 여러 딥러닝 기법이 컴퓨터를 통해 다양한 분야에 적용되어 활용하고 있다[10].

3. 딥러닝 모델

3.1 YOLO 계열

YOLO는 You only Live Once 의 약자로 “인생은 오직 한 번뿐”이라는 의미로 개인의 자유로운 라이프 스타일을 나타낸다[11]. 그러나 딥러닝에서는 “You Only Look Once” 즉, “한 번에 보고 바로 처리하겠다”라는 의미로 속도가 빠르다는 장점을 내세운 객체 인식이다[11]. YOLO는 YOLOv1부터 2020년 7월 기준으로 YOLOv5, 그리고 최근에는 YOLOv7 까지 공개되어 개발자뿐만 아니라 학생들까지 많은 사람이 인용하고 관련 사항을 활용 및 연구 중이다[11]. 이 절에서는 YOLOv1부터 YOLOv7까지 모델 구조와 특징들에 대해 알아보고자 한다.

3.1.1 YOLOv1¹⁾

객체 검출은 단일창(single window)나 영역 제안법(regional proposal methods) 등을 기반으로 바운딩 박스(bounding box)를 영역내 탐지된 bbox에 대해 분류 수행하는 2단계(2 stage detection)으로, 영상검출(image detection)을 완료하는 알고리즘을 통해 진행되었다. 복잡한 파이프라인으로 학습과 예측이 느려지게 되고 최적화도 느려지는 문제가 있었다. YOLOv1은 특정 컨벌루션 네트워크를 기반으로 대상의 위치와 클래스를 예측하기 때문에 YOLO가 처음 등장했을 때 높은 관심과 주목을 받았으며, YOLOv1의 장점으로 기존 인식 모델들에 비해 학습 파이프라인이 간단하고, 학습과 예측의 속도가 빠르며 모든 이미지의 학습 대상과 이미지 전체와 특징을 학습하고 일반적인 대상의 특징 학습을 통해 다른 영역으로의 확장에도 뛰어난 성능을 보이는 장점이 있다.

YOLOv1의 모델 구조의 특징은 bbox 좌표와 분류를 통해 동일 신경망 구조가 동시에 진행하는 통합인식(Unified Detection)을 나타내는 특징을 가지고 있고 그림 2와 같이 이미지를 $S \times S$ 의 그리드(Grid)로 분할하고 이미지의 특징을 추출하기 위해 전체를 신경망에 넣은 후 예측 텐서(Prediction Tensor) 생성한다[12]. 예측 텐서는 그리드별 bbox의 정보, 신뢰할 수 있는 점수, 분류된 클래스 확률을 포함하게 되며[12], 그리드별 예측정보를 바탕으로 테두리 박스 조정 및 분류작업을 수행한다[12]. 그리고 개개의 그리드 셀(Grid cell)은 bbox와 또 다른 bbox에 대한 신뢰점수(Confidence-Score)를 가지게 되고 각각의 그리드 셀은 C개의 조건을 가진 클래스 확률(Conditional Class Probability)를 가지게 된다. 각각의 bbox는 x, y 좌표와 w(width), h(height), 신뢰점수를 지니게 되며 x와 y는 bbox의 중심 좌표이다[12].

YOLO 네트워크는 그림 3과 같이 224×224 크기의 ImageNet 분류 레이어로 사전학습을 진행하고 448×448 이미지를 받아 24개의 컨벌루션 레이어(Conv Layer) 중 앞의 20개의

1) <https://blog.nerdfactory.ai/2021/05/06/You-Only-Look-Once.-YOLO.html>

컨벌루션 레이어는 고정하고 뒷부분 4개의 레이어와 2개의 FC 레이어(Fully Connected Layer)만 객체 검출에 맞게 학습하게 된다[13]. 이후 예측 텐서를 구분해 내고 이것을 바탕으로 검출 및 분류를 수행하게 된다[13].

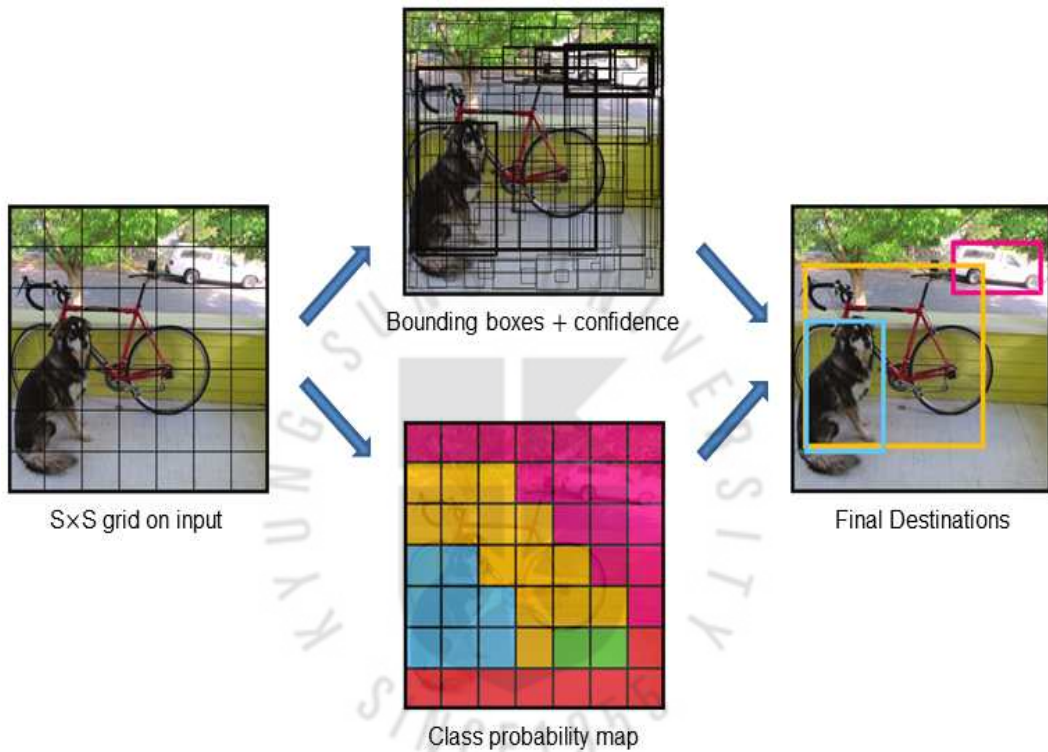


그림 2. YOLOv1 모델

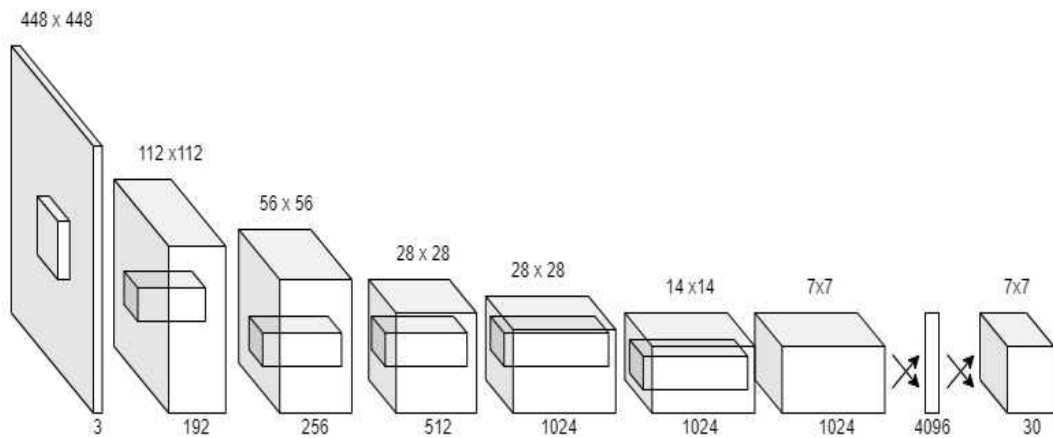


그림 3. YOLOv1의 구조

3.1.2 YOLOv2

YOLOv1에서의 한계는 여러 물체들이 여러 겹으로 겹쳐 있을 경우 정확한 예측이 어려운 한계가 있다. 또한 물체가 작을수록 정확도가 감소하며 bbox 형태가 데이터를 통해 학습되므로 새로운 형태의 bbox인 경우는 정확히 예측하지 못한다는 단점이 있다[13]. 이러한 한계와 문제점을 보완하여 정확도를 높인 것이 YOLOv2이다. YOLOv2에서의 핵심은 YOLOv1과는 다르게 앵커 박스(Anchor Box)의 개념을 도입하였고, 배치 정규화(Batch Normalization) 적용하였으며 특징 맵 크기를 7×7 에서 13×13 으로 변경 하는 등의 방법을 통해 성능 개선을 이루었다[14].

그 세부 개선 사항으로는 배치 정규화를 활용하여 YOLO의 다양한 데이터 분포를 배치 별로 정규화하고 적용하여 mAP를 약 2% 향상시켰다[15]. 또한 고해상의 분류를 적용한 YOLOv1은 VGG16 모델을 기반으로 224×224 크기의 해상도로 학습을 하고, 448×448 크기의 이미지에 대해서 객체검출을 수행하도록 설계되어 성능이 좋지 않았지만, YOLOv2에서는 학습 전 이미지 분류 모델을 큰 해상도의 이미지에 대해 미세조정(fine-tuning) 단계를 거쳐 고해상도 이미지로 CNN 신경망

을 학습할 수 있었으며, 그 결과 약 4% 증가의 결과를 도출하였다[15]. 기존 앵커 박스가 있는 컨벌루션 YOLOv1은 24개의 컨벌루션 레이어와 마지막 단락의 2개의 연결 계층으로부터 bbox의 좌표정보를 예측하였고, YOLOv2는 마지막 단의 완전한 연결 계층을 떼어내고 컨벌루션 네트워크 형태로 예측값을 계산한다[15]. 그리고 앵커 박스의 개념을 도입하여 bbox의 좌표를 예측하기 보다는 사전에 정의한 앵커 박스에서 상쇄한 것을 좀더 간단하게 예측하여 학습할 수 있다. 앵커 박스는 사전에 크기와 비율 전체가 결정되어 있는 박스 기반 학습을 통해서 이 박스의 위치나 크기를 세부적으로 조정하는 것을 말한다[15]. Dimension Cluster YOLOv2는 앵커 박스를 수정하면서 bbox를 예측한다. 따라서 그림 4와 같이 실제 bbox를 클러스팅(Clustering)하여 최적의 앵커 박스를 찾는다. 클러스팅 갯수 k 를 크게 늘릴수록 클러스팅 결과와 라벨과의 IoU가 커져 재현율(Recall)이 상승한다. 그러나 k 가 커지면 높은 정확도를 얻을 수 있지만 속도가 느려지기 때문에 YOLOv2에서는 $k=5$ 라는 값을 사용하여 그 결과 재현율과 정밀도 측면에서 성능을 향상 시킬 수 있었다[16].

또한 그림 5와 같이 직접적인 위치 예측을 통해 YOLOv2에서는 앵커 박스에 따라 하나의 셀에서 5차원 벡터로 이루어진 bbox를 예측하며, bbox가 그리드 셀에서 벗어나지 않도록 제한을 두고, 기존의 YOLO가 그리드의 중심점을 예측하였다면 YOLOv2에서는 왼쪽 상단의 꼭지점으로부터 얼마나 이동하는지 예측하게 된다[15].

그리고 13×13 의 크기는 작은 물체 검출의 문제해결을 위해 상위 레이어의 특징 맵을 하위 특징 맵에 합쳐주는 통과층(passthrough layer)을 도입했다[16]. 이전 레이어의 26×26 특징 맵을 가지고 와서 13×13 특징 맵에 이어 붙인다. 크기가 달라 그냥 이어붙일 수 없는 것을 26×26 크기의 특징 맵에 고해상도 특징이 담겨 있는 것을 활용하여 $26 \times 26 \times 512$ 의 특징 맵을 $13 \times 13 \times (512 \times 4)$ 의 특징 맵으로 변환하였다[16].

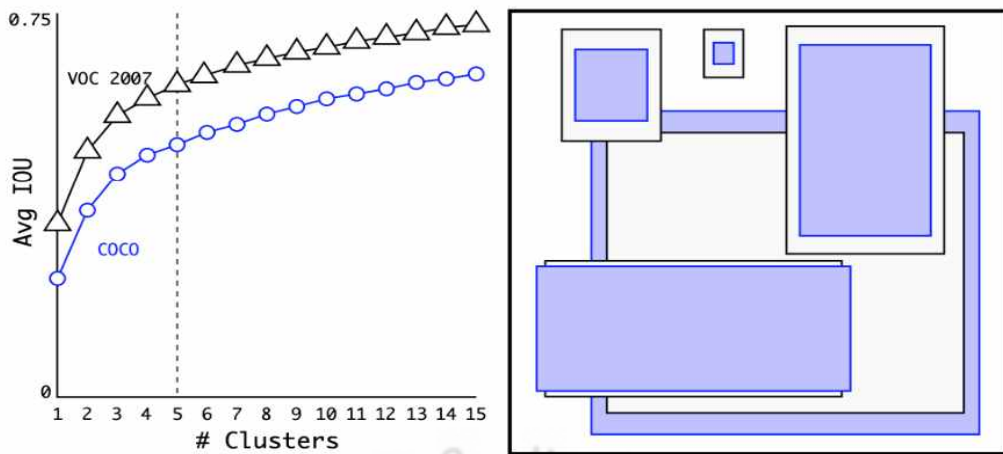


그림 4. VOC와 COCO 데이터셋에 대한 클러스터링 박스

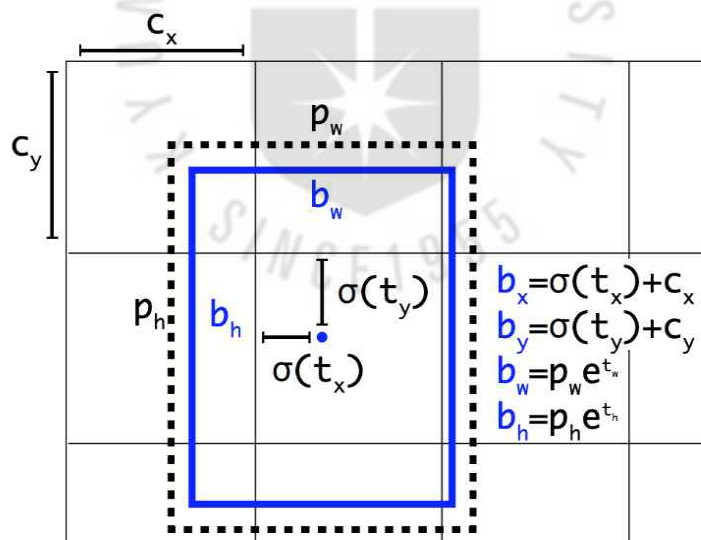


그림 5. 바운딩 박스와 검출 위치 추정

3.1.3 YOLOv3

YOLOv3는 YOLOv2의 개선된 버전으로 bbox를 예측하는 방법은 YOLOv2와 동일하고 tx, ty, tw, th가 예측된다. 그리고 bx, by, bw, bh로 변형한 후 L2 loss를 통해 학습한다[17]. 예측한 bbox마다 신뢰가 높은 점수를 함수를 적용하여 구하며, 앵커 박스와 원본의 실제 값을 가진 박스(ground truth box)와의 IoU 값이 가장 높은 박스만 매칭한다[18]. 원본의 실제 값을 가진 박스에 할당되지 못한 bbox는 bbox의 회귀 손실(bounding box regression loss)을 유발하지 않고, 오직 신뢰가 높은 점수에 대한 손실만 발생시킨다[17]. 또한 각 bbox는 다중 레이블 분류(multi-label classification)를 수행을 하게 되는데 소프트맥스(softmax) 함수를 사용하여 클래스를 예측할 경우 성능 면에서 좋지 않기 때문에 이진 교차점(binary cross-entropy)를 사용한다[17].

YOLOv3는 다중 스케일 특징 맵을 얻는 방법은 FCN(Fully Convolutional Network)과 유사하다. 먼저 416×416 크기의 이미지를 네트워크에 입력하여 특징 맵이 크기가 52×52 , 26×26 , 13×13 이 되는 레이어에서 특징 맵을 추출한다[20]. 그리고 해상도가 가장 낮은 특징 맵을 1×1 , 3×3 컨벌루션 레이어로 구성된 작은 FCN에 입력하고, 이후 FCN 출력채널(output channel)이 512 지점에서 특징 맵을 추출한 뒤 2배로 업샘플링(upsampling)을 수행하게 된다[20,21,22].

YOLOv3의 구조는 그림 6과 같이 이미지를 입력하여 지정한 레이어에서 투입되는 이미지는 416×416 사이즈에 추출된 특징 맵 과정을 거쳐 산출된 52×52 , 26×26 , 13×13 사이즈의 특징 맵이 추출된다[20]. FCN의 기능 피라미드 구축의 투입 이미지는 52×52 , 26×26 , 13×13 사이즈에 피라미드 구축 과정을 거쳐 산출된 $52 \times 52(255)$, $26 \times 26(255)$, $13 \times 13(255)$ 사이즈의 특징 맵이 추출된다[19].

손실 함수에 의한 YOLOv3의 학습은 다중 스케일 특징 맵을 손실함수를 통해 학습을 진행하게 되며, YOLOv2가 MSE(Mean Squared Error) 적용하여 예측한 반면 YOLOv3는 BCE(Binary Cross Entropy)를 주로 사용한다[20].

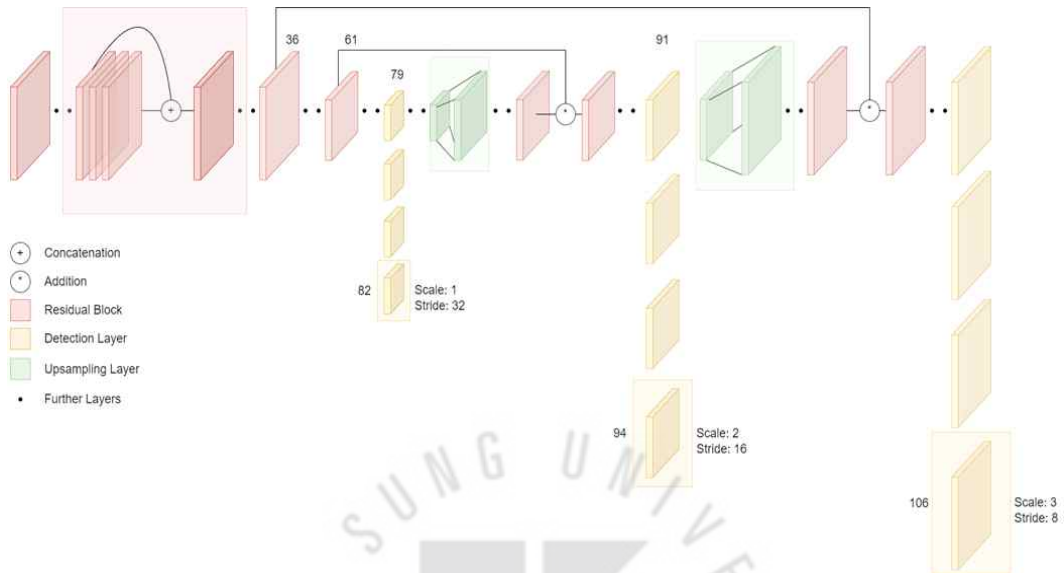


그림 6. YOLOv3의 구조[20]

3.1.4 YOLOv4

YOLOv4는 Alexey Bochkovsky에 의해 2020년 4월에 소개되었으며, 물체의 감지의 최적 속도 및 정확도를 기반으로 한 고품질의 초고속 물체를 감지하는 것을 목표로 하고 있다[23].

YOLOv4는 1단 검출기(one-stage detector)로써 기존의 YOLOv3를 기반으로 처리 속도를 개선한 모델로서 그림 7과 같은 구조이다. YOLOv4는 딥러닝의 여러 기법을 활용하여 모델을 설계하고 학습하였으며, 정확도와 처리 속도의 성능을 크게 개선하였다. 객체 인식은 ImageNet에서 사전 학습된 backbone과 class, bbox를 예측하는 head(Dense Prediction)로 구성되어 있고, Backbone과 head 사이에서 서로 다른 스케일(scale)의 특징 맵을 수집하는 레이어인 Neck이 있으며, 입력 이후 블록이 설명은 다음과 같다[24].

Backbone은 이미지 투입 이후에 추출기 역할을 하는 딥러닝 구조이다. 모든 backbone 모델은 분류 모델로 구성되어 있으며, 입력 이미지를 특징 맵으로 변형하는 역할을 담당하고 있다[25].

Neck은 backbone과 Head를 연결하는 부분으로, 특징 맵을 세분화하고 재구성한다. 대표적으로 FPN(Feature Pyramid Network), PAN(Path Aggregation Network), BiFPN, NAS-FPN 등이 있다[25].

head는 크게 Dense Prediction과 Sparse Prediction으로 나뉘는데, 이는 객체검출의 종류인 1-단인지 2-단인지와 직결된다[25]. Sparse Prediction는 헤드를 사용하는 2-단 검출기는 대표적인 모델로 Faster R-CNN, R-FCN 등이 있다. 예측 클래스와 bbox 회기 부분이 분리되어있는 것이 특징이다[26]. Dense Prediction은 head를 사용하는 One-Stage Detector의 대표적인 모델로 YOLO, SSD 등이 있다[25]. Two-Stage Detector과 다르게 1-단 검출기는 예측 클래스와 bbox 회기가 통합되어있는 것이 특징이다[26]. 따라서 YOLOv4는 구조를 구성하는 것이 매우 중요한데 예를 들면 CSPResNeXt50은 ImageNet 데이터에서 이미지 분류하는 일은 CSPdarknet보다 뛰어나고, CSPdarknet이 MSCOCO 데이터에서 객체를 찾아내는 일에는 CSPResNeXt보다 뛰어나다[25]. 이러한 각기 기능이 뛰어난 것을 비교하여 찾고 수용필드(Receptive field)를 강화시키는 모듈을 선택하는 것인데 원활히 작은 객체를 감지와, 레이어를 증가 시키기 위한것과 단일 이미지에서 크기가 다른 여러 객체를 감지할 수 있게 하는 것이 있는데 YOLOv4는 YOLOv3에 사용된 FPN대신 PANet을 사용하여 backbone level에서 다양한 검출기 레벨(detector level)로 집계 시킬 수 있는 장점이 있다[24]. 결과적으로 YOLOv4는 CSPdarknet53을 backbone으로 SPP 블록(block)을 추가했으며, 탐지 넥(neck)으로 PANet을 사용하고, 탐지 헤드(head)는 YOLOv3의 구조를 따른다.[20,24,27]

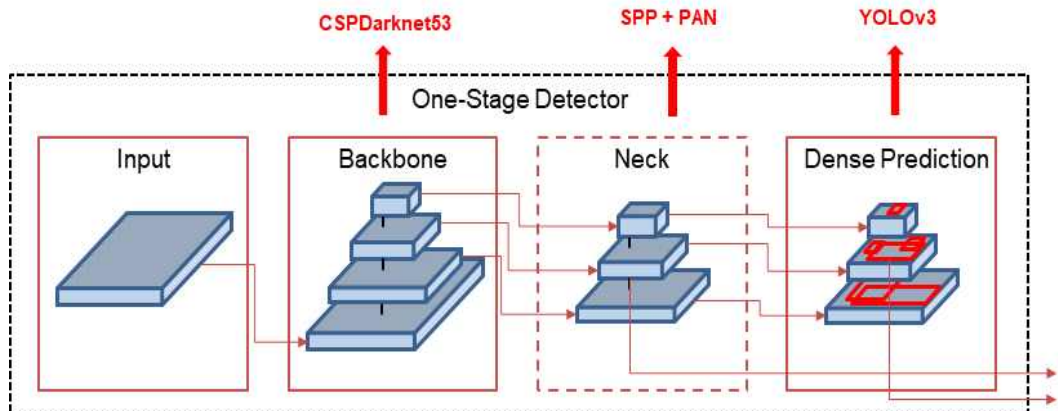


그림 7. YOLOv4의 구조[24]

3.1.5 YOLOv5

YOLOv5 모델은 논문으로 소개된 것이 아니라 알고리즘이 먼저 공개가 되었다 [28]. 학술적인 부분에서는 여러 논란이 존재하지만 본 논문에서는 모델의 구조에 대해서 설명할 것이다. YOLOv5의 구조는 그림 8과 같이 Darknet 대신 PyTorch를 사용한다. CSPDarknet53을 백본(backbone)으로 사용하고, 백본은 큰 백본에서 반복적인 기울기 정보를 해결하고 기울기 변경을 특징 맵에 통합하여 추론 속도를 줄이고 정확도를 높이며 매개변수를 줄여 모델 크기를 줄인다. 경로 집계 네트워크(PANet)를 사용하여 정보 흐름을 향상시킨다. PANet은 여러 상향식 및 하향식 계층을 포함하는 새로운 FPN(feature pyramid network)를 채택하고, 이와 같이 진행하게 되면 모델에서 저수준 기능의 전파가 향상된다. PANet은 하위 레이어의 현지화를 개선하여 객체의 현지화 정확도를 향상시킨다[28]. 또한 YOLOv5의 헤드는 YOLOv4 및 YOLOv3과 동일하며 다중 스케일 예측을 달성하기 위해 특징 맵의 세 가지 다른 출력을 생성한다. 그리고 모델에서 작은 개체부터 큰 개체까지 효율적으로 예측하는데 도움이 된다. 이미지는 기능 추출을 위해 CSPDarknet53에 공급되고 기능 융합을 위해 PANet에 다시 공급된다[29].

YOLO 계층은 결과를 생성하게 되는데 아키텍처를 보면 포커스 레이어(Focus layer)는 YOLOv3 구조에서 발전되었고, YOLOv3의 처음 3개 계층을 대체하고 YOLOv5에서 단일 계

층을 생성하는 것을 알 수 있다. 또한 여기서 컨벌루션은 컨벌루션 레이어를 나타낸다[14]. C3는 3개의 컨벌루션 레이어와 다양한 병목 현상이 있는 모듈로 구성되고, 공간 피라미드 풀링(pooling)은 네트워크의 고정 크기 제약을 제거하는 데 사용되는 풀링 계층이다. 업샘플링(Upsample)은 가장 가까운 마디에서 이전 레이어 융합을 업샘플링하는 데 사용되며 연결(Concat)은 스케일링 레이어(slicing layer)이며 이전 레이어를 스케일링하는데 사용된다. 그리고 마지막 3개의 Conv2d는 네트워크 헤드에서 사용되는 탐지 모듈이다[30].

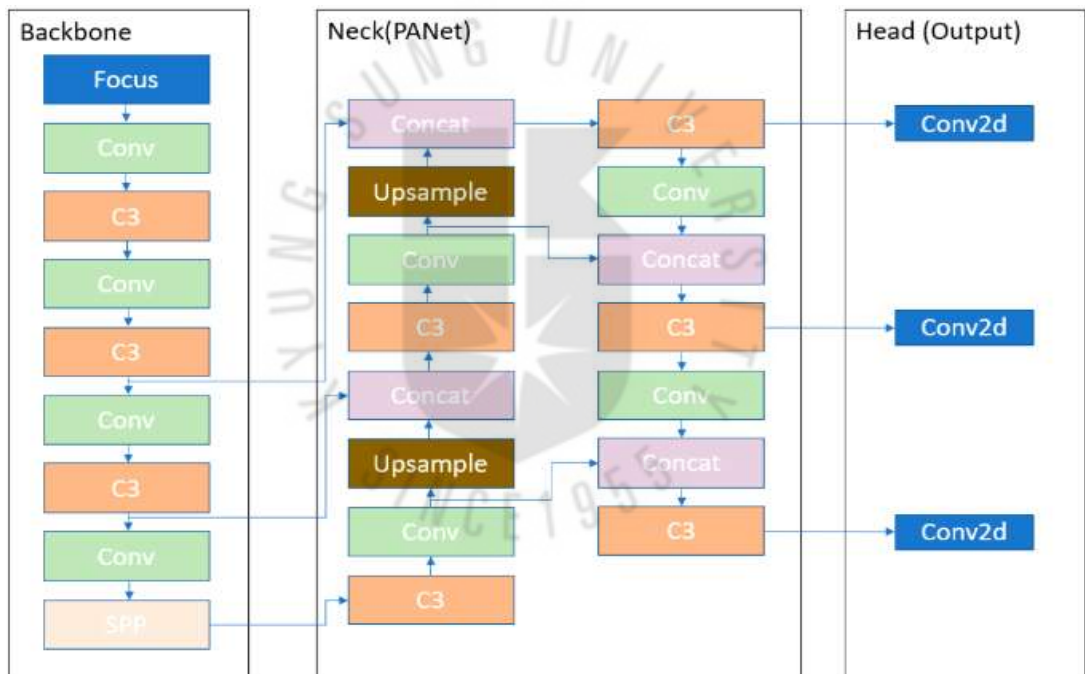


그림 8. YOLOv5의 기본적인 구조[30]

3.1.6 YOLOv7

YOLOv7은 지금까지 존재하는 object detector 부분에서 최고의 성능을 보여주고 있으며, 최근 실시간 객체탐지에 중요한 요소로 형성되고 발전되고 있다. 그림 9와 같이 YOLOv7은 5 fps~160 fps 범위에서 보면 존재하는 모든 객체 검출의 성능을 능가하며, GPU V100에서 30 fps 이상으로 알려진 모든 객체 검출 중 가장 높은 정확도인 56.8% AP(average precision)를 가지고 있다[32]. 또한, YOLOv7-E6 객체 검출은 변환(transformer) 기반의 검출인 SWINL Cascade-Mask R-CNN보다 속도면에서 509%, 정확도면에서 2% 뛰어난 성능을 보이고, 컨벌루션 기반의 검출인 ConvNeXt-XL Cascade-Mask R-CNN보다 속도면에서 551%, 정확도면에서 0.7% 뛰어난 성능을 보이는데 이것은 MS COCO 데이터셋 만으로 학습된 모델에서 나오는 결과이다[32].

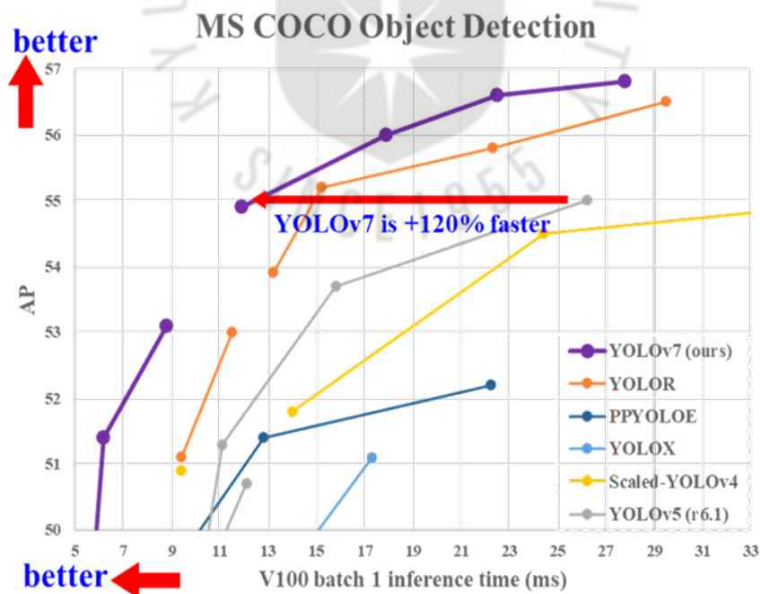


그림 9. MS COCO Object Detection[32]

3.2 SSD

SSD(Single Shot Multibox Detector)는 YOLO의 한계인 정확도 측면을 개선한 모델인데 YOLO의 문제점은 입력 이미지를 7×7 크기의 그리드에 bbox 확률(probability)의 예측을 진행하기 때문에 그리드 크기보다 작은 물체를 인식하지 못하는 문제가 있었다[32]. 그리고 신경회로망을 모두 통과하면서 컨벌루션과 풀링을 거쳐 핵심적이지 못한 정보만 남은 마지막 특징 맵만 사용하기 때문에 정확도가 떨어지는 한계가 있었다[33]. 이러한 문제를 해결하기 위해 SSD 네트워크 중간에 존재하는 컨벌루션 레이어의 특징 맵을 추출하여 인식 할 때 사용하는 방법을 제안한다. 아래 그림 10과 그림 11을 살펴보면 특징 맵을 추출하는 컨벌루션 레이어와 추출되어진 특징 맵을 보면 입력 이미지는 300×300 이며, Base network conv4_3 layer에서 $38 \times 38 \times (512)$ 크기의 특징 맵을 추출하게 된다[32]. 그 다음 base network의 conv7 레이어에서 $19 \times 19 \times (1024)$ 크기의 특징 맵을 추출하고 다음으로 보조 네트워크(auxiliary network)의 conv8_2, conv9_2, conv10_2, conv11_2 레이어에서 각각 $10 \times 10 \times (512)$, $5 \times 5 \times (256)$, $3 \times 3 \times (256)$, $1 \times 1 \times (256)$ 크기의 특징 맵을 추출한다[33]. 이와 같은 각 과정을 통해 총 6개의 척도를 가진 특징 맵을 얻을 수 있으며, 이를 통해 다양한 척도의 특징 맵을 사용하였으며, 다양한 크기의 객체를 탐지하는 것이 가능할 수 있다[33].

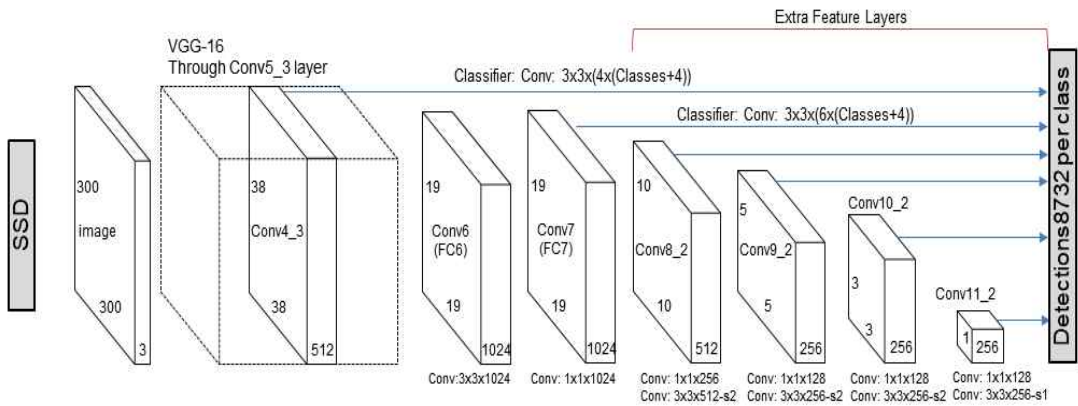


그림 10. SSD 모델의 레이어 구조[33]

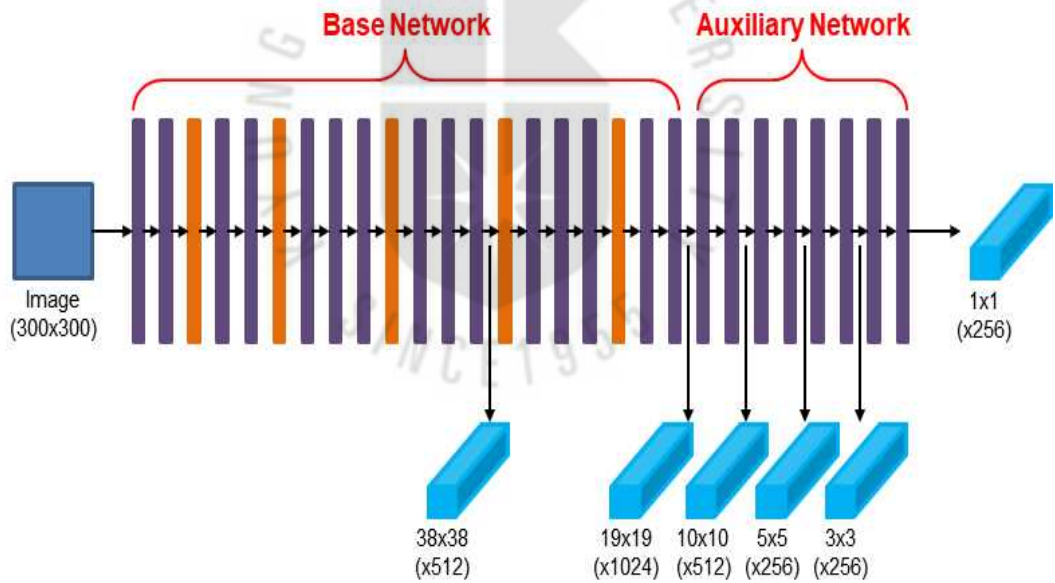


그림 11. SSD의 모델 구조[33]

3.3 EfficientNet

EfficientNet은 이미지 분류에 기반한 모델이며, 기존 CNN 모델에 비해 파라미터 연산량이 적으면서 높은 정확도를 보이는 모델로 3가지 파라미터 사이의 고정된 상수로 스케일링하는 방법이다. 이를 통하여 제한된 파라미터로 최적의 성능을 내는 모델을 만들었다. 3가지 파라미터는 레이어(depth), 필터(width), 높이(resolution)이며, 이미지 해상도가 증가할 때 수용필드의 증가가 없으면 이미지의 작은 영역을 보고 인식하기 때문에 정확도가 떨어지게 된다. 따라서 수용필드를 키우기 위해서는 모델의 레이어(Depth)를 증가시켜야 하고 투입 이미지의 해상도를 높이(Resolution)는 만큼 패턴에 대해 정밀한 판별을 위해 필터(Width) 수도 증가시킨다. 또한 그림 12와 같이 필터(Width) 수와 레이어(Depth)를 일정 수준 이상으로 늘려도 성능 향상은 적으며, 이미지의 해상도가 높은 경우 더 많은 이미지들을 추상화 시킬 수 있다[35].

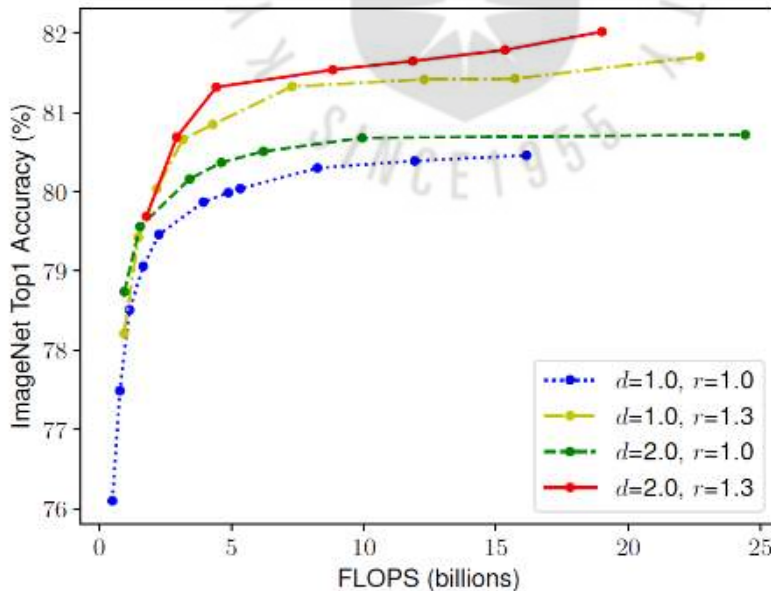


그림 12. 레이어 깊이에 따른 성능 변화

Ⅲ. 제안하는 이상행동 감지 딥러닝

1. 인공지능 개요

AI 기술을 통한 제품생산과 서비스 개발에 필요한 AI 인프라를 지원함으로써 누구나 필요한 자료와 정보를 손쉽게 획득할 수 있는 AI 통합 플랫폼이다. 과학기술정보통신부와 한국지능정보사회진흥원의 지능정보산업 인프라조성 사업의 일환으로 구축되었으며, AI 학습용 데이터와 국내·외 기관과 기업이 보유한 데이터를 구축 및 공개하는 사업을 추진중에 있다. 현재 한국어, 영상이미지, 헬스케어, 재난 안전 환경, 농축수산, 교통물류의 6개 분야로 나누어 구축되어 있다.

AI는 인간의 삶과 질적 향상 그리고 지속적인 경제 성장, 생산성과 혁신을 가속화하는 잠재력을 보유하고 있다. 특히 코로나19로 인해 우리 삶 전반에 걸쳐 많은 변화가 있었으며, 특히 비대면과 마스크 착용 등 코로나19 이전에는 볼 수 없었던 생활 패러다임이 나타나게 되었고, 이는 AI가 우리 생활에 접목되어 다양한 분야에 활용을 가속화 되었다[40].

AI의 이러한 다양한 장점과 기술 개발에 있어서 수 많은 혜택을 가져 올 수 있지만 범죄 등으로 악용될 때 큰 재앙으로 돌아올 수 있다. 따라서 AI의 위험요소를 확인하고 통제하며 장점을 활용해 여러 혜택을 얻기 위해서는 AI로 인해 발생하는 여러 상황을 지속적 확인과, 신뢰를 형성되는 것이 중요할 것이다[41].

2. AI의 국내 환경

한국정보화진흥원(NIA)에서는 AI 국가정책 발굴 가이드라인[42]에 따라 공공분야에 미칠 AI의 영향을 예측하고 국내 AI 정책 방향을 구체화하기 위해 전문가 자문과 함께, AI의 원칙과 정책 방향의 포괄적인 내용을 우리나라 수준에 맞추어 구체적인 내용으로 발전시켰으며 신뢰 가능한 AI를 구현하기 위해 6가지 정책과 세부 시행계획을 도출하였다[42]. 또한 신뢰 가능한 AI 구현을 위한 우리나라의 수준을 진단하였으며, 신뢰할 수 있는 AI 시스템을 개발하기 위해서는 신뢰 있는 AI 지식 공유 플랫폼이 필요하였다. 이러한 AI 생태계 변화와 분석 그리고 AI 활동 주체와 이해관계자가 AI 지식의 발전, 그리고 주변 공유를 통하여 개인에 대한 정보보호, 지적 재산권 침해 방지 및 다양한 권리를 존중하면서 AI 지식에 대해 다양한 논의가 이루어졌다[42]. 그리고 데이터 생성, 관리, 소유권에 대한 긴밀한 협의가 필요한데 공공데이터 등 공개 데이터의 경우 공개 시점 대비 현재 기술에 맞는 양과 질의 데이터가 생성과 공개가 필요하며, 정부 기관은 AI 환경의 확산을 위해 기술을 가지고 있는 기업과 기술이 필요로 하는 기업을 연결해 주는 역할이 필요할 것이다. 또한 데이터를 생산하는 소비자에게도 생산하는 데이터에 대한 소유권과 이익을 공유할 수 있는 제도와 생태계 조성이 필요하다[42].

3. AI 학습용 데이터 구축

AI 학습용 데이터란 머신러닝, 딥러닝 등 AI 모델 학습을 위해 활용되는 데이터를 총칭하는 것이며[42], 원본 데이터와 원본 데이터에 활용 목적에 따라 표시 작업을 한 라벨링 데이터 모두를 AI 학습용 데이터라 한다[42]. 원본 데이터는 하나지만 라벨링 데이터는 사용 목적에 따라 다양한 형식으로 가공하고 AI 학습용 데이터는 그림 13과 같이 목적에 따라 학습 데이터, 검증 데이터, 평가 데이터로 구분하고 있으며 일반적으로 전체 데이터에서 학습, 검증, 시험평가용으로 각각 70%, 15%, 15%로 구성한다[40].

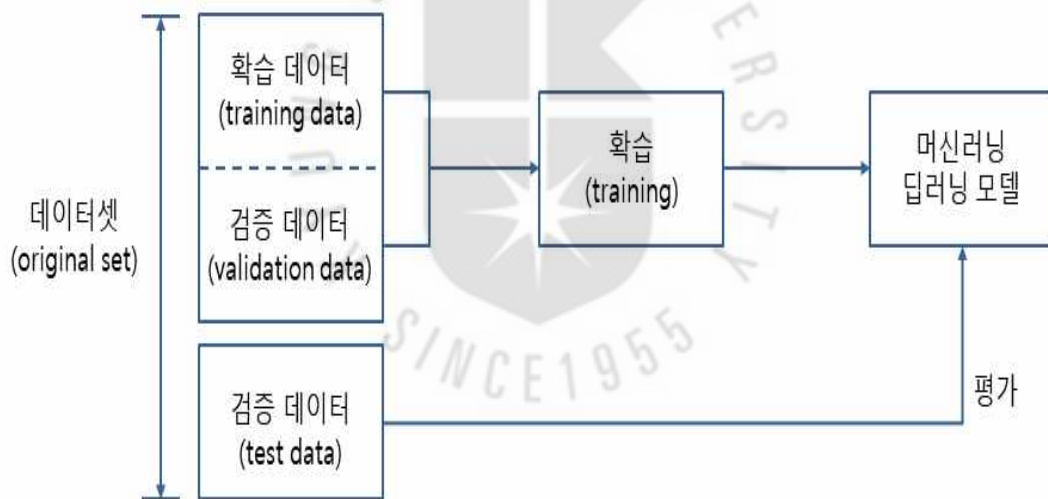


그림 13. 목적에 따른 AI 학습용 데이터 활용 체계[42]

3.1 AI 학습용 데이터 구축과 서비스 출시 절차

AI 학습용 데이터 수집 및 분석 서비스 출시 과정은 그림 14와 같이 4단계로 나누어 볼 수 있다.

데이터 수집의 1단계는 AI 모델생성을 위한 데이터 수집 및 제작을 통해 학습용 데이터를 구축하는 과정이 된다.

2단계는 데이터 가공 단계인데 데이터의 종류에 속성과 표시 작업이 이루어지는 단계가 된다[42,43]. 가공과 라벨링 단계에서는 학습 목적에 맞춰 나열된 텍스트에 속성값을 부여하는 방식으로 라벨링 하는 텍스트가 있고, 사진 속의 특정 객체를 bbox로 구분하거나 이미지 자체를 분류와 분할을 통해 라벨링 하는 이미지가 있다[42]. 그리고 이미지와 동일하게 bbox로 구분 또는 분할을 하며 영상 GT 제작 도구를 이용해 이미지 라벨링 하는 영상이 있다[39,42,43].

3단계는 AI 모델생성 단계로 가공된 데이터를 학습을 진행하고 여러 문제가 발생했을 시 해당 모델의 수정을 통해 최종 모델을 생성하는 단계이다. AI 모델은 응용 서비스에 부합하도록 다양한 형태를 가진다. 일반적으로 컨벌루션 레이어, 다운 샘플링을 하는 풀링 레이어, 신경망으로 구성되는 풀 레이어로 구성된다. AI 학습을 할 때는 구축된 GT와 모델의 출력을 비교하여 오차가 발생하면 이를 활용하여 각 레이어의 계수를 갱신하여 오차를 줄인다.

사용자에게 제공 가능한 정확도가 나오는 최종 단계가 되면 앱 개발을 통한 서비스 출시와 보완 업 데이터를 하는 단계가 마지막 단계가 된다[42,43].

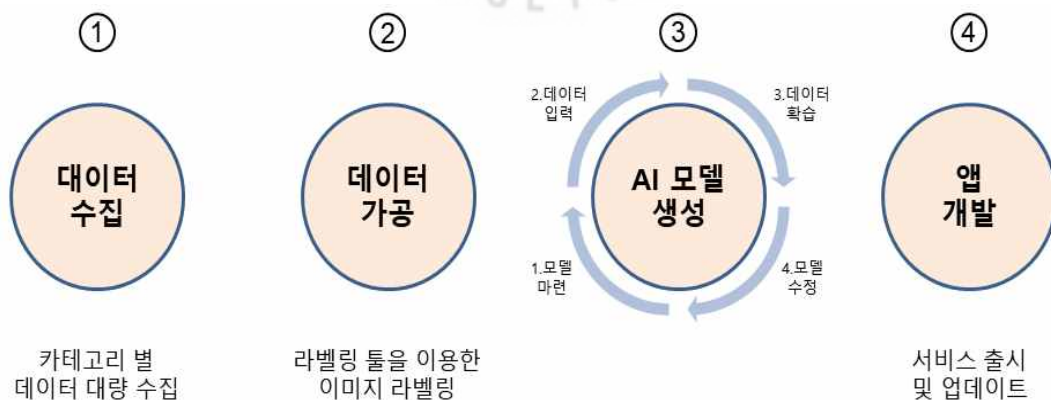


그림 14. AI 학습용 데이터 수집 및 서비스 출시 단계[42]

3.2 AI 학습용 데이터 구축 방법

학습을 위한 데이터 구축에는 여러 방법이 있으며 주변 여건과 구축 비용을 고려하여 최적화된 방식으로 구축한다[42]. 그림 15와 같은 데이터 수집 단계로 구성되어 있으며 데이터가 존재하지 않는 경우 클라우드소싱 또는 데이터 합성을 통해 새로운 데이터를 만들고, 기존 데이터는 보완을 진행하며 통합 및 재가공을 통해 필요 데이터를 획득한다[42,43].

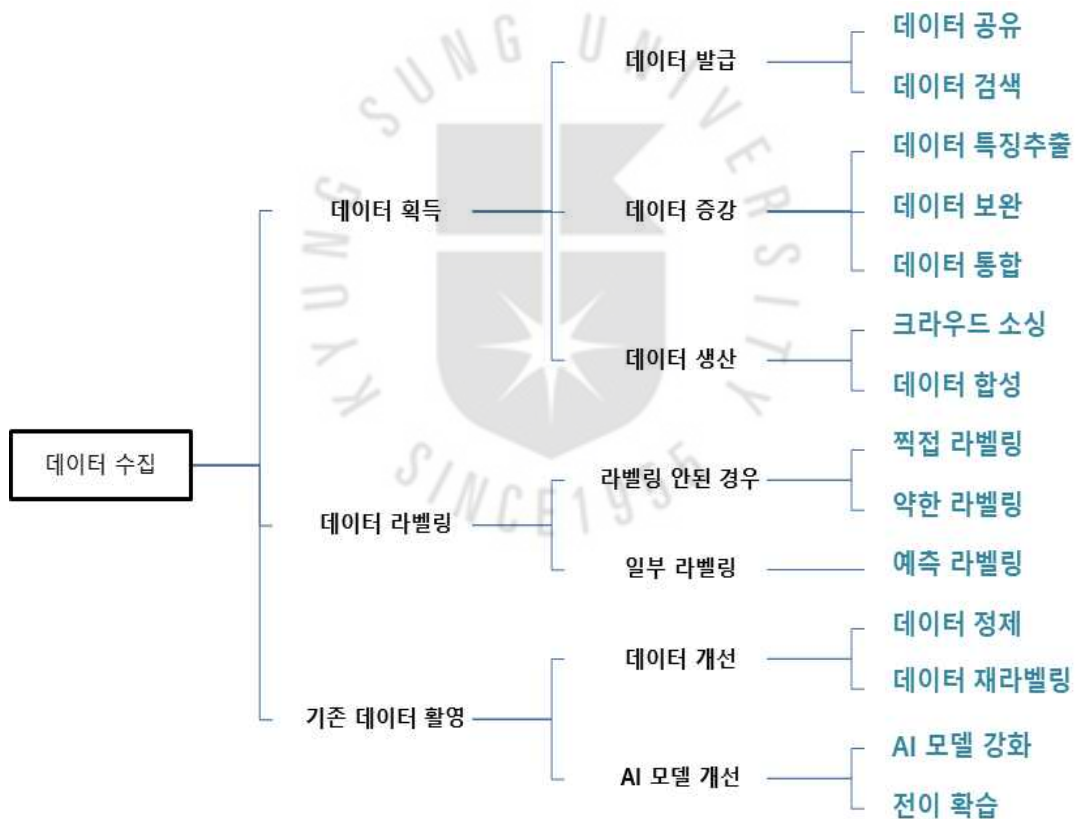


그림 15. AI 학습용 데이터 구축 방법[39]

AI 학습용 데이터 구축 방법[42]은 데이터 획득, 레이블링, 데이터 촬영으로 구성된다.

데이터 획득(discovery)은 데이터가 없는 경우 클라우드 소싱이나 데이터 합성을 통하여 신규 데이터를 만들고, 기존 데이터가 있는 경우는 기존 데이터를 보완, 통합하거나 재가공하여 데이터를 획득한다.

데이터 획득은 세분하여 데이터 발굴, 증강, 생산으로 구분된다. 데이터 발굴은 데이터 공유, 검색으로 구분된다. 데이터 공유는 데이터셋을 웹에 게시, 클라우드 기반 서비스를 기반으로 공유하고, 분석 관리 및 활용한다[42]. 데이터 검색은 다양한 메타 데이터를 사용하여 원하는 데이터셋을 쉽게 검색할 수 있도록 만든 기업 내부 및 외부 시스템을 활용한다.

데이터 증강에는 데이터 특징 추출, 보완, 통합으로 나눌 수 있다. 데이터 특징 추출은 텍스트 데이터에서 데이터의 주요 특징을 분석, 잠재 의미를 새롭게 파생하여 학습용 데이터셋을 획득할 수 있도록 한다. 데이터 보완은 기존 데이터셋의 누락된 부분에 관련성이 높은 데이터를 추가하는 등 기존 데이터셋을 풍부하게 보완한다. 데이터 통합은 기존 데이터셋에 새로 획득한 데이터셋을 더하여 데이터셋 자체를 확장한다[42].

데이터 생산은 클라우드 소싱과 데이터 합성으로 통하여 구현할 수 있다. 클라우드 소싱은 사람을 활용하여 전체적인 데이터셋을 만들기 위하여 필요한 작업 즉, 데이터 수집 및 전처리 등을 제공한다. 데이터 합성은 자동 기술 등을 활용하여 새로운 합성 데이터를 생성하거나 기존 데이터의 누락 부분을 보완한다[42].

데이터의 라벨링은 사람이 데이터를 직접 라벨링하는 방식, 자동화 기술을 통한 대용량의 라벨링 방법을 활용 가능하다[42].

기존 데이터를 학습용 데이터셋을 활용할 수 있다. 기존 데이터의 편향, 노이즈 제거, 재 라벨링 등 품질 향상을 통하여 기존 데이터를 정제하거나 이미 학습된 AI 모델을 재학습시키는 방식으로 AI 모델을 개선할 수 있다[42].

4. 팽창 컨벌루션을 적용한 YOLOv5

팽창 컨벌루션의 수행 과정은 다음 그림 16과 같다. 3x3 컨벌루션과 같은 계산량으로 보다 넓은 범위에 대해서 컨벌루션을 수행하고 특징맵을 생성할 수 있다.

본 논문에서는 도시철도 역사의 객체 즉 사람이 대체로 작은 크기이며, 색상 등의 분포가 밀집도가 낮기 때문에 팽창 컨벌루션을 적용함으로써 YOLOv5 n 버전의 성능을 개선하고자 한다.

팽창 컨벌루션을 적용하면 수용 영역(receptive field)이 커지는 효과를 얻을 수 있다. 이는 하이퍼 파라미터를 늘리지 않으면서 큰 수용 영역을 획득할 수 있는 장점이 있다. 또한 팽창 계수를 조정하여 다양한 크기(scale)에 대한 조정이 가능하다.

일반적인 컨벌루션은 풀링 즉 다운 샘플링을 한 후 업샘플링을 하게 되면 해상도가 떨어지게 된다. 팽창 컨벌루션은 수용 영역을 크게 함으로써 상대적으로 해상도를 개선할 수 있다[43].

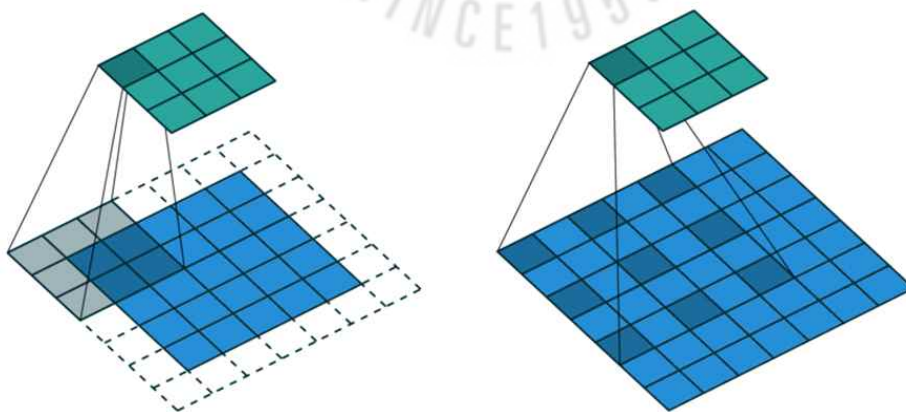


그림 16. 3x3 컨벌루션과 5x5 팽창 컨벌루션

팽창 컨볼루션은 연속된 요소 사이에 공간을 배치하여 커널(kernel)을 확장하는 방법이다. 그것은 컨볼루션과 같지만 픽셀을 건너뛰는 것으로 구성되어 있다. 그림 17은 팽창 컨볼루션을 적용한 결과로 객체를 다운샘플링 후 업샘플링을 하게 되면 해상도가 현저히 떨어지는 결과를 볼 수 있다. 하지만 아래 경로와 같이 팽창 컨볼루션을 하게 되면 밀도가 찬 이미지로 복원하게 되는 결과를 확인할 수 있다[43].

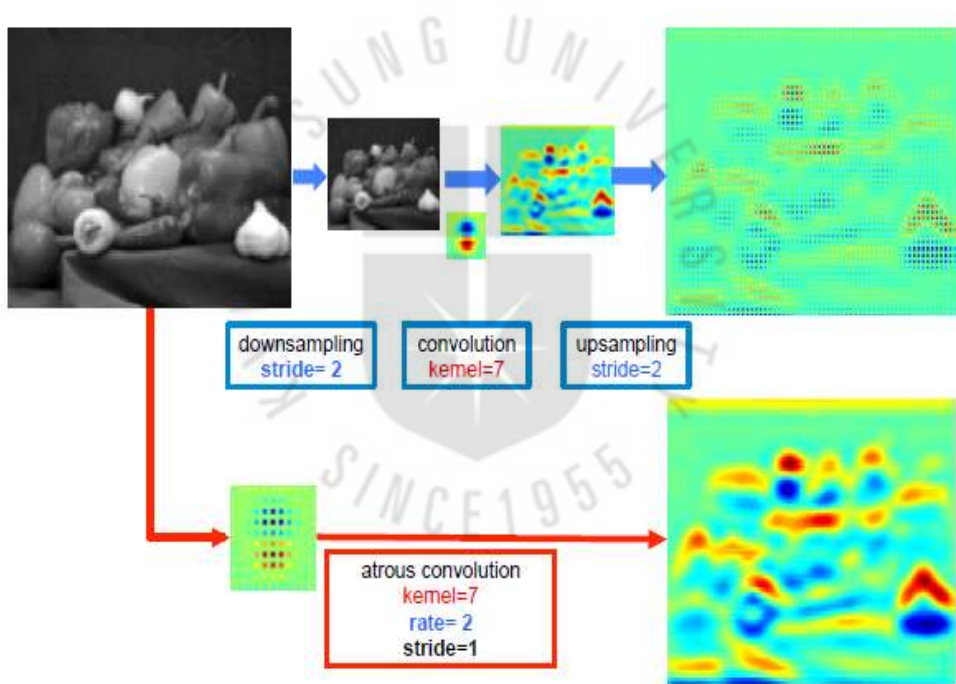


그림 17 팽창 컨볼루션을 적용한 결과 예[43]

IV. 실험 및 결과

1. 실험 환경

1.1 학습데이터 생성

도시철도 이상행동 자동 감지 시스템 구현을 위하여 이상행동이 포함된 영상 데이터가 필요하다. 본 논문에서는 딥러닝 학습을 위하여 한국정보화진흥원에서 구축한 AI Hub의 도시철도 이상행동 데이터를 활용한다. 도시철도 역사에 설치된 CCTV 카메라로부터 수집한 영상을 직접 수집하여 학습 데이터로 활용하는 것이 바람직하지만 영상 개인정보 침해 우려가 있다. 개인정보보호법 제25조 1항에 따라 정부기관 및 공공기관에서 CCTV 영상정보를 수집과 저장하는 행위에 대해 엄격히 제한되고 도시철도 기관 또한 영상정보 제공에 대한 소극적이기 때문에 연구 목적으로 활용하기에는 자유롭지 못하다.

한국정보화진흥원의 AI Hub를 통해 제공되는 데이터셋은 개인정보 활용 동의를 받은 사람이 영상에 나타나기 때문에 관련 법률에 저촉되지 않고 일부 일반인에 대하여 비식별화 즉 마스킹 처리가 되어 있다.

그림 18과 같이 AI Hub는 한국어 데이터 93종, 영상이미지 데이터 78종 헬스케어 데이터 67종, 농축수산물 데이터 41종, 교통물류 데이터 46종 6개 분야 325종을 구축하여 학습용 데이터셋을 민간 및 연구기관, 학교 등에 제공하고 있다.

한편 본 연구에서는 재난 안전 환경 분야의 도시철도 이상행동 영상을 확보하고 이를 재가공하여 학습 및 시험용 데이터를 구성하였다.



그림 18. AI Hub 구축 플랫폼[5]

1.2 이상행동 학습데이터

AI Hub의 도시철도 이상행동 데이터는 다음 그림 19와 같이 대상 객체가 정상 상태로 걸어오면서 쓰러짐 행동이 발생한다. 정상상태부터 쓰러짐까지의 동작이 이상행동 폴더 하나로 구성된 GT가 제공되고 있기 때문에 객체 검출 및 분류에 부적합하다. 본 논문에서는 도시철도 이상행동 영상을 재가공하여 쓰러짐 행동만을 라벨링하여 bbox로 구성한다.



그림 19. AI Hub 도시철도 이상행동 데이터 예

본 논문에서는 이상행동 자동 감지를 위하여 2단계 실험을 수행한다. 첫 번째는 이상행동 감지를 하는데 있어, 정상상태, 쓰러짐 2 클래스를 감지하는 실험을 수행하고, 두 번째는 정상상태, 쓰러짐, 폭행, 기물파손 4 클래스를 분리 감지하는 실험을 수행한다. 도시철도 역사 내에서 다양한 위험 상황들이 있지만 주취나 질환에 의한 쓰러짐이 가장 민감한 상황이기 때문에 2 클래스 분류를 통하여 쓰러짐만을 감지하는 시스템이 필요하고, 다양한 위험 상황을 감지하기 위하여 4 클래스를 감지를 수행한다.

AI Hub의 이상행동 분류가 CNN 딥러닝 학습에는 부적절하여 재가공을 수행하였다. 이상행동 감지를 위한 재가공 라벨링을 위해서 그림 20과 같이 Roboflow 플랫폼을 활용한다. 첫 번째 2 클래스 분류는 Roboflow에 영상을 업로드 후 개별 객체 마다 bbox 처리를 하고 쓰러짐과 정상으로 구분하는 전처리를 과정을 거쳐 학습 784개, 검증 206개, 시험 세트 10개의 원천 영상과 2 클래스로 구분하여 학습한다. 두 번째 4 클래스 분류는 학습 1,677개, 검증 387개, 시험 231개로 원천 영상과 4 클래스로 구분하여 학습과 시험을 수행 한다.

본 연구를 위해 이상행동 데이터 영상을 확인 한 결과 이벤트가 이루어지는 시점을 특정하기가 어려운 문제가 있었다. 이와 같은 문제를 해결하기 위해 그림 21과 같이 이벤트 시점을 개별적으로 확인하여 이미지로 구분한다. 그림 22과 같이 사람이 쓰러진(falldown) 모습을 bbox로 가로 box 형태로 처리하였으며, 그림 21과 같이 정상상태(normal)는 세로 형태, 그림 23과 같이 쓰러짐과 정상상태가 같이 있는 경우 2종류로 구분하여 전처리 과정을 거치게 되었다. 4 클래스에 대해서도 2 클래스와 같이 bbox를 만들고 정상, 쓰러짐, 폭행 그리고 기물파손으로 분류한다.

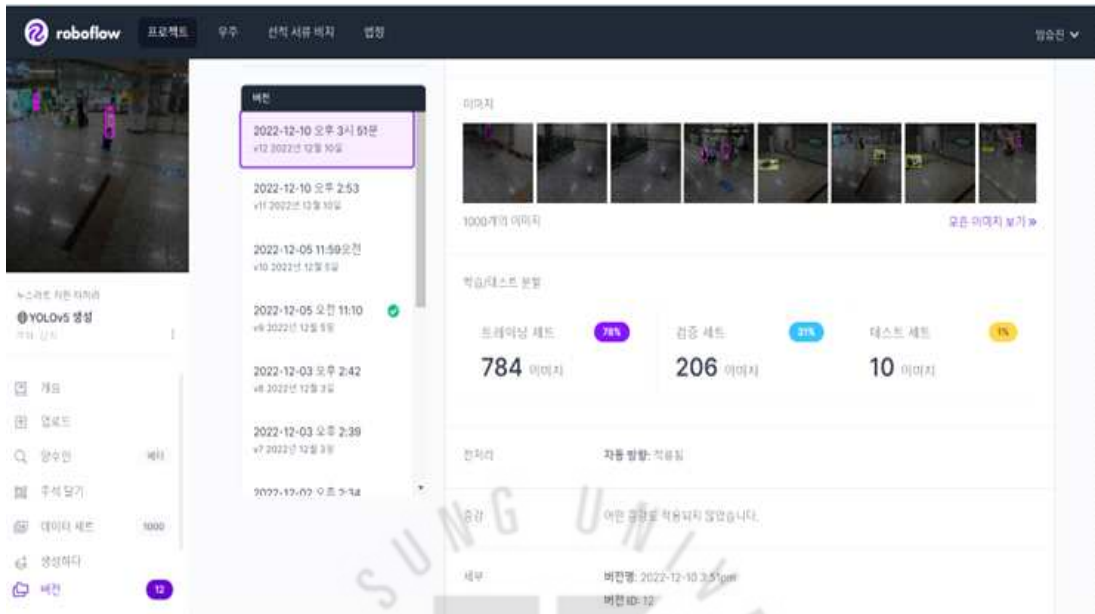


그림 20. Roboflow 플랫폼 및 가공 현황



그림 21. 정상상태 영상 라벨링



그림 22. 쓰러짐 영상 라벨링



그림 23. 쓰러짐과 정상상태 영상 라벨링 결과

1.3 YOLOv5 구조와 모델

YOLOv5 구조는 그림 24와 같이 백본(backbone), 넥(neck) 그리고 헤드(head)로 구성되며, 각각은 CSPDarkNet, PANet 그리고 YOLO를 활용한다.

YOLOv5 모델은 그림 25와 같이 n, s, m, l x 5가지의 버전을 알 수 있는데 5가지 버전 중 n이 가장 레이어가 적은 모델이며, x가 가장 레이어가 많은 모델이다. YOLOv5 n, s, m, l, x 버전의 레이어 수는 각각 157, 166, 212, 267 그리고 322 개의 레이어를 갖는다. YOLOv5의 각 버전은 n 버전이 성능 성능이 낮지만 처리 속도는 가장 빠르다. 레이어의 수에 따른 모델의 성능 차이가 발생할 수 있어 레이어 수도 함께 판단하여 사용 모델을 선정한다.



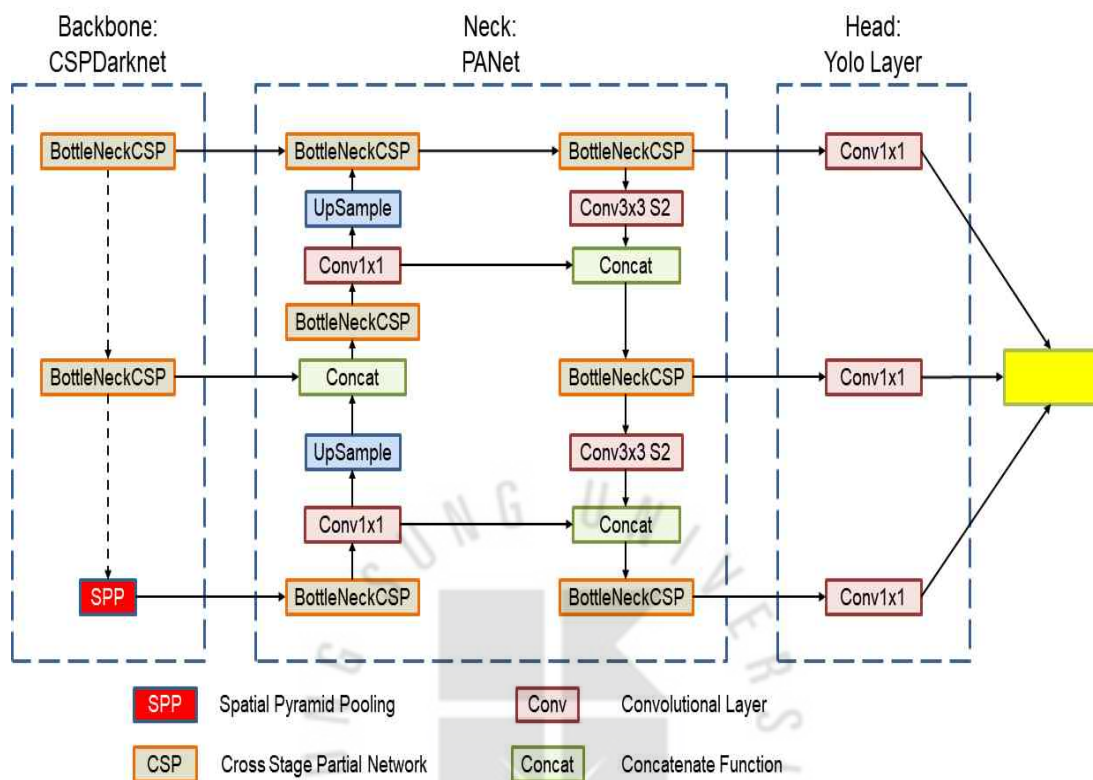


그림 24. Proposed YOLOv5 Structure

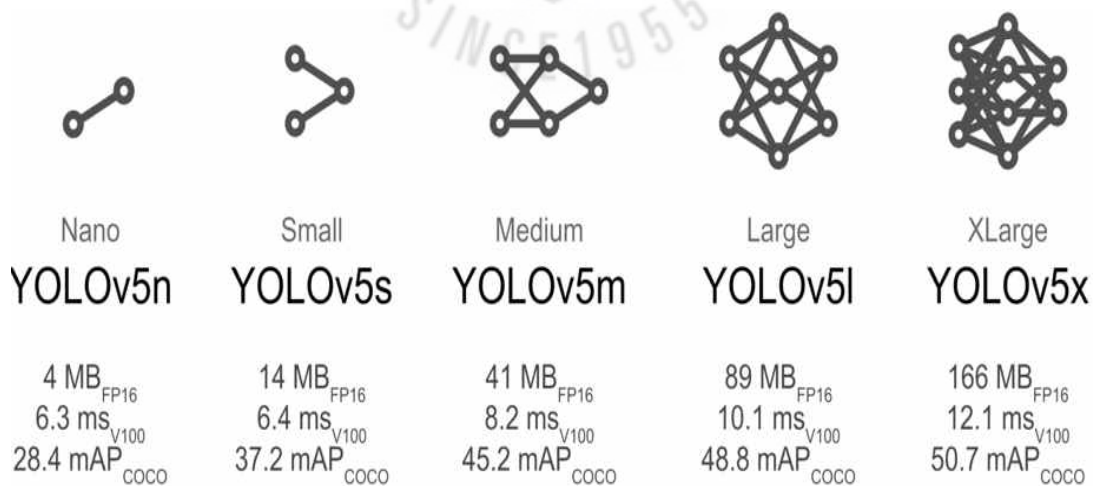


그림 25. YOLOv5 5가지 모델

2. YOLOv5 2 클래스 이상행동 감지 시뮬레이션 결과 및 고찰

2.1 YOLOv5 n버전 2 클래스 분류 학습 결과

아래 그림 26의 그래프는 AI Hub의 도시철도 이상행동 데이터셋을 재가공하여 쓰러짐 데이터셋을 구성하고 YOLOv5 n버전으로 학습한 결과이다. 재현률, 정밀도 그리고 mAP50이 수렴하는 것을 확인할 수 있다. 그림 27은 YOLOv5에 대한 혼란 행렬(confusion matrix)이다. 쓰러짐, 정상상태 검출은 각각 95%, 90%이다. 쓰러짐과 정상상태 검출 간의 상호 오감지는 각각 2%, 3%이다. 정상상태를 배경으로 오감지가 8%이다. 배경에 대하여 정상상태, 쓰러짐으로 오감지는 각각 84%, 16%이다. 이는 도시철도의 유리 등에 나타나는 사람을 정상상태로 감지한 것이다.

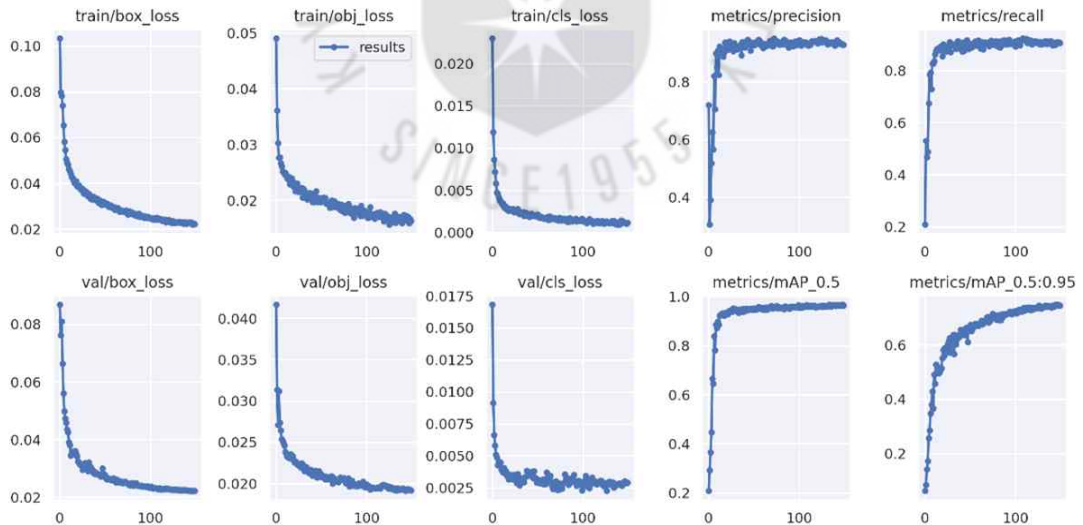


그림 26. YOLOv5 학습 그래프

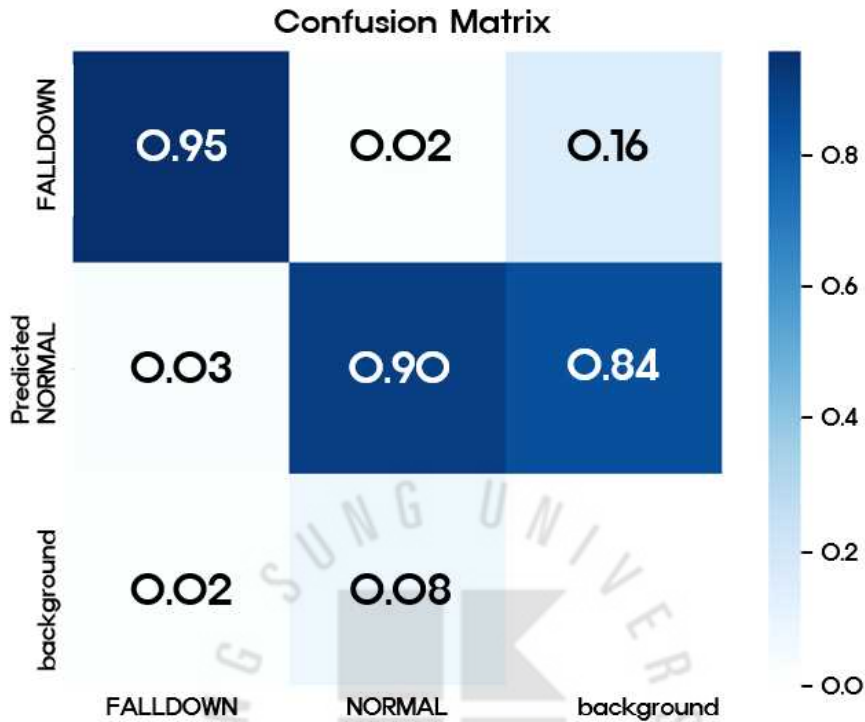


그림 27. YOLOv5 학습결과 혼란 행렬(confusion matrix)

학습 결과는 정밀도(precision), 재현률(recall), 정밀도-재현률(precision-recall) 그리고 F1-score로 나타낸다. 정밀도는 기계학습 등의 오감지에 대한 성능 평가 척도로써 다음 식과 같다.

$$P = \frac{TP}{TP + FP} \quad (1)$$

P 는 정밀도 TP 는 정상검출, FP 은 미검출이다. 정밀도가 높을수록 오감지가 작은 것이다. 정밀도는 실시간 감시를 관제센터 등에는 오감지가 많을 경우 업무의 효율을 떨어질 수 있다. 따라서 실시간 감시에서는 정밀도가 중요 척도가 될 수 있다.

재현률은 기계학습 등에서 특정 사건 또는 객체를 감지 또는 검출 성능을 평가하는 척도로써 다음 식과 같다.

$$R = \frac{TP}{TP + FN} \quad (2)$$

R은 재현률이며, FN은 미검출이다. 재현률이 1에 가까이 갈수록 검출 및 감지 성능이 우수한 것이다. 오감지가 어느 정도 발생하더라도 특정한 이벤트 또는 객체를 감지에 주요 목적을 두고 특정한 이벤트 또는 객체 검출을 사후에 조사 점검하고자 할 때 유용한 척도로써 이벤트를 기록하는 시스템에서는 중요한 척도로 활용되고 있고 있다.

F1-score는 정밀도와 재현률의 조화평균으로 정상적인 검출 성능과 오감지 성능을 동시에 나타낼 수 있는 척도(measurement)이다. 지능형 CCTV 시스템으로 관제센터를 운영할 때, 주로 많이 활용되는 척도이다.

$$F1 - Score = \frac{2PR}{P + R} \quad (3)$$

재현률 뿐만아니라 정밀도 성능이 좋아야 F1-score가 높게 나타난다. 그림 28, 29, 30 그리고 31은 YOLOv5 n버전의 정밀도, 재현률, 정밀도-재현률 그리고 F1-Score 그래프이다. 정밀도는 신뢰 수준(confidence level) 94.6%에서 모든 클래스에 대해서는 100% 이며, 재현률은 신뢰 수준 0%에서 100% 이다. 정밀도-재현률 그래프는 mAP50으로 96.5%를 나타낸다. F1-Score은 신뢰 수준 52.3%에서 92%로 나타난다.

그림 32와 33은 시험에 사용한 GT와 추정 결과이다. 정상상태와 쓰러짐을 대체로 정확하게 감지하는 것을 확인할 수 있다.

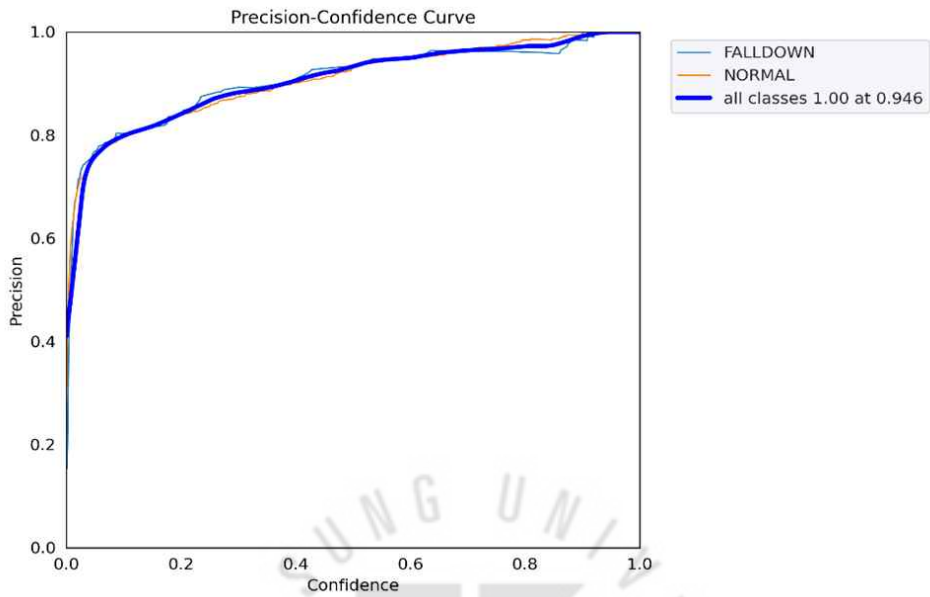


그림 28. YOLOv5 n 버전 학습결과 Precision-Confidence 그래프

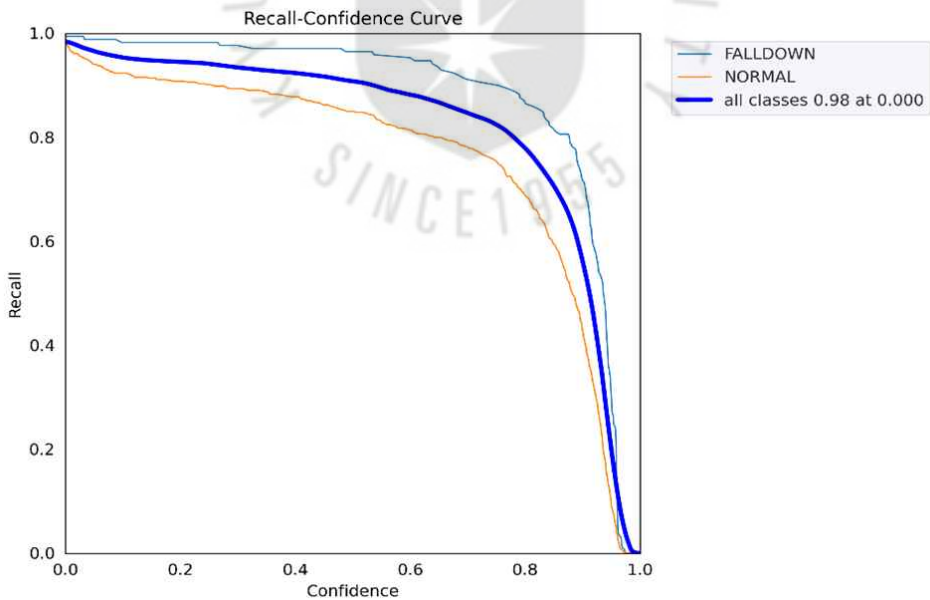


그림 29. YOLOv5 n 버전 학습결과 Precision-Recall 그래프

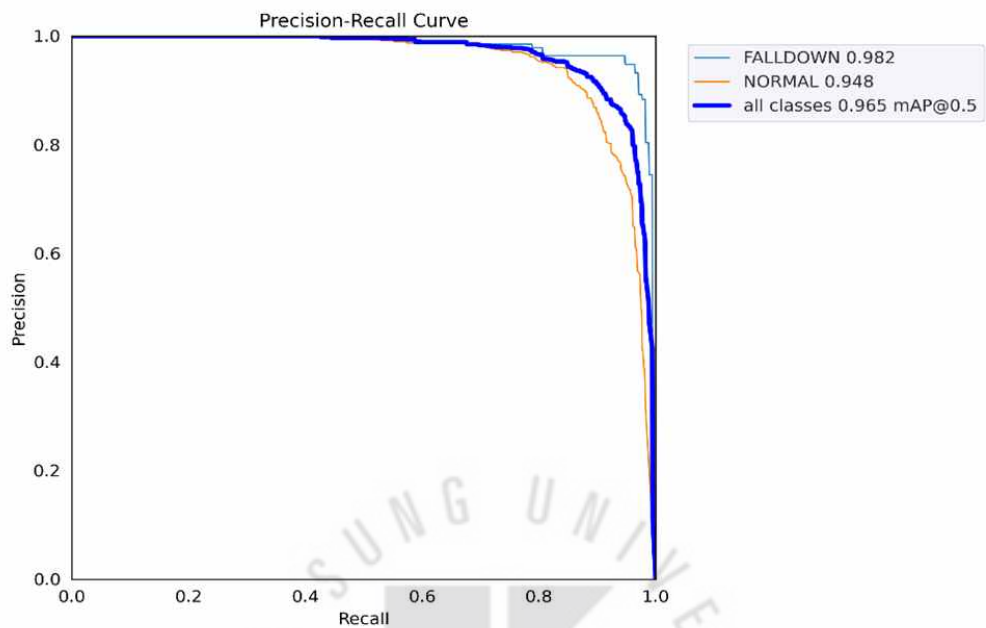


그림 30. YOLOv5 n 버전 학습결과 Precision-Recall Confidence 그래프

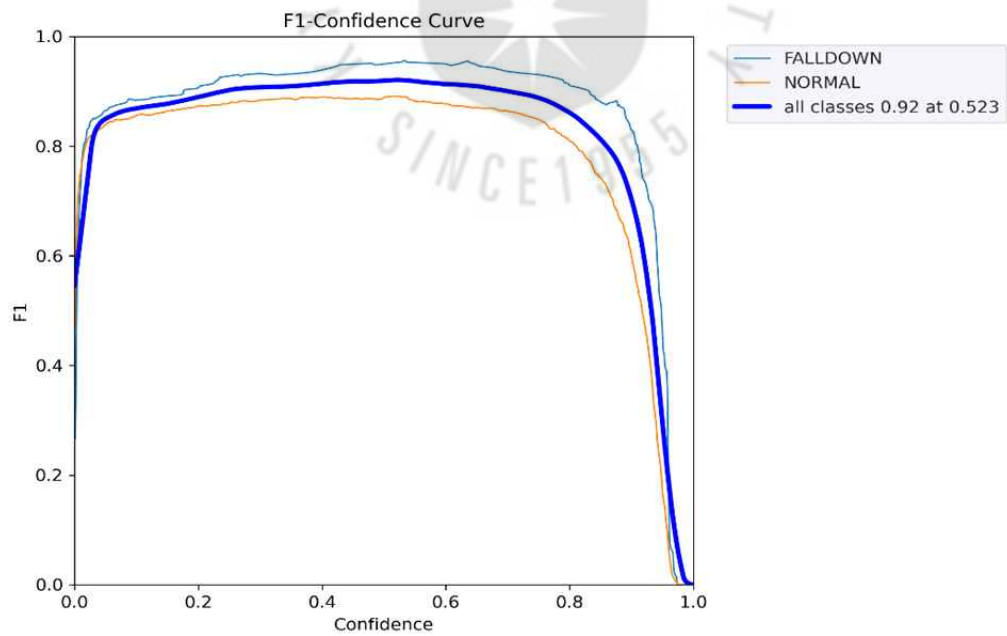


그림 31. YOLOv5 n 버전 학습결과 F1-Confidence 그래프



그림 32. YOLOv5 GT 영상 및 라벨



그림 33. YOLOv5 n버전 감지 결과

2.2 YOLOv5 s버전 2 클래스 분류 학습 결과

그림 34, 35, 36 그리고 37은 YOLOv5 s버전의 정밀도, 재현률, 정밀도-재현률 그리고 F1-Score 그래프이다. 정밀도는 신뢰 수준(confidence level) 98.2%에서 모든 클래스에 대해서는 100%이며, 재현률은 신뢰 수준 0%에서 99%이다. 정밀도-재현률 그래프는 mAP50으로 97.1%를 나타낸다. F1-Score은 신뢰 수준 47.13%에서 93%로 나타난다. 그림 38은 YOLOv5 s버전 감지 결과로써 입력 영상에 대하여 대체로 정상적인 분류를 하는 것을 확인할 수 있다.

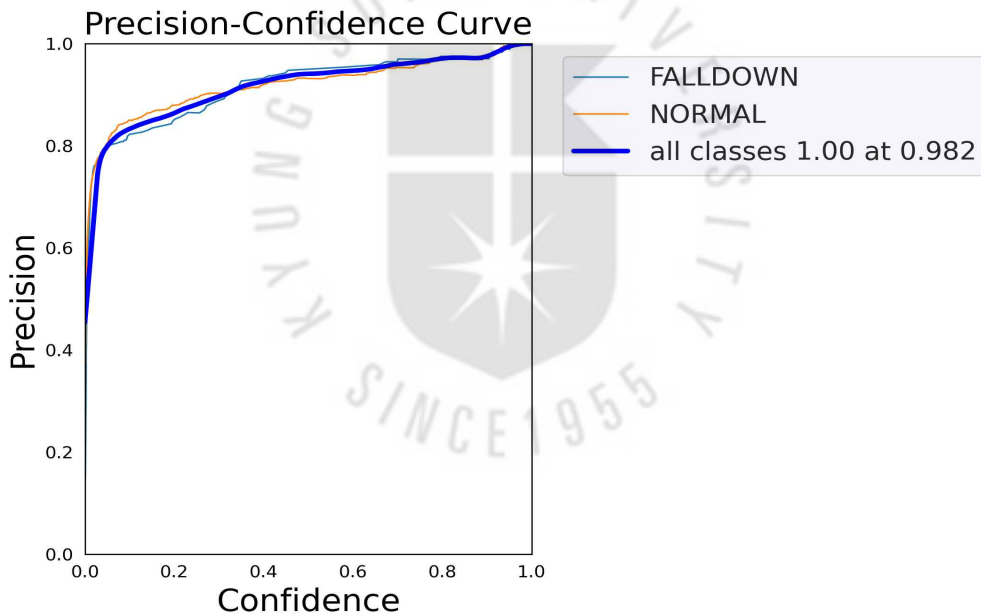


그림 34. YOLOv5 s버전 학습결과 Precision-Confidence 그래프

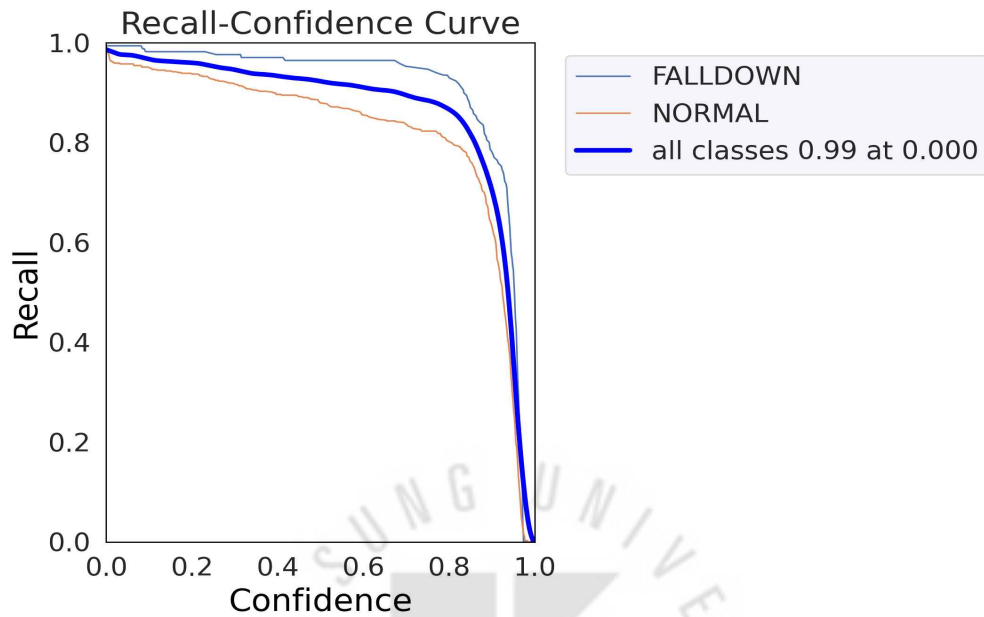


그림 35. YOLOv5 s버전 학습결과 Precision-Recall 그래프

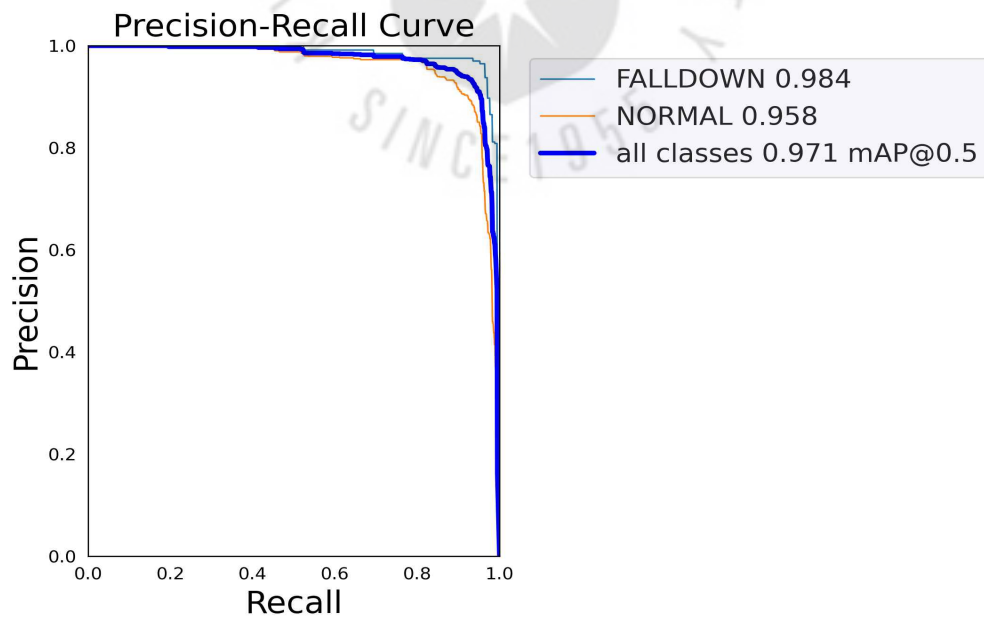


그림 36. YOLOv5 s버전 학습결과 Precision-Recall Confidence 그래프

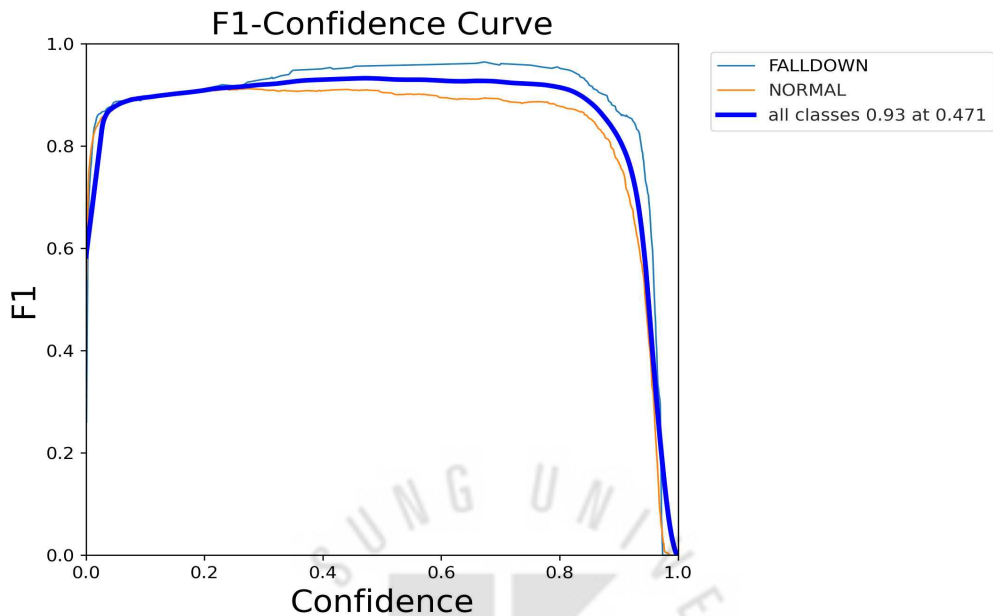


그림 37. YOLOv5 s버전 학습결과 F1-Confidence 그래프

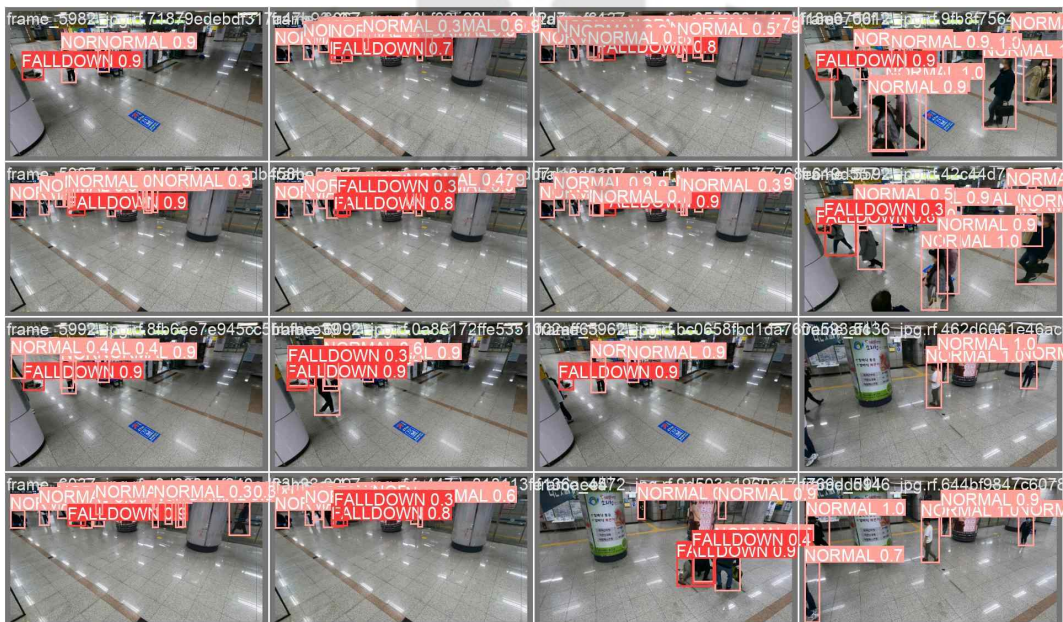


그림 38. YOLOv5 s버전 감지 결과

2.3 YOLOv5 m버전 2 클래스 분류 학습 결과

그림 39, 40, 41 그리고 42는 YOLOv5 m버전의 정밀도, 재현률, 정밀도-재현률 그리고 F1-Score 그래프이다. 정밀도는 신뢰 수준(confidence level) 99.3%에서 모든 클래스에 대해서는 100%이며, 재현률은 신뢰 수준 0%에서 98%이다. 정밀도-재현률 그래프는 mAP50으로 96.9%를 나타낸다. F1-Score은 신뢰 수준 55.7%에서 94%로 나타난다.

그림 43는 YOLOv5 m버전 감지 결과로써 입력 영상에 대하여 정상과 쓰러짐을 분류한 감지 결과이다. 대체로 정상 상황과 쓰러짐 이벤트를 분류하고 있는 것을 확인할 수 있다.

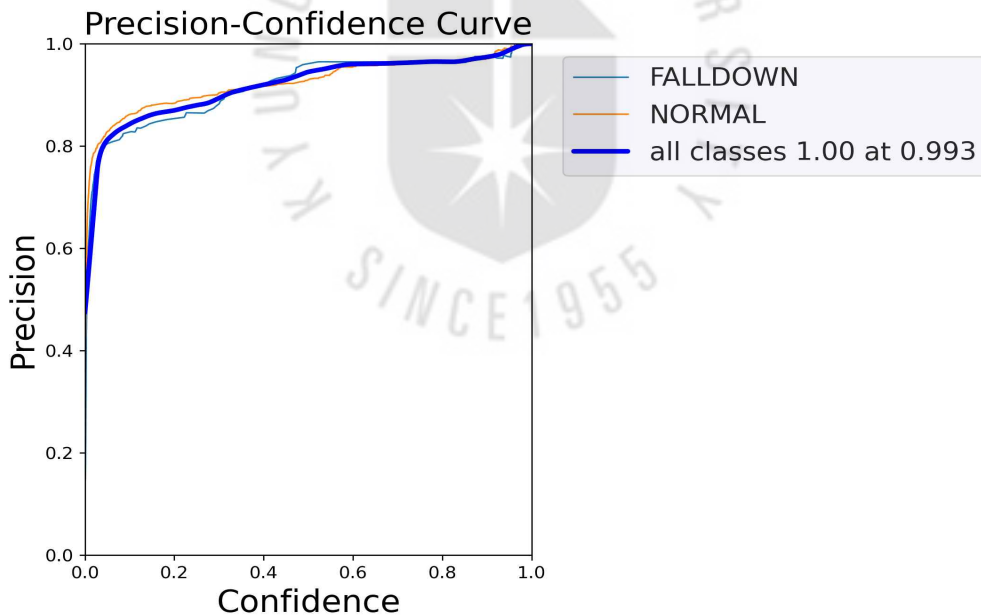


그림 39. YOLOv5 m버전 학습결과 Precision-Confidence 그래프

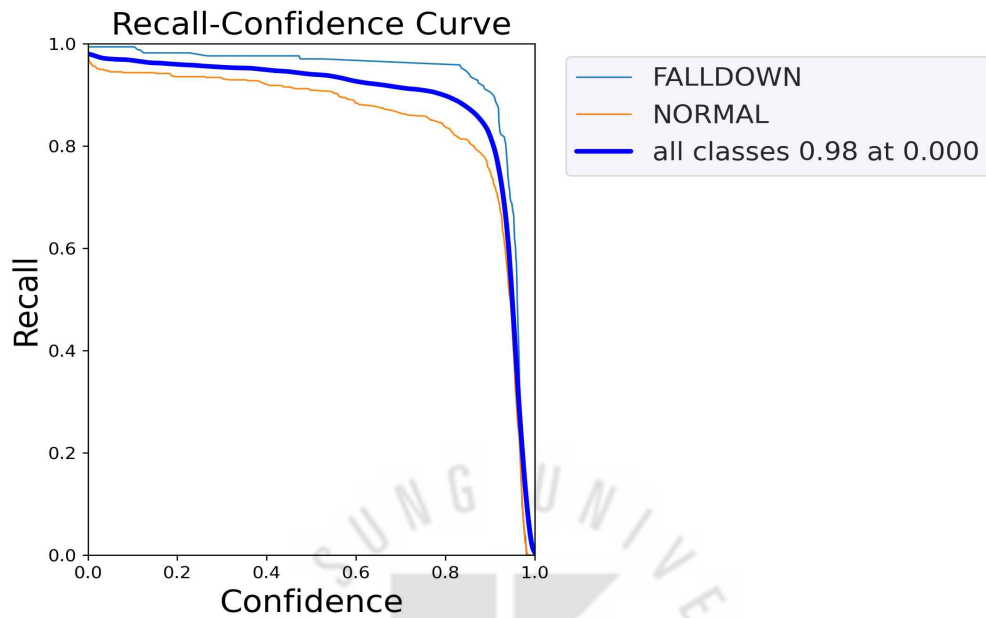


그림 40. YOLOv5 m 버전 학습결과 Precision-Recall 그래프

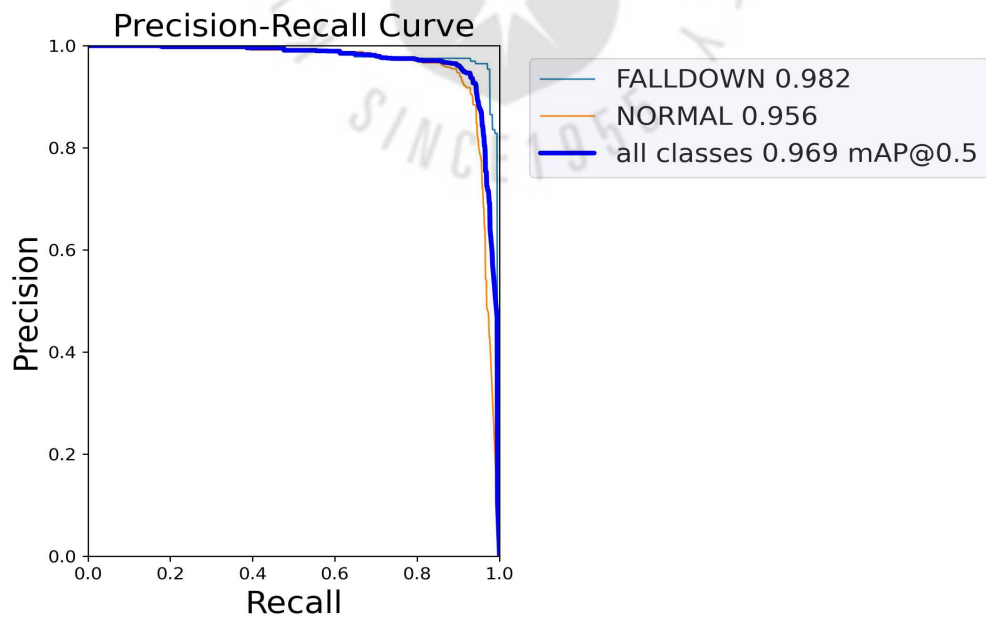


그림 41. YOLOv5 m 버전 학습결과 Precision-Recall Confidence 그래프

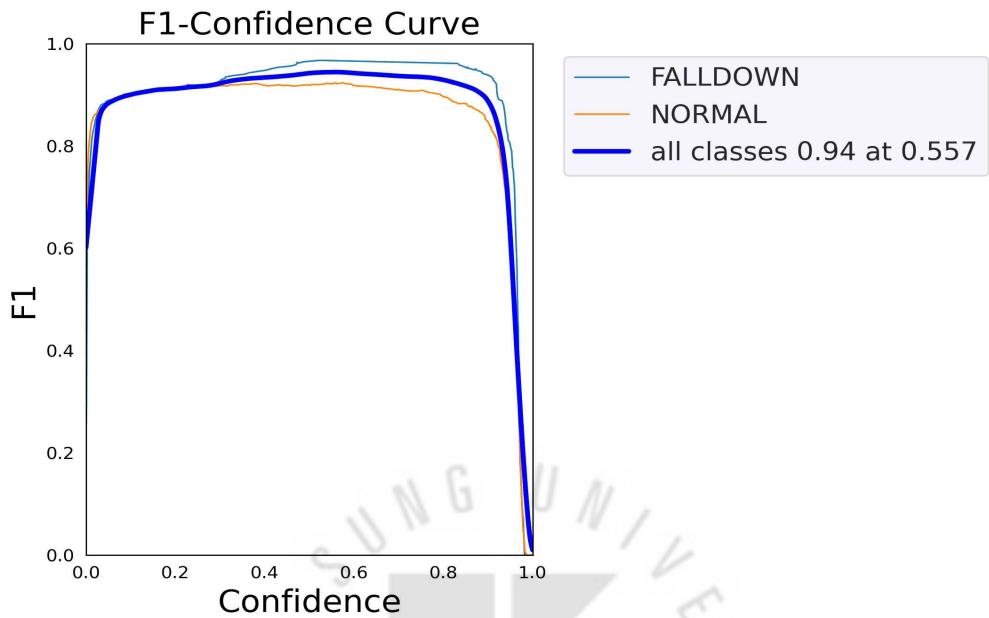


그림 42. YOLOv5 m버전 학습결과 F1-Confidence 그래프

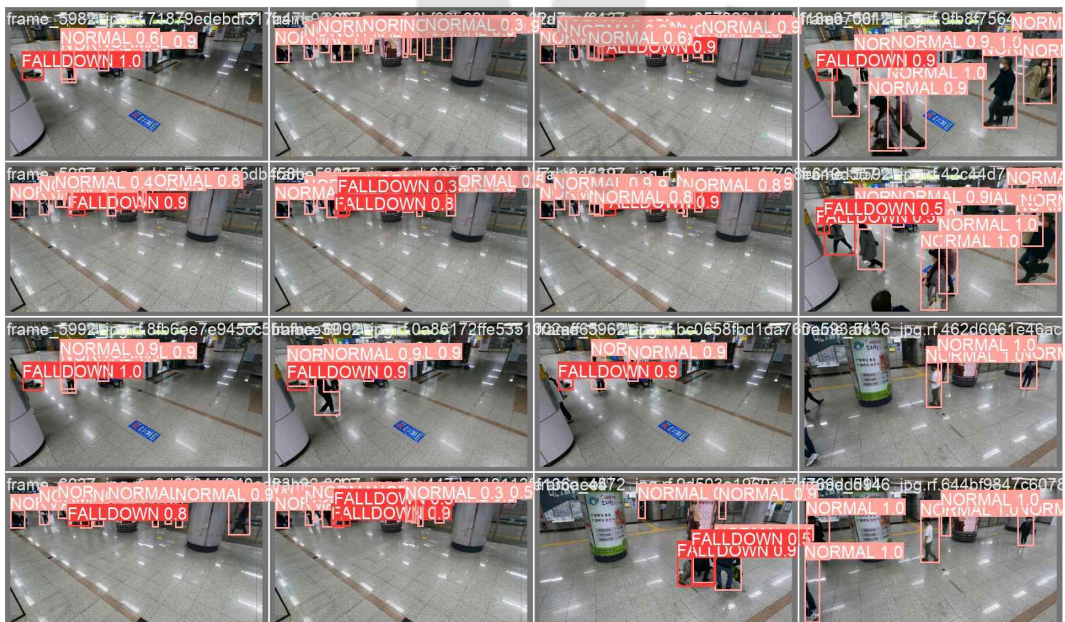


그림 43. YOLOv5 m버전 감지 결과

2.4 YOLOv5 1버전 2 클래스 분류 학습 결과

그림 44, 45, 46 그리고 47은 YOLOv5 1버전의 정밀도, 재현률, 정밀도-재현률 그리고 F1-Score 그래프이다. 정밀도는 신뢰 수준(confidence level) 98.4%에서 모든 클래스에 대해서는 100%이며, 재현률은 신뢰 수준 0%에서 99%이다. 정밀도-재현률 그래프는 mAP50으로 97.3%를 나타낸다. F1-Score은 신뢰 수준 51.5%에서 94%로 나타낸다.

그림 48는 입력 영상에 대하여 정상과 쓰러짐을 분류한 YOLOv5 1버전 감지 결과이다. 대체로 정상 상황과 쓰러짐 이벤트를 분류하고 있는 것을 확인할 수 있다.

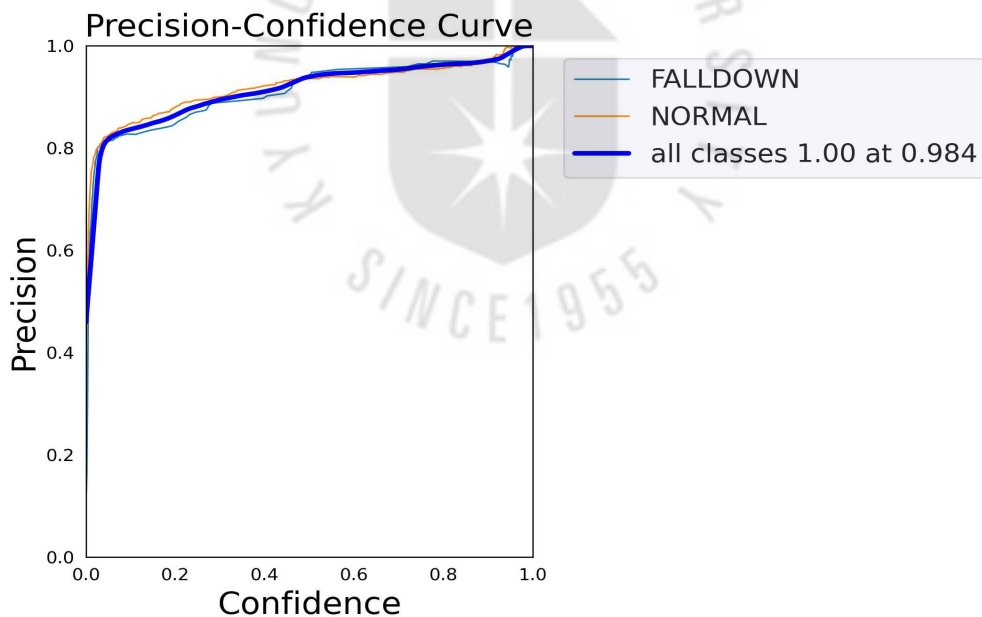


그림 44. YOLOv5 1버전 학습결과 Precision-Confidence 그래프

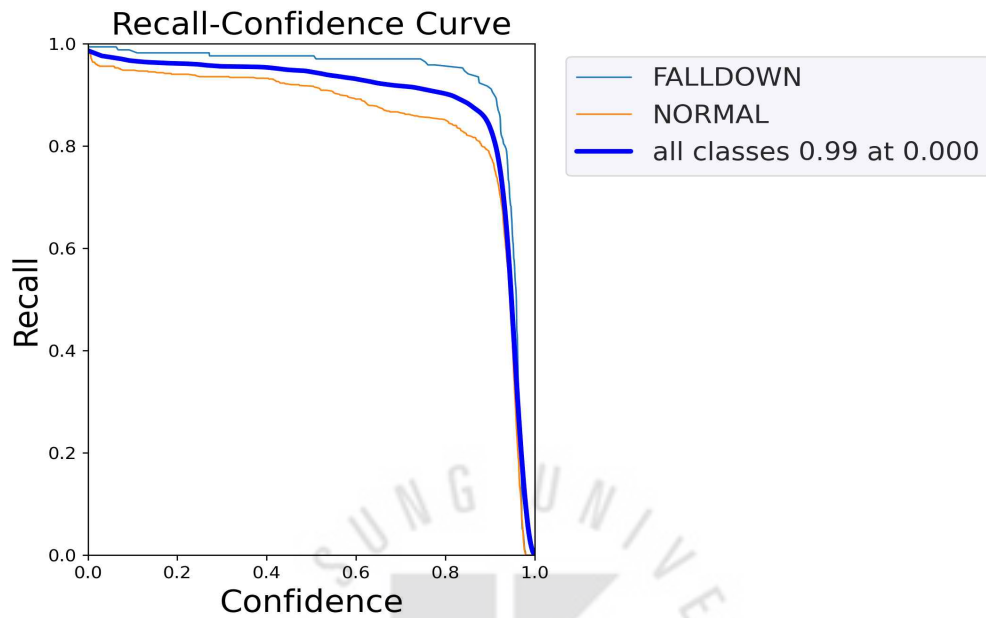


그림 45. YOLOv5 1버전 학습결과 Precision-Recall 그래프

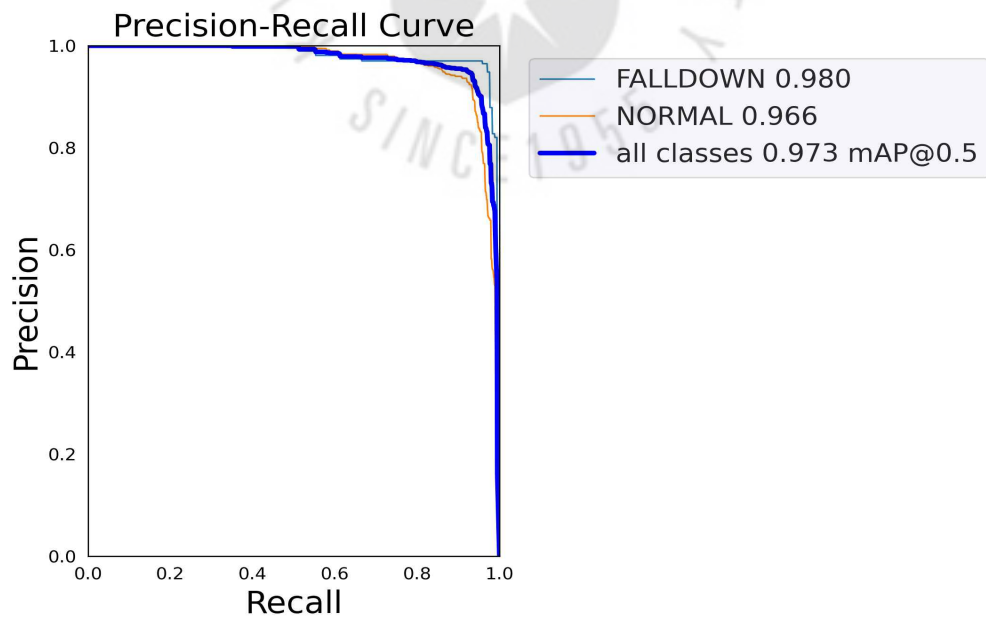


그림 46. YOLOv5 1버전 학습결과 Precision-Recall Confidence 그래프

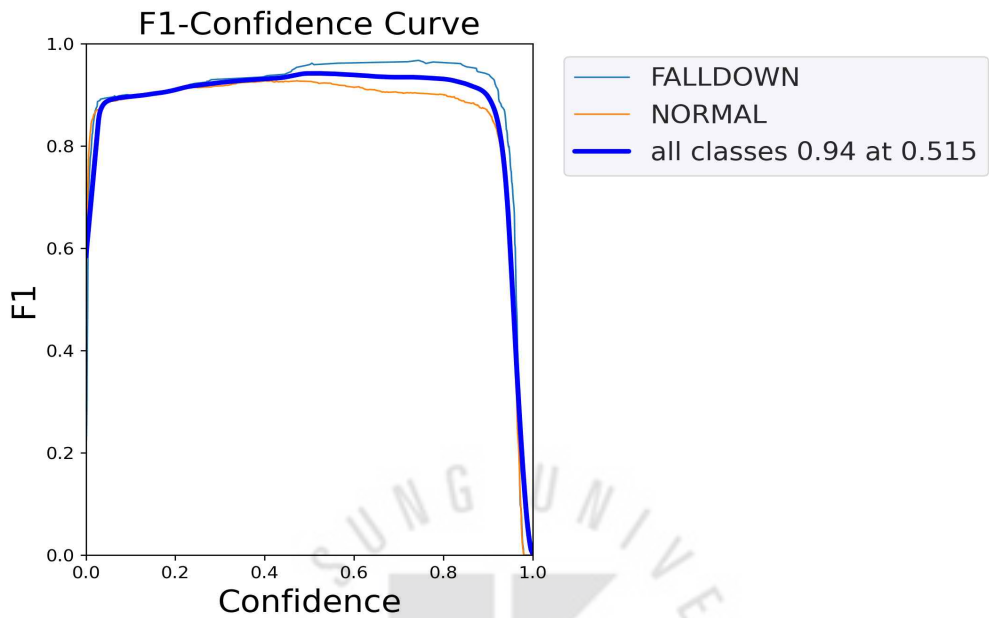


그림 47. YOLOv5 1버전 학습결과 F1-Confidence 그래프

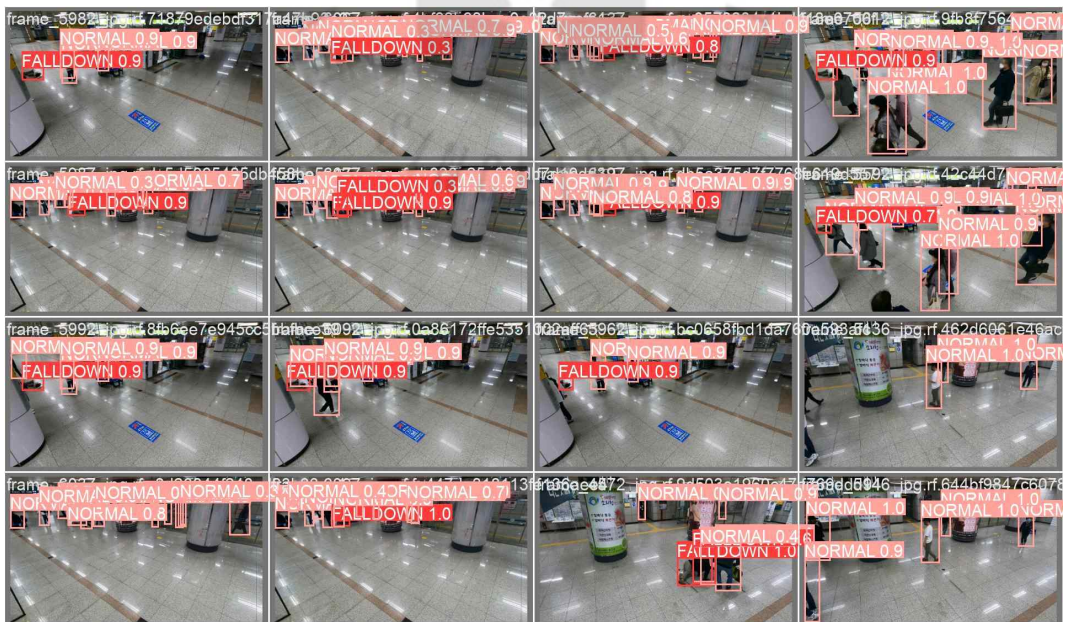


그림 48. YOLOv5 1버전 감지 결과

2.5 YOLOv5 x버전 2 클래스 분류 학습 결과

그림 49, 50, 51 그리고 52는 YOLOv5 x버전의 정밀도, 재현률, 정밀도-재현률 그리고 F1-Score 그래프이다. 정밀도는 신뢰 수준(confidence level) 99.7%에서 모든 클래스에 대해서는 100%이며, 재현률은 신뢰 수준 0%에서 99%이다. 정밀도-재현률 그래프는 mAP50으로 97.4%를 나타낸다. F1-Score은 신뢰 수준 58%에서 95%로 나타난다.

그림 53는 YOLOv5 x버전 감지 결과이다. 원거리에 대해서 이벤트에 대한 확률이 다소 낮지만 정상과 쓰러짐 이벤트를 분류하는 것을 확인할 수 있다.

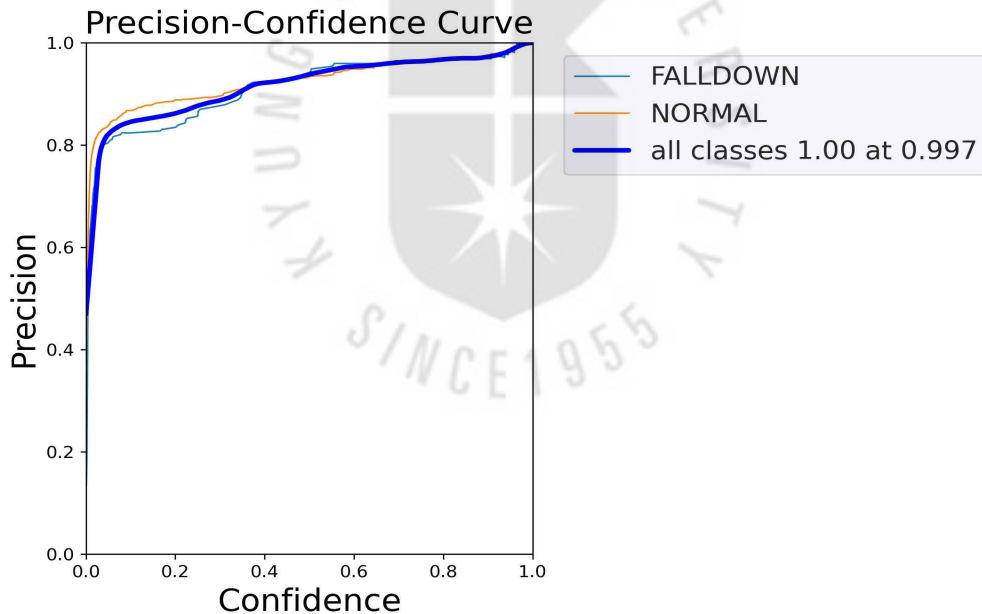


그림 49. YOLOv5 x버전 학습결과 Precision-Confidence 그래프

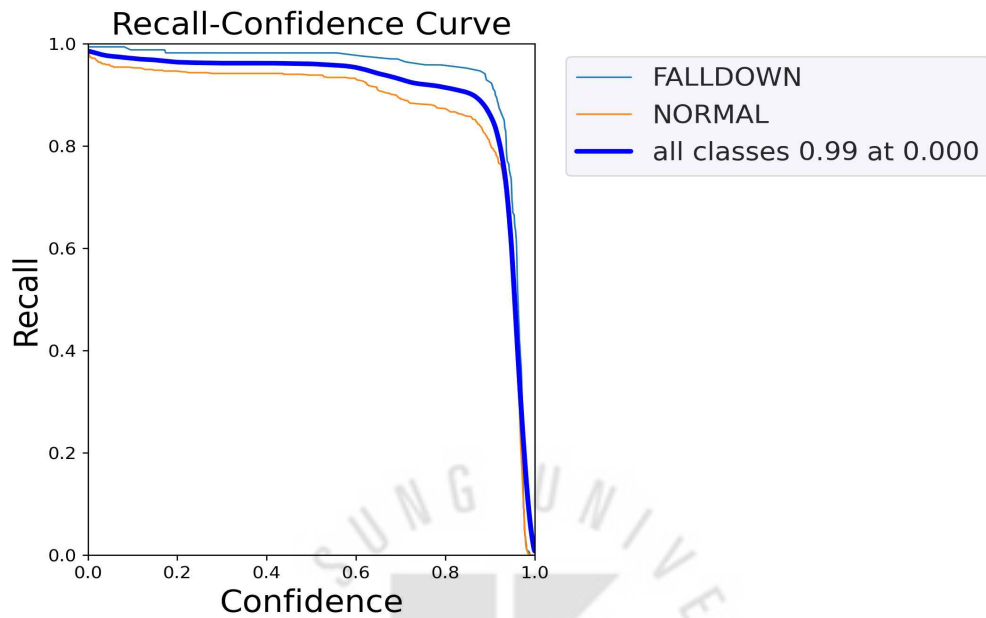


그림 50. YOLOv5 x 버전 학습결과 Precision-Recall 그래프

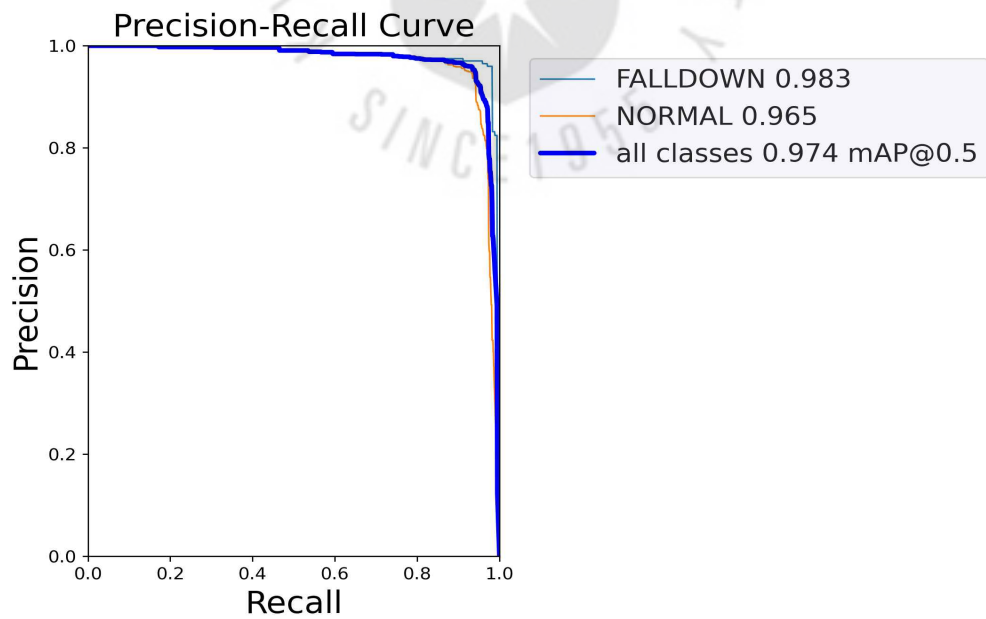


그림 51. YOLOv5 l 버전 학습결과 Precision-Recall Confidence 그래프

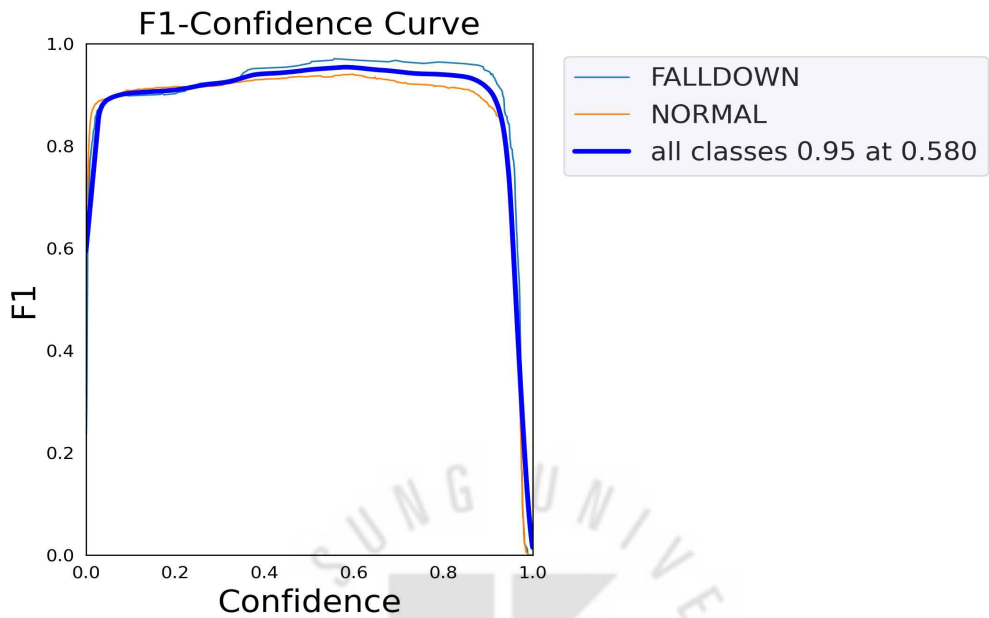


그림 52. YOLOv5 x버전 학습결과 F1-Confidence 그래프

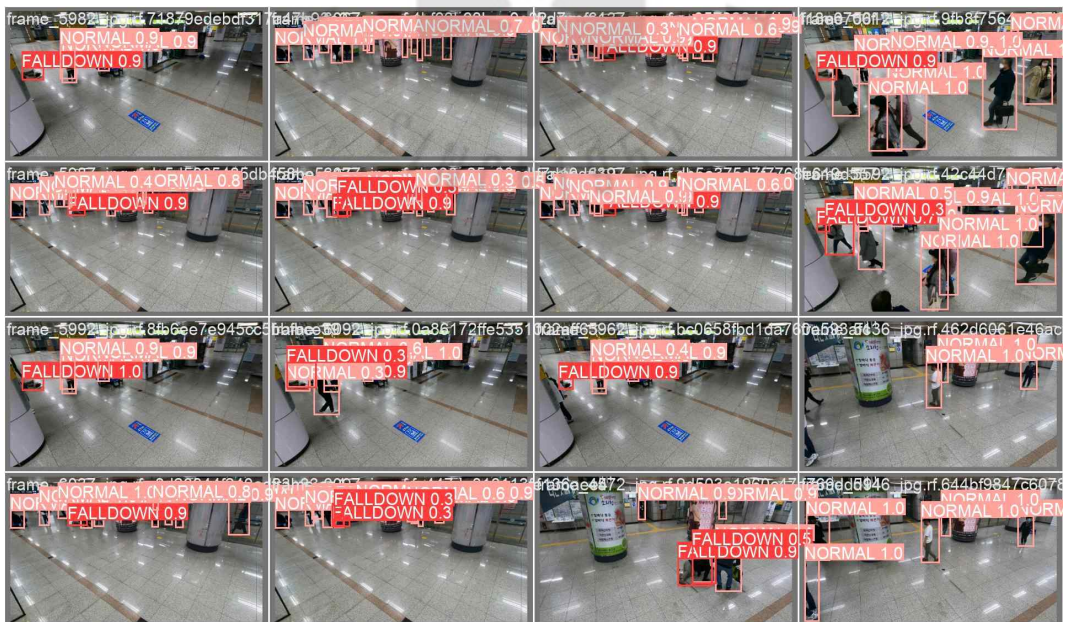


그림 53. YOLOv5 x버전 감지 결과

임베디드 및 에지 컴퓨팅에 적용하기 위한 적절한 모델 선정을 위하여 YOLOv5 n, s, m, l, x 버전에 대하여 학습 결과를 비교 분석한다.

그림 53에서 각 버전에 대하여 학습한 결과 n, s, m, l, x 버전의 mAP50 성능은 각각 96.5%, 96.7%, 96.8%, 97.1% 그리고 97.1%로 나타났다. 일반적으로 mAP50을 주로 활용하고 있어 쓰러짐 이상행동 감지에 대해서 YOLOv5의 성능 96% 이상 우수한 것으로 판단된다. n 버전과 x 버전의 mAP 성능 차이는 0.6%로 나타났다.

표 1과 그림 54에서는 mAP50-95는 각각 74.9%, 79%, 80.2%, 81.1% 그리고 81.8%로 n 버전과 x버전의 차이는 6.9% 정도 차이가 있음을 확인할 수 있다. IoU가 높을수록 성능의 차이가 있다.

그림 55는 YOLOv5의 학습에 소요되는 시간을 비교한 것이다. 딥러닝 모델의 복잡도가 높을수록 학습 시간이 많이 소요되지만 2.5 시간 내에 학습이 완료되는 것을 확인할 수 있다. 데이터셋이 1,000 여건으로 많지 않기 때문에 학습 시간의 차이는 크지 않다. 성능의 개선을 위하여 학습 데이터를 확대한다면 학습의 시간 차이는 크질 것으로 예상된다. 성능 개선을 위하여 2 클래스를 4 클래스로 증가시키고 지속적으로 학습데이터를 확대하고자 한다.

표 1. YOLOv5 학습결과 성능 비교표

구 분	Class	Images	Instances	P	R	mAP	mAP-95
YOLOv5n	all	206	809	0.933	0.907	0.965	<u>0.749</u>
	FALLDOWN	206	170	0.935	0.965	0.982	0.81
	NORMAL	206	639	0.93	0.85	0.948	0.688
YOLOv5s	all	206	809	0.946	0.925	0.967	<u>0.79</u>
	FALLDOWN	206	170	0.953	0.965	0.979	0.849
	NORMAL	206	639	0.939	0.886	0.955	0.731
YOLOv5m	all	206	809	0.958	0.936	0.968	<u>0.802</u>
	FALLDOWN	206	170	0.965	0.97	0.982	0.854
	NORMAL	206	639	0.95	0.901	0.954	0.749
YOLOv5l	all	206	809	0.946	0.938	0.971	<u>0.811</u>
	FALLDOWN	206	170	0.949	0.971	0.98	0.859
	NORMAL	206	639	0.943	0.906	0.961	0.763
YOLOv5x	all	206	809	0.956	0.953	0.971	<u>0.818</u>
	FALLDOWN	206	170	0.96	0.981	0.983	0.867
	NORMAL	206	639	0.953	0.925	0.959	0.769

YOLOv5n	Model summary: 157 layers, 1761871 parameters, 0 gradients							
	Class	Images	Instances	P	R	mAP50	mAP50-95:	
	all	206	809	0.933	0.907	0.965	0.749	
	FALLDOWN	206	170	0.935	0.965	0.982	0.81	
	NORMAL	206	639	0.93	0.85	0.948	0.688	
YOLOv5s	Model summary: 166 layers, 7056607 parameters, 0 gradients							
	Class	Images	Instances	P	R	mAP50	mAP50-95:	
	all	206	809	0.946	0.925	0.967	0.79	
	FALLDOWN	206	170	0.953	0.965	0.979	0.849	
	NORMAL	206	639	0.939	0.886	0.955	0.731	
YOLOv5m	Model summary: 212 layers, 20856975 parameters, 0 gradients							
	Class	Images	Instances	P	R	mAP50	mAP50-95:	
	all	206	809	0.958	0.936	0.968	0.802	
	FALLDOWN	206	170	0.965	0.97	0.982	0.854	
	NORMAL	206	639	0.95	0.901	0.954	0.749	
YOLOv5l	Model summary: 267 layers, 46113663 parameters, 0 gradients							
	Class	Images	Instances	P	R	mAP50	mAP50-95:	
	all	206	809	0.946	0.938	0.971	0.811	
	FALLDOWN	206	170	0.949	0.971	0.98	0.859	
	NORMAL	206	639	0.943	0.906	0.961	0.763	
YOLOv5x	Model summary: 322 layers, 86180143 parameters, 0 gradients							
	Class	Images	Instances	P	R	mAP50	mAP50-95:	
	all	206	809	0.956	0.953	0.971	0.818	
	FALLDOWN	206	170	0.96	0.981	0.983	0.867	
	NORMAL	206	639	0.953	0.925	0.959	0.769	

그림 54. YOLOv5 2 클래스 학습결과 성능 비교

YOLOv5n	150 epochs completed in 2.164 hours.
YOLOv5s	150 epochs completed in 2.032 hours
YOLOv5m	150 epochs completed in 1.971 hours
YOLOv5l	150 epochs completed in 1.990 hours.
YOLOv5x	150 epochs completed in 2.652 hours

그림 55. YOLOv5 2 클래스 학습결과 시간 비교

2.6 팽창(dilated) 컨벌루션 YOLOv5 2 클래스 학습 및 추론 결과

YOLOv5의 버전별로 성능과 학습 속도를 비교 분석한 결과 mAP50에서 적은 레이어를 가지는 n 버전과 레이어가 2배 정도 많은 x 버전의 차이가 0.6% 정도로 작은 것을 확인하였다.

본 논문에서 임베디드 에지 컴퓨팅 시스템으로 이상상황 감지 시스템을 구현하는데 있어 레이어가 적은 n 버전을 선정하는 것이 효율적이라고 판단한다. 그리고 n 버전의 성능 개선을 위하여 정상상태와 영상과 쓰러짐 영상 간의 가로 세로 비율의 차이가 크고, 밀집도 낮은 영상에 대하여 성능을 개선하기 위하여 팽창 컨벌루션을 적용한 YOLOv5를 제안하고 이를 검증한다. 팽창 컨벌루션을 활용하면 계산량의 변화없이 성능을 개선할 수 있다.

YOLOv5 n 버전에 대하여 첫 번째 레이어에 대하여 3x3 컨벌루션을 대신하여 5x5 팽창 컨벌루션을 적용하여 학습 결과를 비교한다.

그림 56와 57은 각각 3x3 컨벌루션과 5x5 팽창 컨벌루션을 적용한 결과이다. 3x3 컨벌루션의 결과는 쓰러짐과 정상상태 검출률은 각각 97%, 88%이며, 배경을 정상상태로 오감지하는 것이 9%이다. 5x5 팽창 컨벌루션을 적용한 결과 쓰러짐과 정상상태 검출은 각각 97%, 91%로 정상상태 검출에 대한 성능이 3% 개선된다. 정상상태에 대한 배경으로의 오감지도 6%로 3% 정도 개선되는 것을 확인할 수 있다. 그림 58과 59은 F1-score를 비교한 것으로 각각 95%, 96%로 팽창 컨벌루션의 결과 1% 성능이 개선된다. 또한 팽창 컨벌루션을 적용하여 이상행동 및 정상상태를 분리 검출한 결과 대체로 정확하게 분류 검출함을 확인할 수 있다.

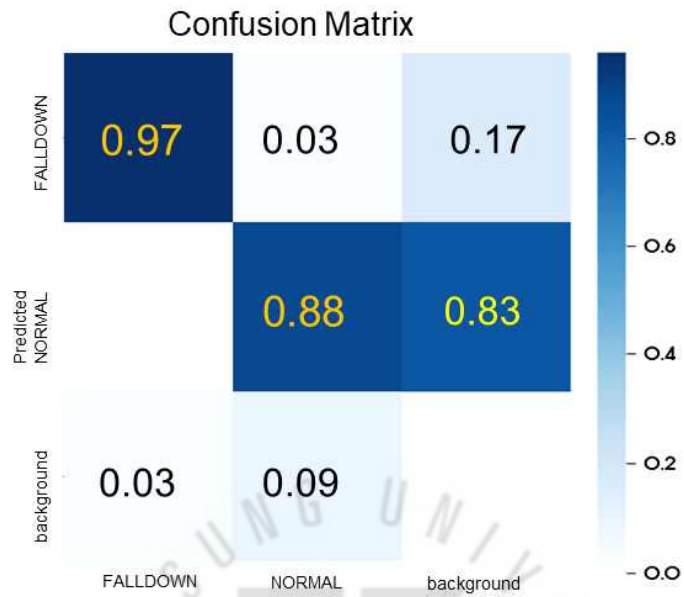


그림 56 3x3 컨벌루션의 학습 결과 혼란 행렬(confusion matrix)

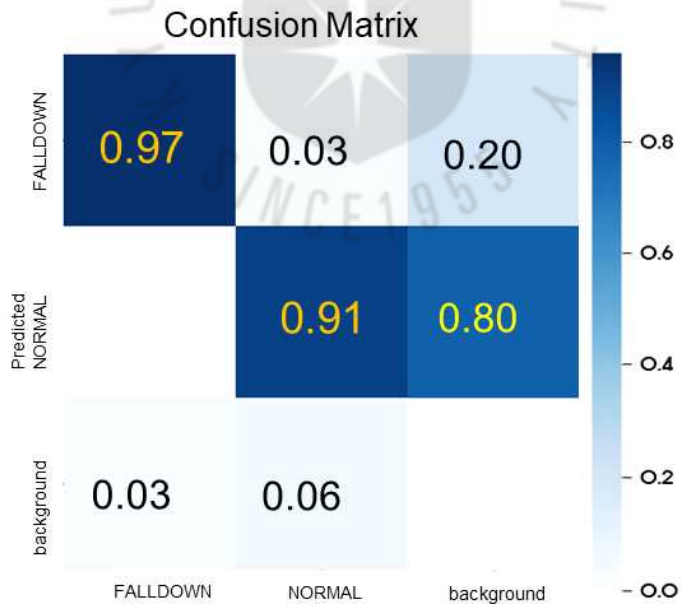


그림 57. 5x5 패딩 컨벌루션 적용한 혼란 행렬(confusion matrix)

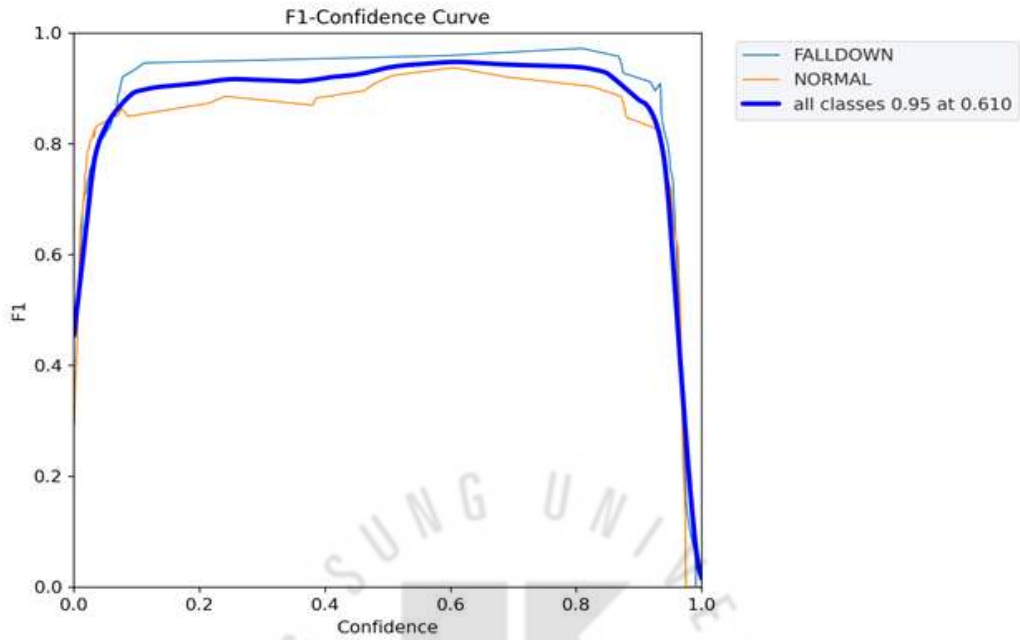


그림 58. 3x3 컨벌루션 YOLOv5 n버전 F1-Confidence 그래프

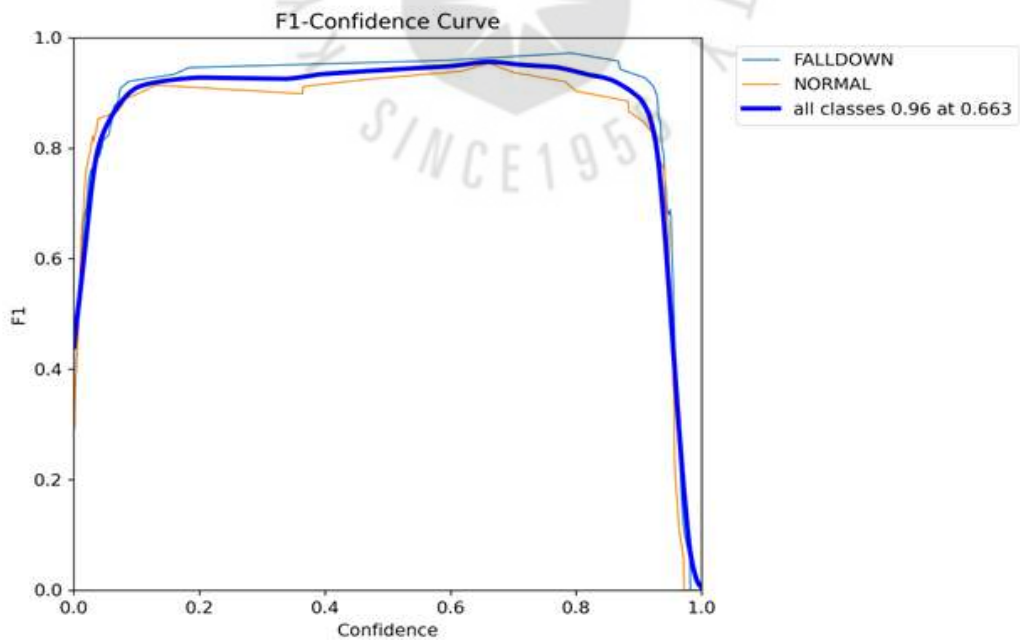


그림 59. 팅창 컨벌루션 YOLOv5 n버전 F1-Confidence 그래프

3. YOLOv5 4 클래스 이상행동 감지 시뮬레이션 결과 및 고찰

3.1 YOLOv5 n버전 4 클래스 분류 학습 결과

그림 60, 61, 62 그리고 63은 YOLOv5 n버전의 4 클래스 쓰러짐, 폭행, 기물파손, 정상상태 검출에 대한 정밀도, 재현률, 정밀도-재현률 그리고 F1-Score 그래프이다. 정밀도는 신뢰 수준(confidence level) 95.8%에서 모든 클래스에 대해서는 100% 이며, 재현률은 신뢰 수준 0%에서 99% 이다. 정밀도-재현률 그래프는 mAP50으로 97.8%를 나타낸다. F1-Score은 신뢰 수준 40.6%에서 95%로 나타난다. 그리고 그림 43은 3x3 컨볼루션 YOLOv5 n버전의 mAP50에서의 분류 정확도를 나타낸 학습 결과이다.

그림 64는 YOLOv5 n버전 감지 결과이다. 4 가지 즉, 폭행, 쓰러짐, 기물파손 그리고 정상에 대해서 대체로 분류된 결과를 확인할 수 있다.

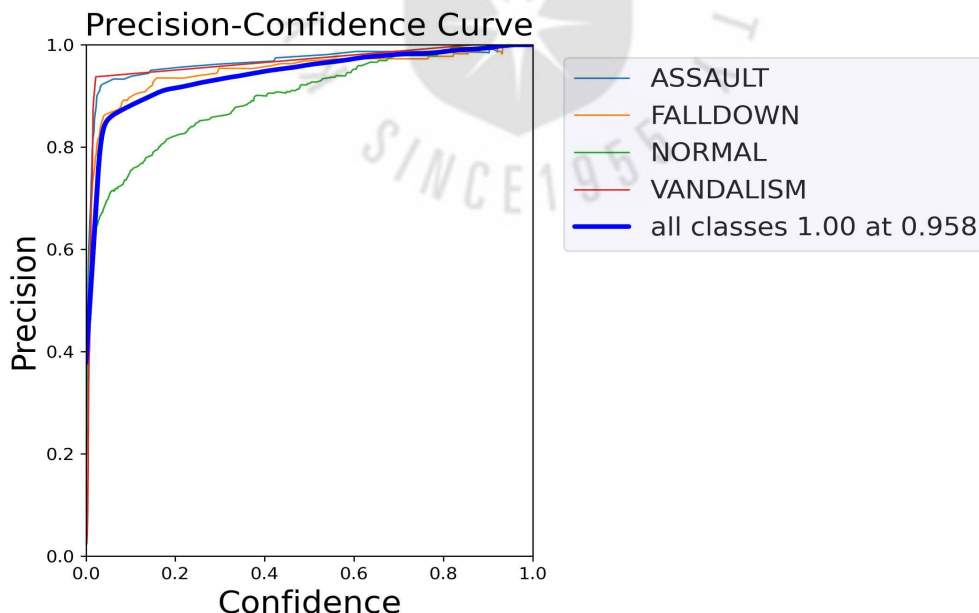


그림 60. YOLOv5 n버전 학습결과 Precision-Confidence 그래프

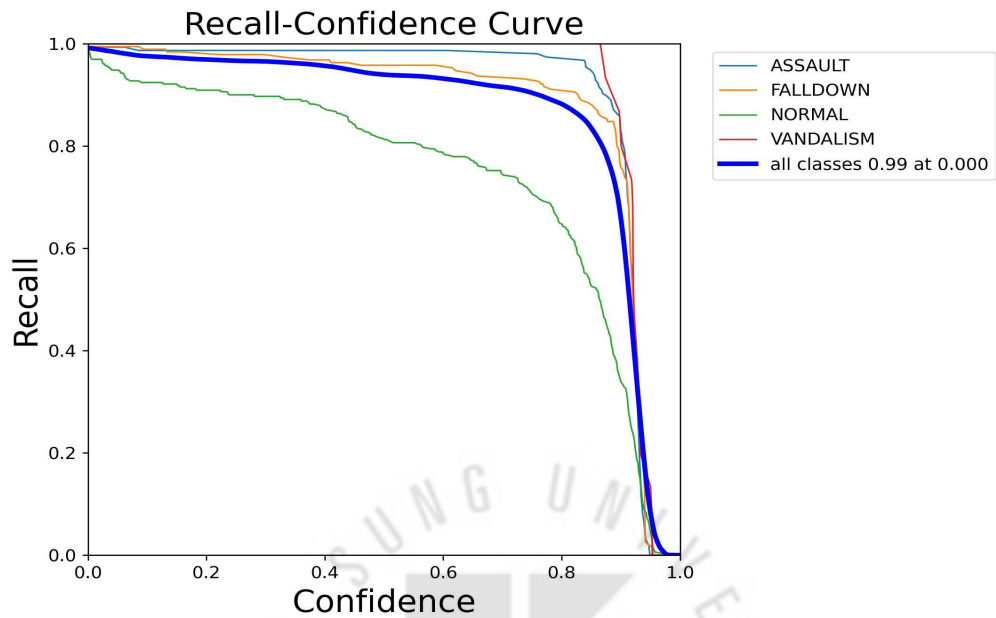


그림 61. YOLOv5 n 버전 학습결과 Recall-Confidence 그래프

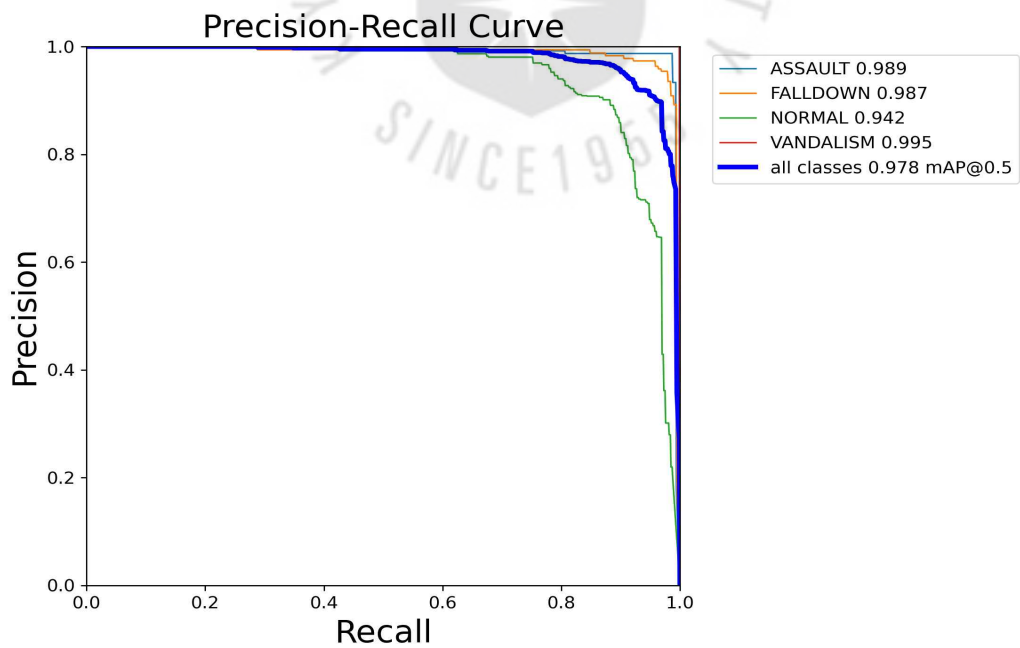


그림 62. YOLOv5 n 버전 학습결과 Precision-Recall Confidence 그래프

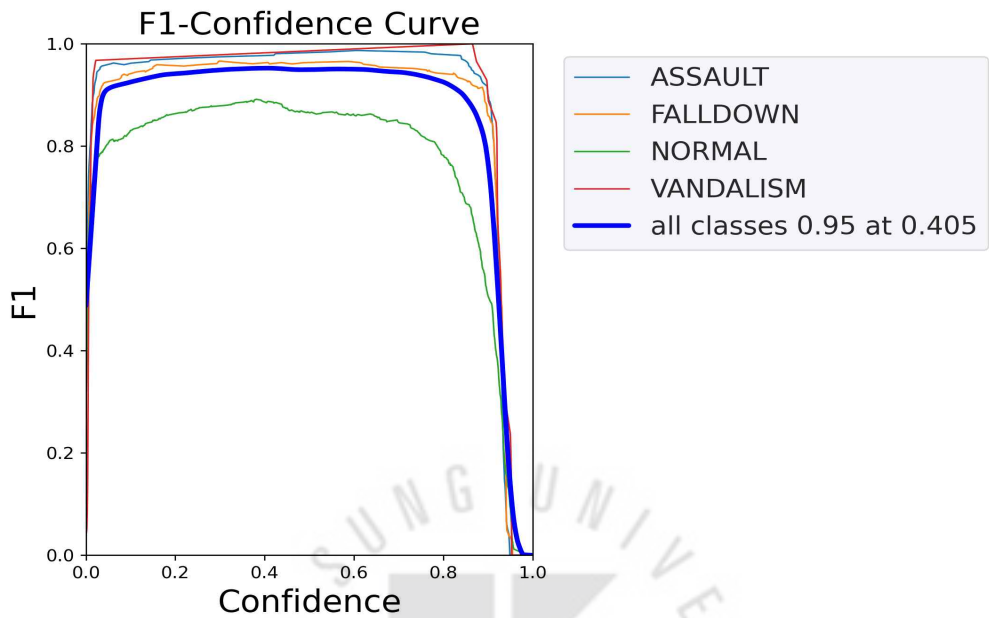


그림 63. YOLOv5 n버전 학습결과 F1-Confidence 그래프

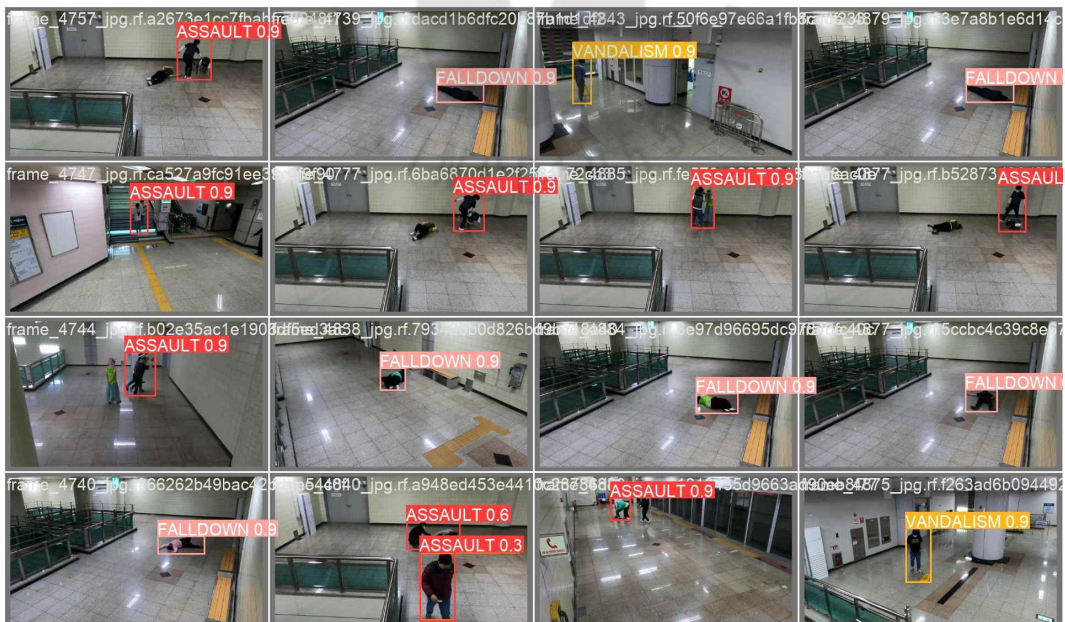


그림 64 YOLOv5 n버전 학습결과

3.2 YOLOv5 s버전 4 클래스 분류 학습 결과

그림 65, 66, 67 그리고 68은 YOLOv5 s버전의 4 클래스 쓰러짐, 폭행, 기물파손, 정상상태 검출에 대한 정밀도, 재현률, 정밀도-재현률 그리고 F1-Score 그래프이다. 정밀도는 신뢰 수준(confidence level) 98%에서 모든 클래스에 대해서는 100% 이며, 재현률은 신뢰 수준 0%에서 99% 이다. 정밀도-재현률 그래프는 mAP50으로 98.3%를 나타낸다. F1-Score은 신뢰 수준 46.3%에서 96%로 나타난다. 그리고 그림 69는 YOLOv5 s버전 감지 결과이다.

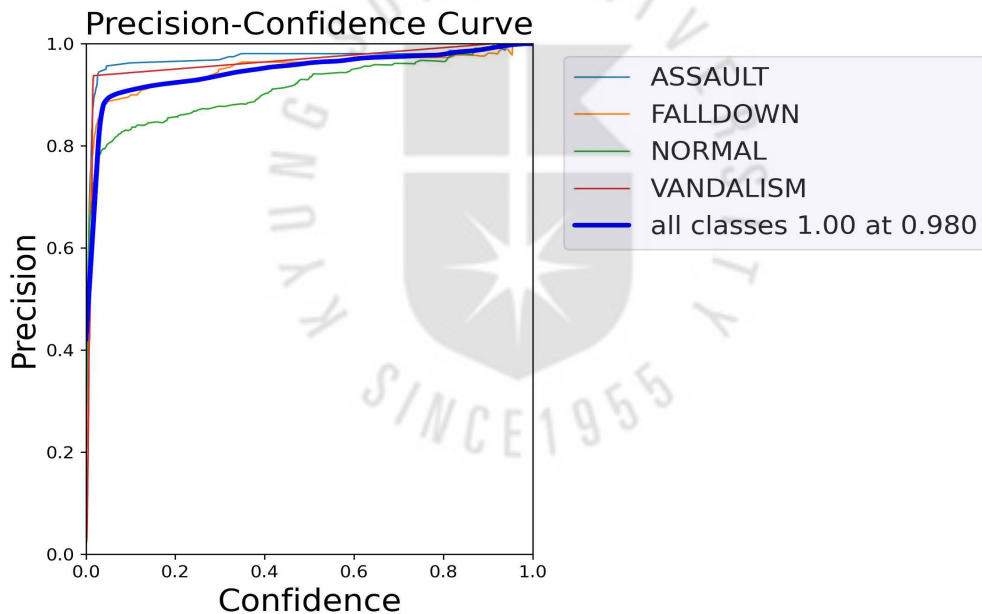


그림 65 YOLOv5 s버전 학습결과 Precision-Confidence 그래프

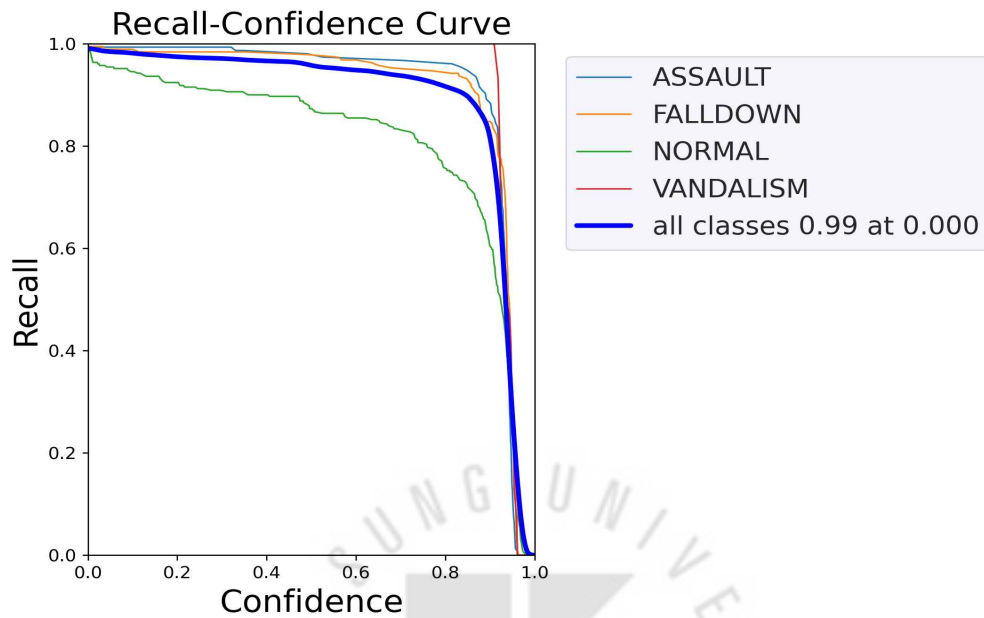


그림 66 YOLOv5 s 버전 학습결과 Recall-Confidence 그래프

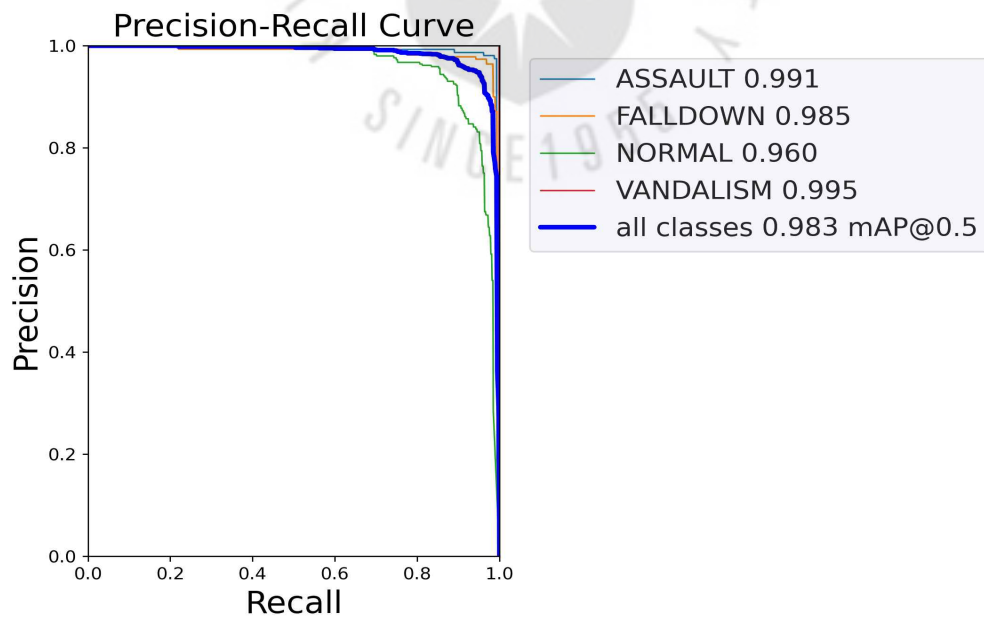


그림 67 YOLOv5 s 버전 학습결과 Precision-Recall Confidence 그래프

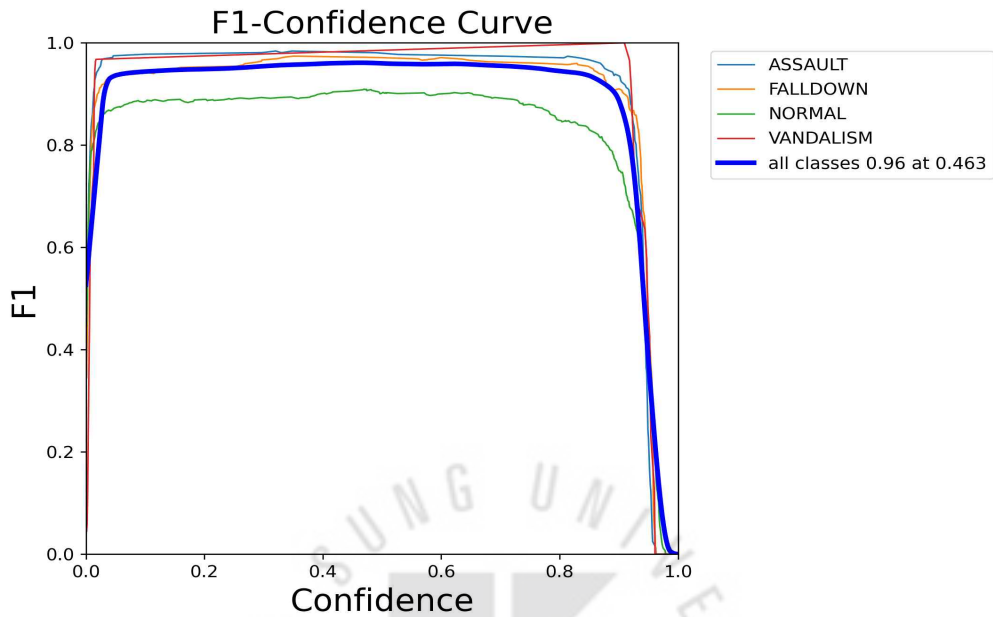


그림 68 YOLOv5 s버전 학습결과 F1-Confidence 그래프

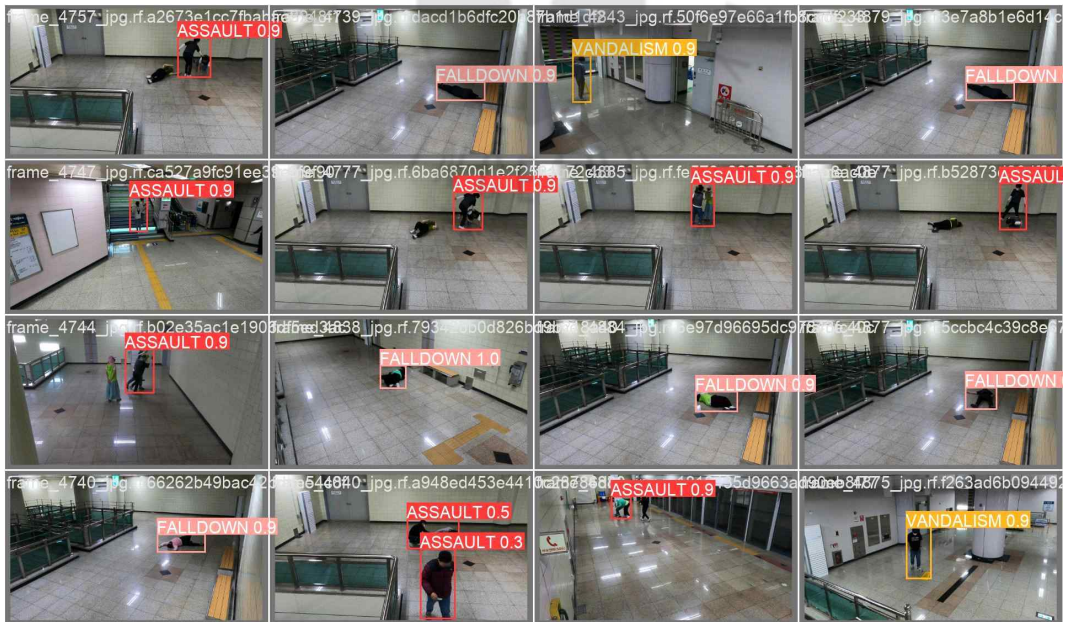


그림 69 YOLOv5 s버전 학습결과

3.3 YOLOv5 m버전 4 클래스 분류 학습 결과

그림 70, 71, 72 그리고 73은 YOLOv5 m버전의 4 클래스 쓰러짐, 폭행, 기물파손, 정상상태 검출에 대한 정밀도, 재현률, 정밀도-재현률 그리고 F1-Score 그래프이다. 정밀도는 신뢰 수준(confidence level) 97%에서 모든 클래스에 대해서는 n 또는 s 버전보다는 떨어지는 97%이며, 재현률은 신뢰 수준 0%에서 99%이다. 정밀도-재현률 그래프는 mAP50으로 98%를 나타낸다. F1-Score은 신뢰 수준 50.6%에서 97%로 나타난다.

그림 74은 3x3 컨벌루션 YOLOv5 m버전의 mAP50에서의 분류 정확도를 나타낸 학습 결과이다.

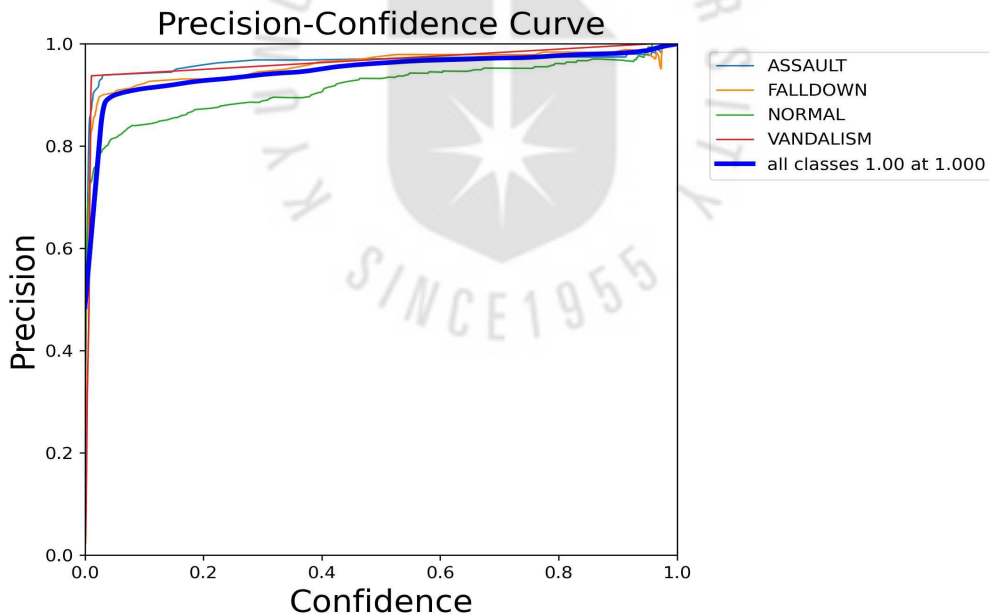


그림 70 YOLOv5 m버전 학습결과 Precision-Confidence 그래프

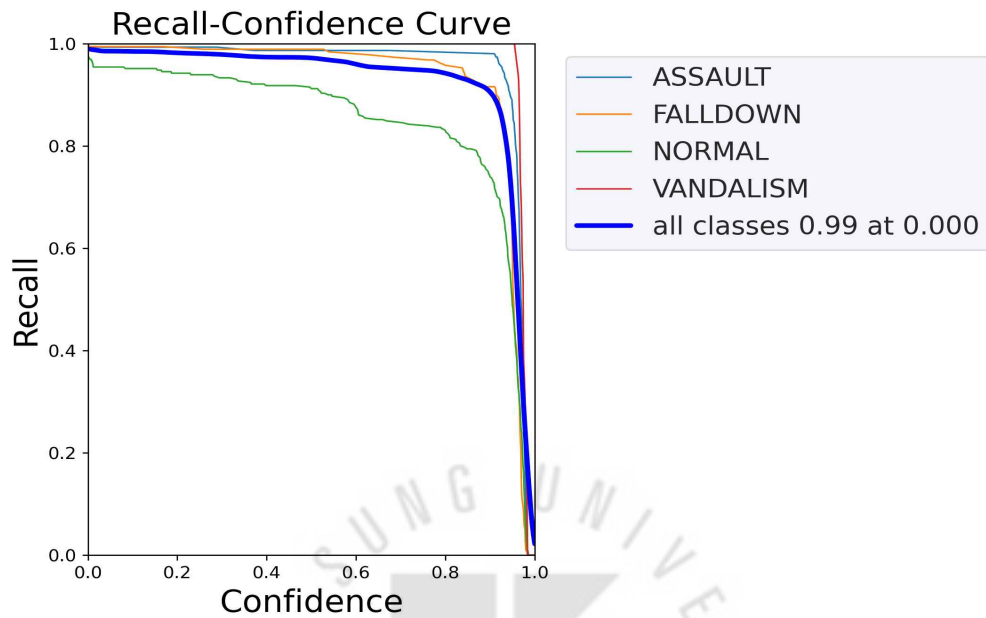


그림 71 YOLOv5 m 버전 학습결과 Recall-Confidence 그래프

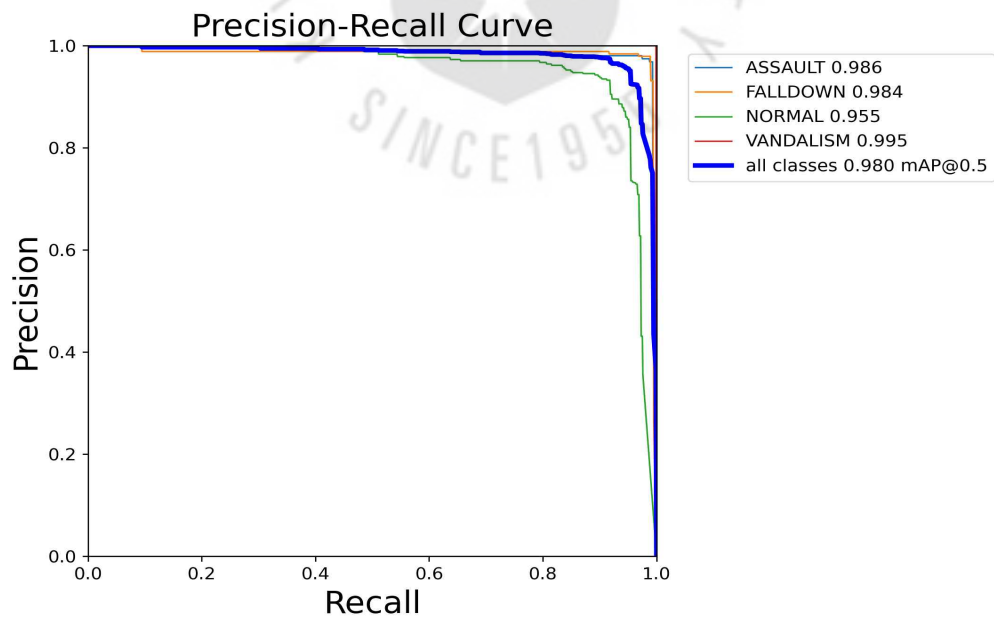


그림 72 YOLOv5 m 버전 학습결과 Precision-Recall Confidence 그래프

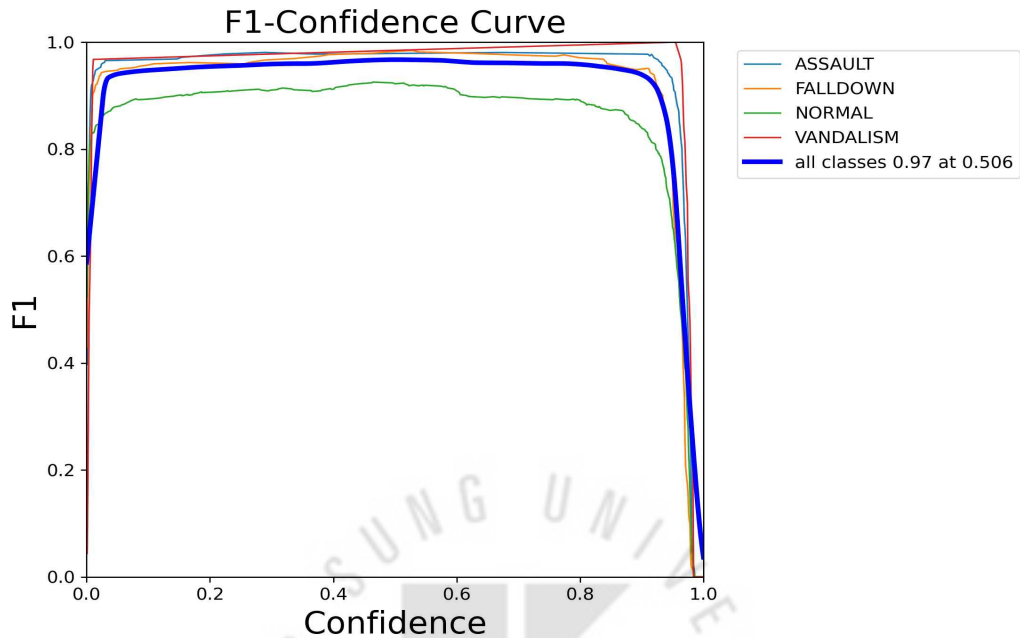


그림 73 YOLOv5 m 버전 학습결과 F1-Confidence 그래프



그림 74 YOLOv5 m 버전 학습결과

3.4 YOLOv5 1버전 4 클래스 분류 학습 결과

그림 75, 76, 77 그리고 78은 YOLOv5 1버전의 4 클래스 쓰러짐, 폭행, 기물파손, 정상상태 검출에 대한 정밀도, 재현률, 정밀도-재현률 그리고 F1-Score 그래프이다. 정밀도는 신뢰 수준(confidence level) 100%에서 모든 클래스에 대해서 100% 이며, 재현률은 신뢰 수준 0%에서 99% 이다. 정밀도-재현률 그래프는 mAP50으로 97.7%를 나타낸다. F1-Score은 신뢰 수준 58.4%에서 97%로 나타난다. 그리고 그림 79는 YOLOv5 1버전 감지 결과이다.

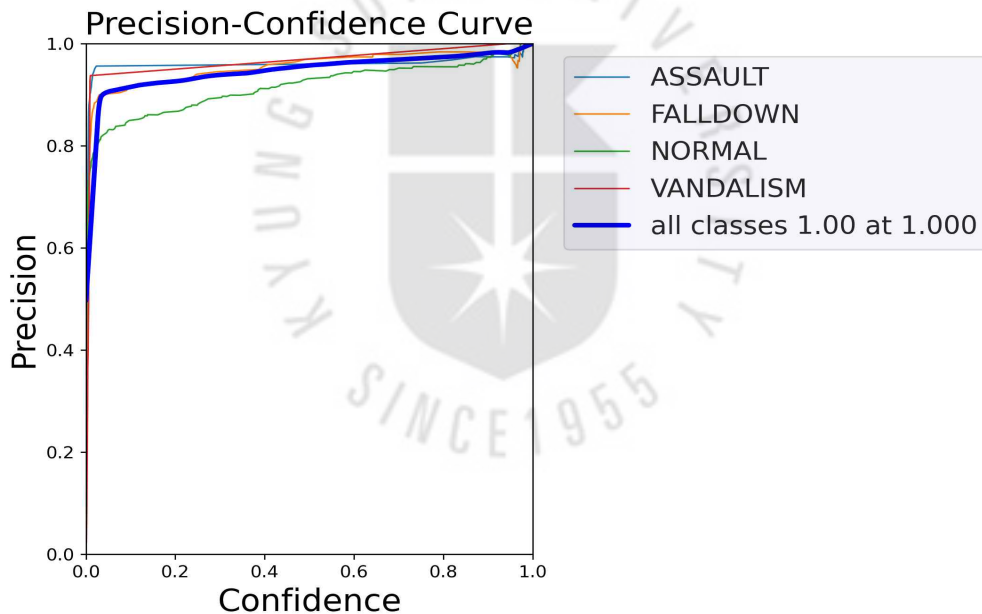


그림 75 YOLOv5 1버전 학습결과 Precision-Confidence 그래프

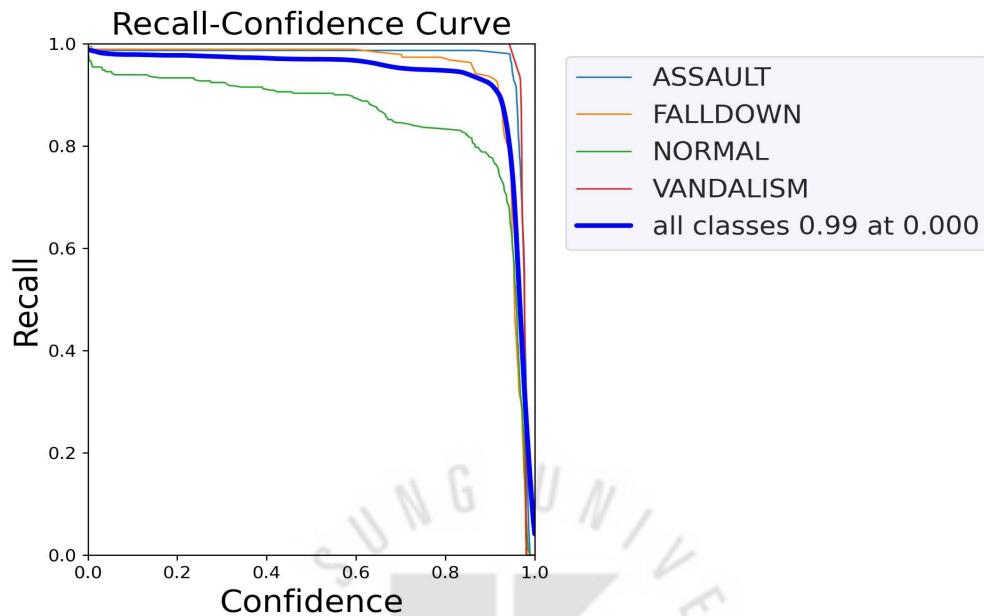


그림 76. YOLOv5 1버전 학습결과 Recall-Confidence 그래프

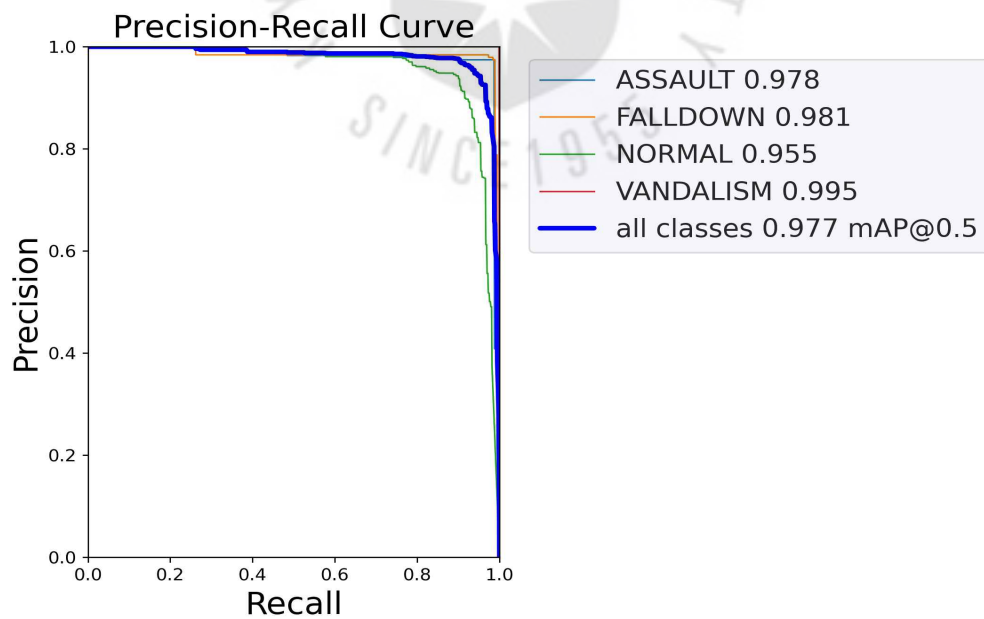


그림 77. YOLOv5 1버전 학습결과 Precision-Recall Confidence 그래프

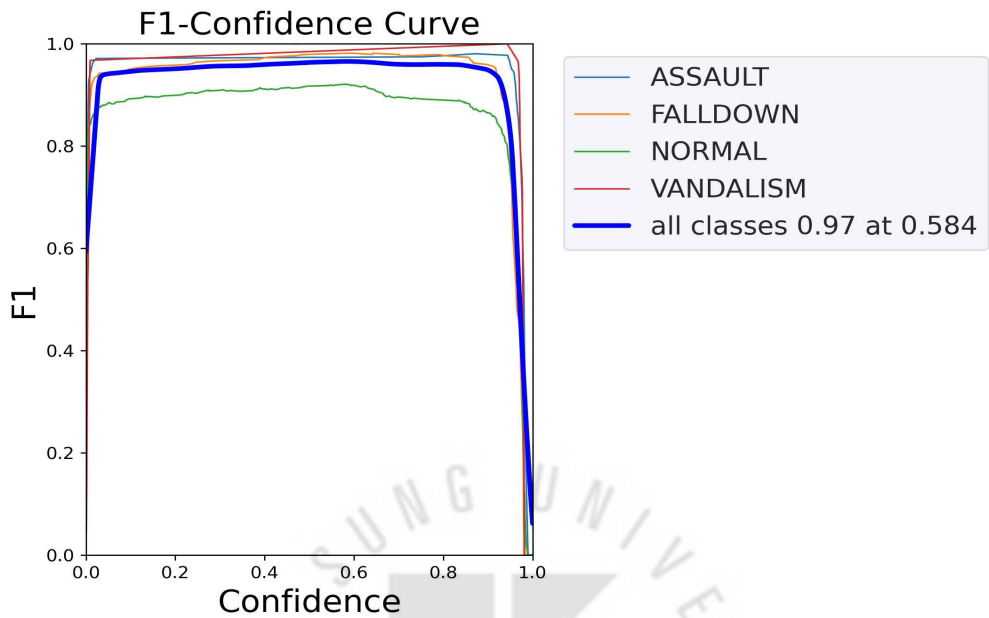


그림 78. YOLOv5 1버전 학습결과 Precision-Recall Confidence 그래프

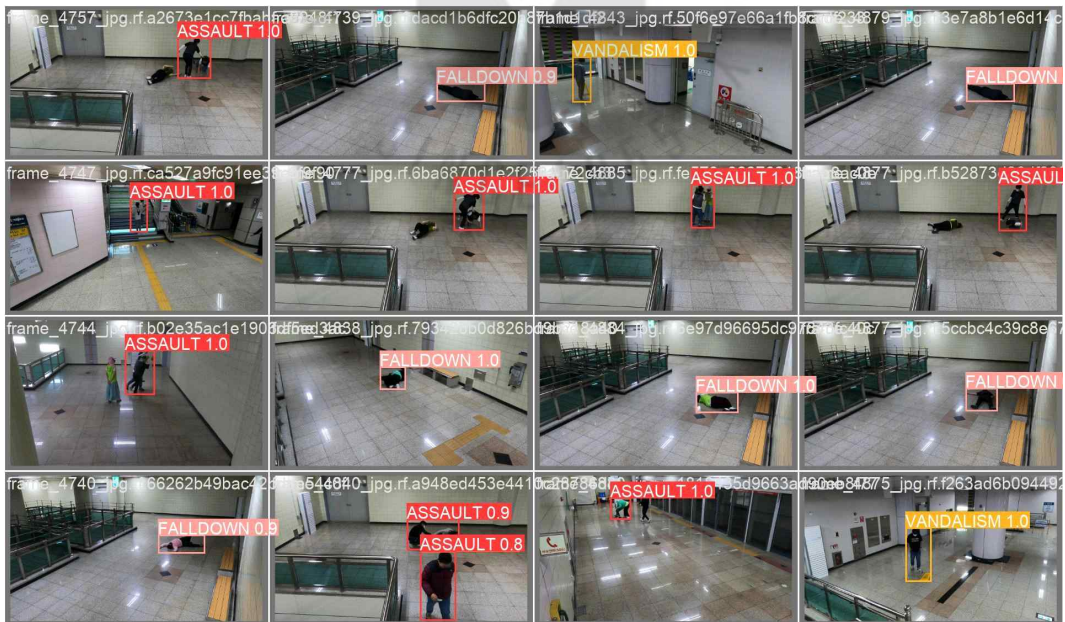


그림 79. YOLOv5 1버전 학습결과

3.5 YOLOv5 x버전 4 클래스 분류 학습 결과

그림 80, 81, 82 그리고 83은 YOLOv5 x버전의 4 클래스 쓰러짐, 폭행, 기물파손, 정상상태 검출에 대한 정밀도, 재현률, 정밀도-재현률 그리고 F1-Score 그래프이다. 정밀도는 신뢰 수준(confidence level) 100%에서 모든 클래스에 대해서 100% 이며, 재현률은 신뢰 수준 0%에서 99% 이다. 정밀도-재현률 그래프는 mAP50으로 97.7%를 나타낸다. F1-Score은 신뢰 수준 60.1%에서 97%로 나타난다. 그리고 그림 84는 YOLOv5 x버전 감지 결과이다.

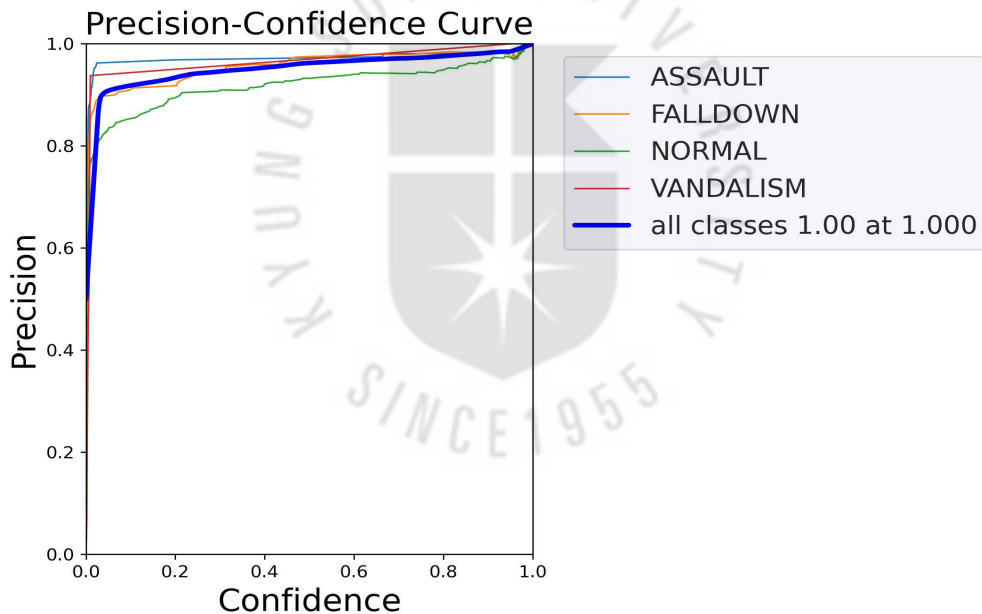


그림 80. YOLOv5 x버전 학습결과 Precision-Confidence 그래프

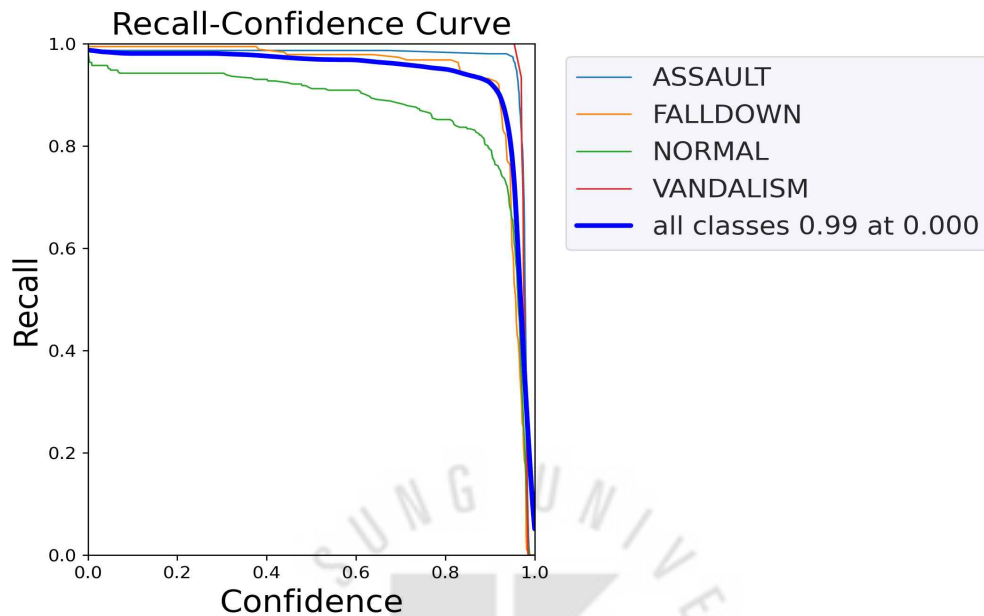


그림 81. YOLOv5 x 버전 학습결과 Recall-Confidence 그래프

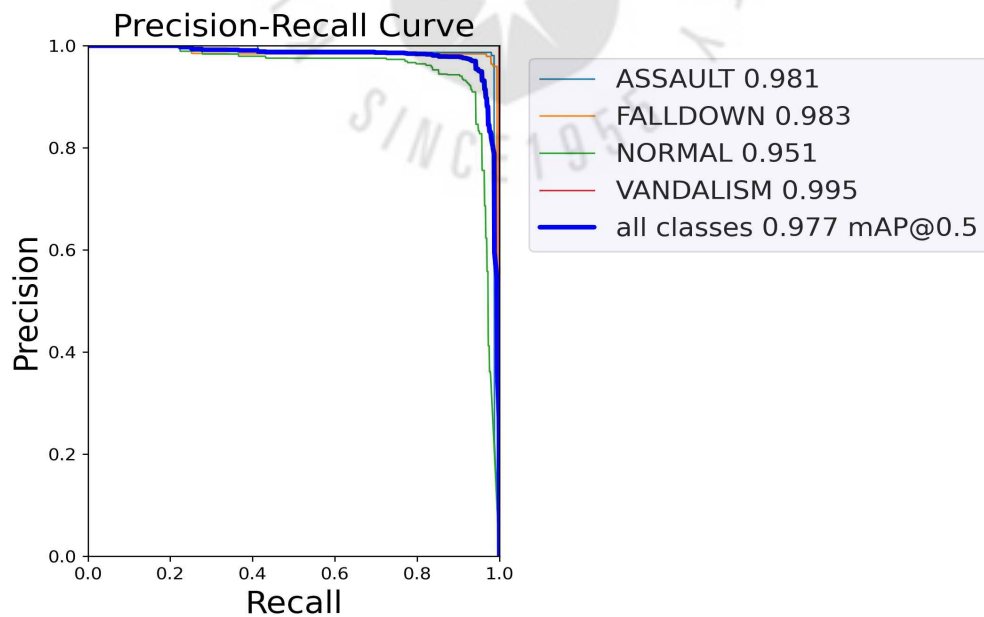


그림 82. YOLOv5 x 버전 학습결과 Precision-Recall Confidence 그래프

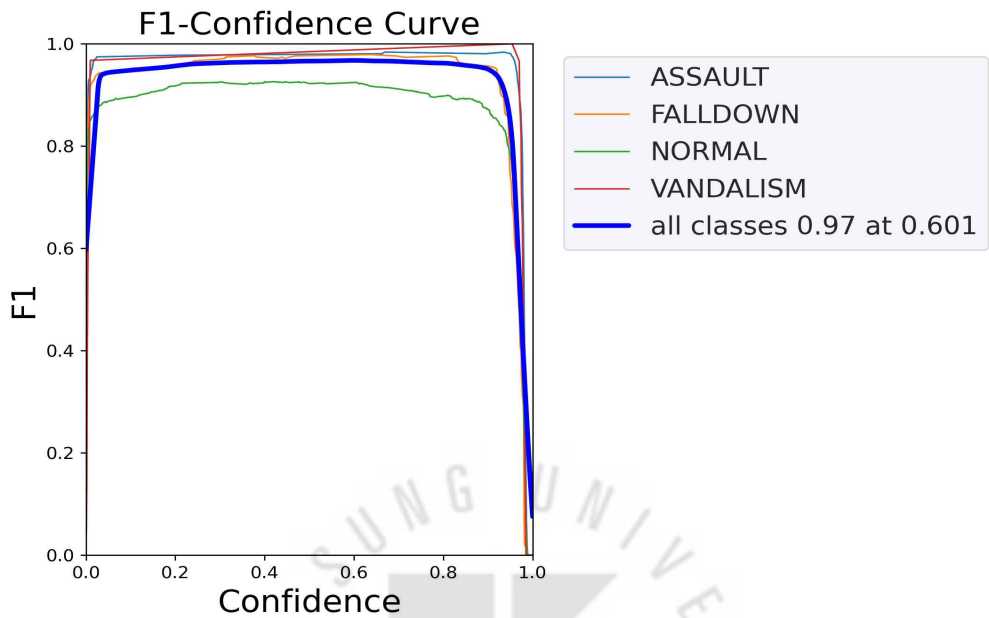


그림 83. YOLOv5 x 버전 학습결과 Precision-Recall Confidence 그래프

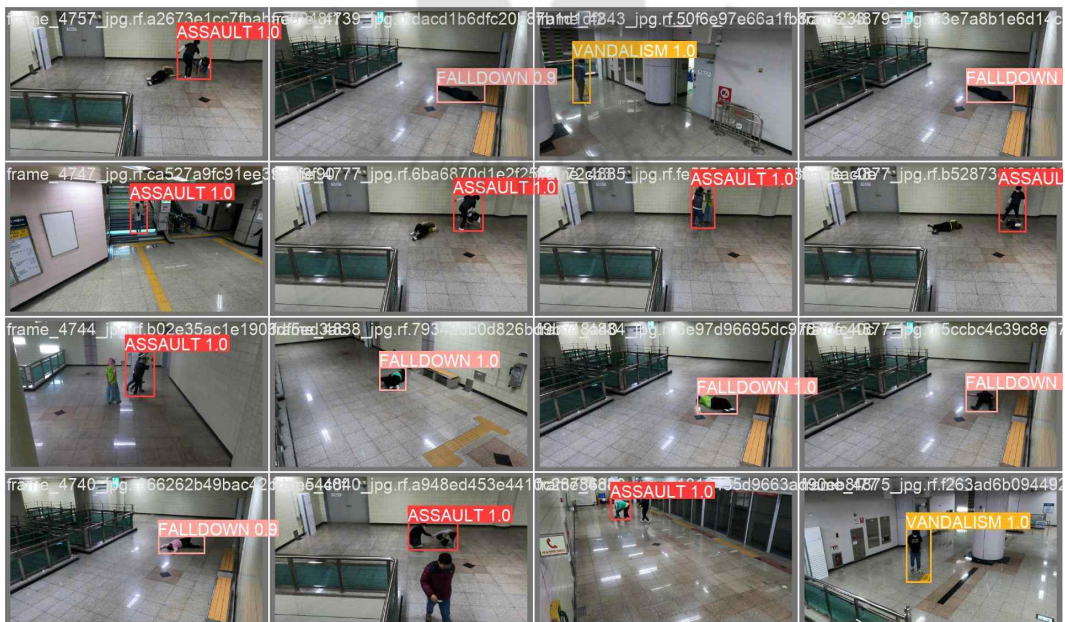


그림 84. YOLOv5 x 버전 학습결과

본 실험에서는 YOLOv5 4 클래스 분류 학습에 대한 결과로 임베디드 및 에지 컴퓨팅에 적용하기 위한 적절한 모델 선정을 위하여 YOLOv5 n, s, m, l, x 버전에 대하여 학습 결과를 비교 분석한다.

표 2와 그림 85와 같이 각 버전에 대하여 학습한 결과 n, s, m, l, x 버전의 mAP50 성능은 각각 97.8%, 98.3%, 98%, 97.7% 그리고 97.7%로 나타났다. 일반적으로 mAP50을 주로 활용하고 있어 쓰러짐 이상행동 감지에 대해서 YOLOv5의 성능 97% 이상 우수한 것으로 판단된다. n 버전과 x 버전의 mAP 성능 차이는 0.1%로 n 버전이 근소하게 앞서는 것으로 나타났다. 또한 2 클래스로 학습한 결과와 비교를 하였을 때 mAP50 성능이 근소하지만 개선됨을 확인 할 수 있었다.

mAP50-95는 각각 78.4%, 78.8%, 80.3%, 82% 그리고 84.2%로 n 버전과 x버전의 차이는 5.8% 정도 차이가 있음을 확인할 수 있다. IoU가 높을수록 성능의 차이가 있다. 2 클래스와 비교를 했을 때 mAP50-95는 n 및 x 버전이 다소 큰 차이가 있었으며 그 외 버전은 4클래스의 학습결과가 근소한 차이로 앞서는 것으로 확인되었다.

그림 86은 YOLOv5의 학습에 소요되는 시간을 비교한 것이다. 딥러닝 모델의 복잡도가 높을수록 학습 시간이 많이 소요 될 것으로 생각했지만 m, l 버전의 시간은 각각 18.8 시간과 28.8 시간이 소요되고 x 버전 약 5.3 시간 내에 학습이 완료되는 것을 확인할 수 있다. 2 클래스로 학습을 진행하였을 때 시간의 의미가 큰차이가 없었던 것으로 확인 하였으나 데이터셋 증가에 따른 4 클래스로 증가는 전체적인 학습 시간의 증가로 이어진 것으로 확인된다.

표 2. YOLOv5 4 클래스 학습결과 성능 비교표

구 분	Class	Images	Instances	P	R	mAP50	mAP-95
YOLOv5n	all	387	692	0.961	0.954	<u>0.979</u>	<u>0.784</u>
	ASSAULT	387	155	0.981	0.987	0.985	0.795
	FALLDOWN	387	191	0.969	0.963	0.985	0.835
	NORMAL	387	331	0.926	0.866	0.953	0.687
	VANDALISM	387	15	0.971	1	0.995	0.82
YOLOv5s	all	387	692	0.95	0.96	<u>0.979</u>	<u>0.788</u>
	ASSAULT	387	155	0.968	0.99	0.989	0.773
	FALLDOWN	387	191	0.96	0.969	0.984	0.838
	NORMAL	387	331	0.909	0.88	0.947	0.672
	VANDALISM	387	15	0.964	1	0.995	0.871
YOLOv5m	all	387	692	0.956	0.963	<u>0.981</u>	<u>0.803</u>
	ASSAULT	387	155	0.975	0.981	0.991	0.764
	FALLDOWN	387	191	0.966	0.963	0.984	0.845
	NORMAL	387	331	0.913	0.908	0.953	0.713
	VANDALISM	387	15	0.969	1	0.995	0.89
YOLOv5l	all	387	692	0.953	0.963	<u>0.983</u>	<u>0.82</u>
	ASSAULT	387	155	0.96	0.981	0.989	0.783
	FALLDOWN	387	191	0.966	0.963	0.985	0.855
	NORMAL	387	331	0.918	0.908	0.962	0.721
	VANDALISM	387	15	0.967	1	0.995	0.919
YOLOv5x	all	387	692	0.967	0.981	<u>0.983</u>	<u>0.842</u>
	ASSAULT	387	155	0.984	0.994	0.993	0.795
	FALLDOWN	387	191	0.974	0.989	0.987	0.876
	NORMAL	387	331	0.94	0.942	0.956	0.779
	VANDALISM	387	15	0.971	1	0.995	0.917

YOLOv5n

Model summary: 157 layers, 1764577 parameters, 0 gradients							
Class	Images	Instances	P	R	mAP50	mAP50-95:	
all	387	692	0.961	0.954	0.979	0.784	
ASSAULT	387	155	0.981	0.987	0.985	0.795	
FALLDOWN	387	191	0.969	0.963	0.985	0.835	
NORMAL	387	331	0.926	0.866	0.953	0.687	
VANDALISM	387	15	0.971	1	0.995	0.82	

YOLOv5s

Model summary: 166 layers, 7062001 parameters, 0 gradients							
Class	Images	Instances	P	R	mAP50	mAP50-95:	
all	387	692	0.95	0.96	0.979	0.788	
ASSAULT	387	155	0.968	0.99	0.989	0.773	
FALLDOWN	387	191	0.96	0.969	0.984	0.838	
NORMAL	387	331	0.909	0.88	0.947	0.672	
VANDALISM	387	15	0.964	1	0.995	0.871	

YOLOv5m

Model summary: 212 layers, 20865057 parameters, 0 gradients							
Class	Images	Instances	P	R	mAP50	mAP50-95:	
all	387	692	0.956	0.963	0.981	0.803	
ASSAULT	387	155	0.975	0.987	0.991	0.764	
FALLDOWN	387	191	0.966	0.974	0.984	0.845	
NORMAL	387	331	0.913	0.891	0.953	0.713	
VANDALISM	387	15	0.969	1	0.995	0.89	

YOLOv5l

Model summary: 267 layers, 46124433 parameters, 0 gradients							
Class	Images	Instances	P	R	mAP50	mAP50-95:	
all	387	692	0.953	0.963	0.983	0.82	
ASSAULT	387	155	0.96	0.981	0.989	0.783	
FALLDOWN	387	191	0.966	0.963	0.985	0.855	
NORMAL	387	331	0.918	0.908	0.962	0.721	
VANDALISM	387	15	0.967	1	0.995	0.919	

YOLOv5x

Model summary: 322 layers, 86193601 parameters, 0 gradients							
val: Scanning /media/user/8TB Hard Disk/Tahira/yolov5/datasetv5/valid/labels.cac							
Class	Images	Instances	P	R	mAP50		
all	387	692	0.966	0.981	0.984	0.842	
ASSAULT	387	155	0.984	0.994	0.993	0.795	
FALLDOWN	387	191	0.974	0.983	0.987	0.876	
NORMAL	387	331	0.934	0.944	0.961	0.779	
VANDALISM	387	15	0.971	1	0.995	0.917	

그림 85. YOLOv5 4 클래스 학습결과 성능 비교

YOLOv5n	150 epochs completed in 4.340 hours
YOLOv5s	150 epochs completed in 3.695 hours
YOLOv5m	150 epochs completed in 18.896 hours
YOLOv5l	150 epochs completed in 28.378 hours
YOLOv5x	150 epochs completed in 5.347 hours.

그림 86. YOLOv5 4 클래스 학습결과 시간 비교

3.6 YOLOv5 n버전 팽창(dilated) 컨벌루션 4 클래스 분류 학습 결과

그림 87, 88, 89 그리고 90는 5x5 팽창 컨벌루션을 적용한 YOLOv5 n버전의 정밀도, 재현률, 정밀도-재현률 그리고 F1-Score 그래프이다. 정밀도는 신뢰 수준 97.3%에서 모든 클래스에 대해서는 100%이며, 재현률은 신뢰 수준 0%에서 99%이다. 정밀도-재현률 그래프는 mAP50으로 97.9%를 나타낸다. 그림 91은 5x5 팽창 컨벌루션 YOLOv5 n버전의 mAP50에서의 분류 정확도를 나타낸 학습 결과이다.

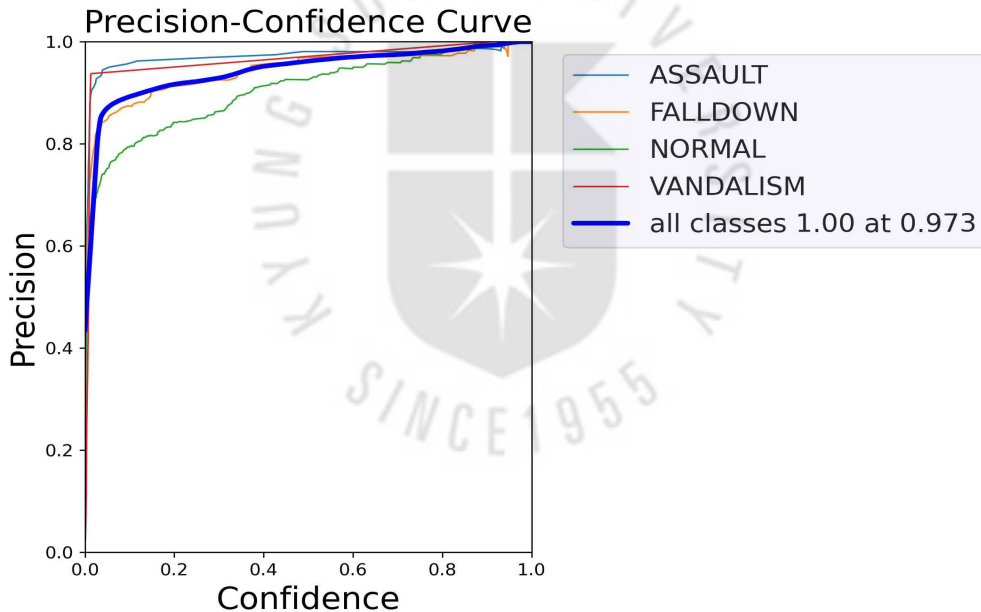


그림 87. 팽창 컨벌루션 YOLOv5 n버전 학습결과 Precision-Confidence 그래프

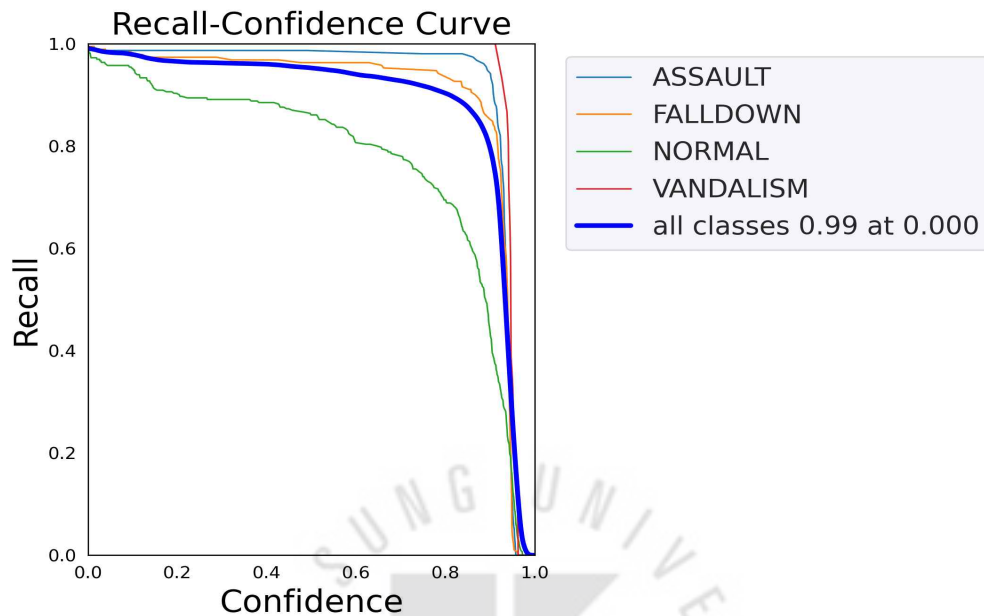


그림 88. 팽창 컨벌루션 YOLOv5 n버전 학습결과 Recall-Confidence 그래프

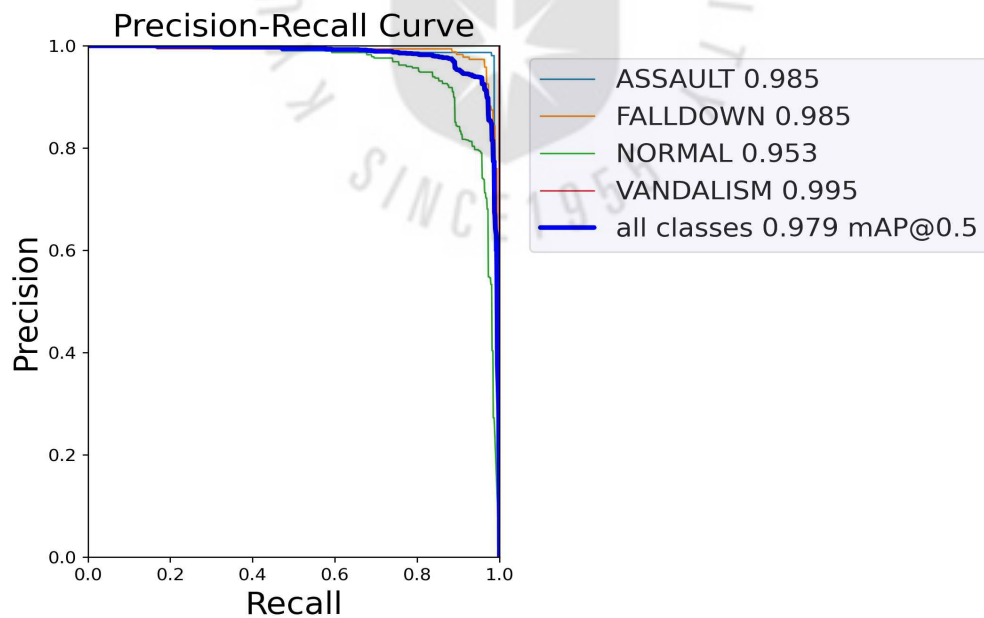


그림 89. 팽창 컨벌루션 YOLOv5 n버전 학습결과 Precision-Recall Confidence 그래프

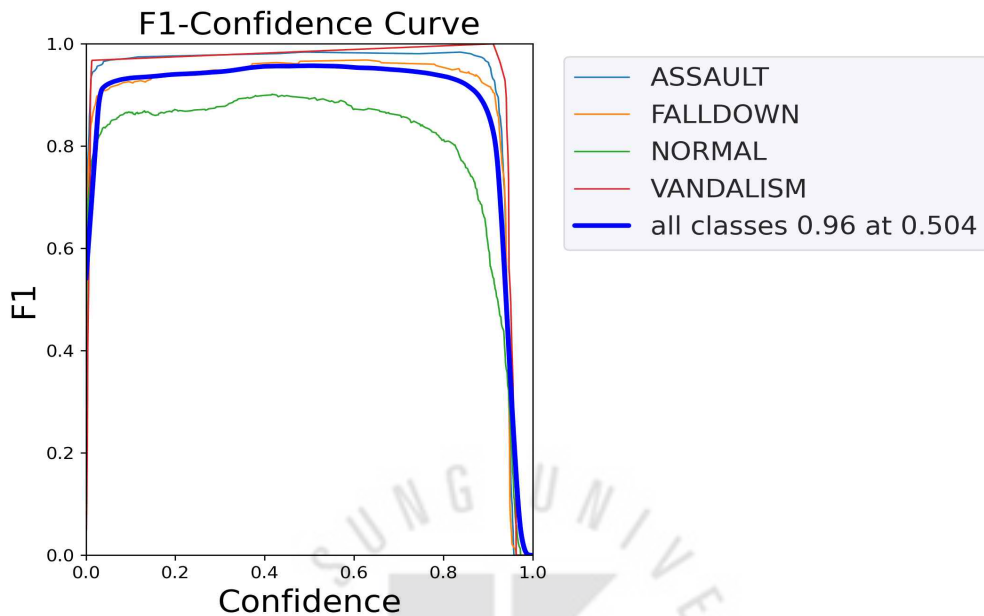


그림 90. 팽창 컨벌루션 YOLOv5 n버전 학습결과 F1-Confidence 그래프

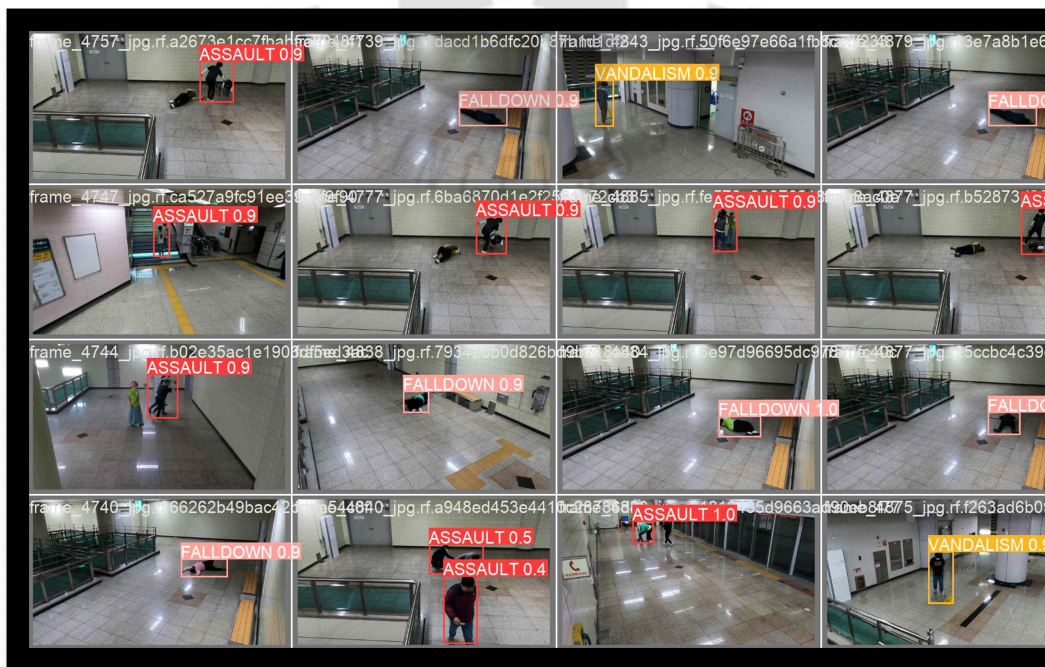


그림 91. 팽창 컨벌루션 YOLOv5 n버전 학습결과

3.7 YOLOv5 s버전 팽창(dilated) 컨벌루션 4 클래스 분류 학습 결과

그림 92, 93, 94 그리고 95는 5x5 팽창 컨벌루션을 적용한 YOLOv5 s버전의 정밀도, 재현률, 정밀도-재현률 그리고 F1-Score 그래프이다. 정밀도는 신뢰 수준 97.9%에서 모든 클래스에 대해서는 100%이며, 재현률은 신뢰 수준 0%에서 99%이다. 정밀도-재현률 그래프는 mAP50으로 97.9%를 나타낸다. F1-Score은 신뢰 수준 42%에서 96%로 나타난다. 그림 96은 5x5 팽창 컨벌루션 YOLOv5 s버전의 mAP50에서의 분류 정확도를 나타낸 학습 결과이다.

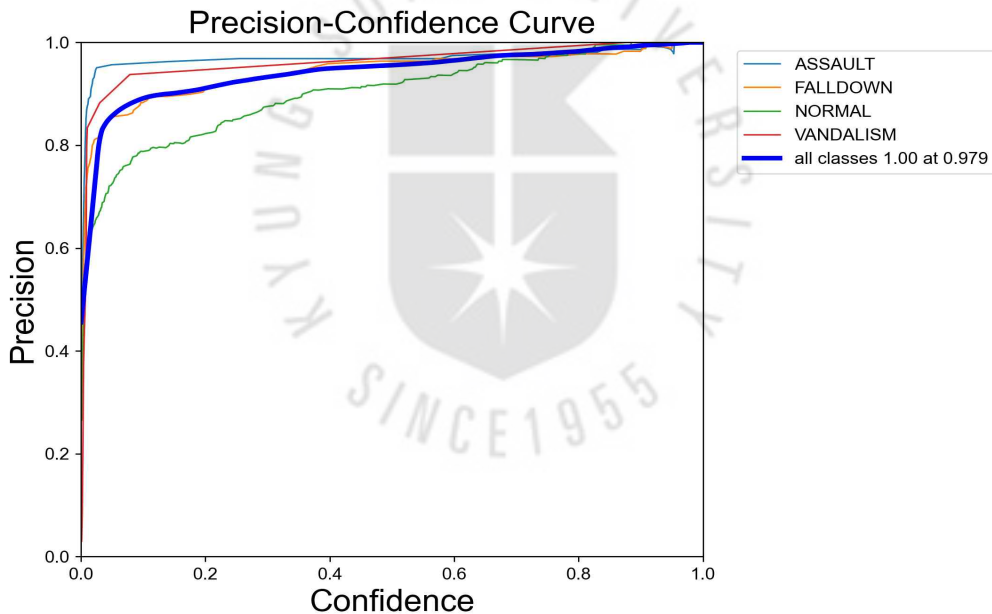


그림 92. 팽창 컨벌루션 YOLOv5 s버전 학습결과 Precision-Confidence 그래프

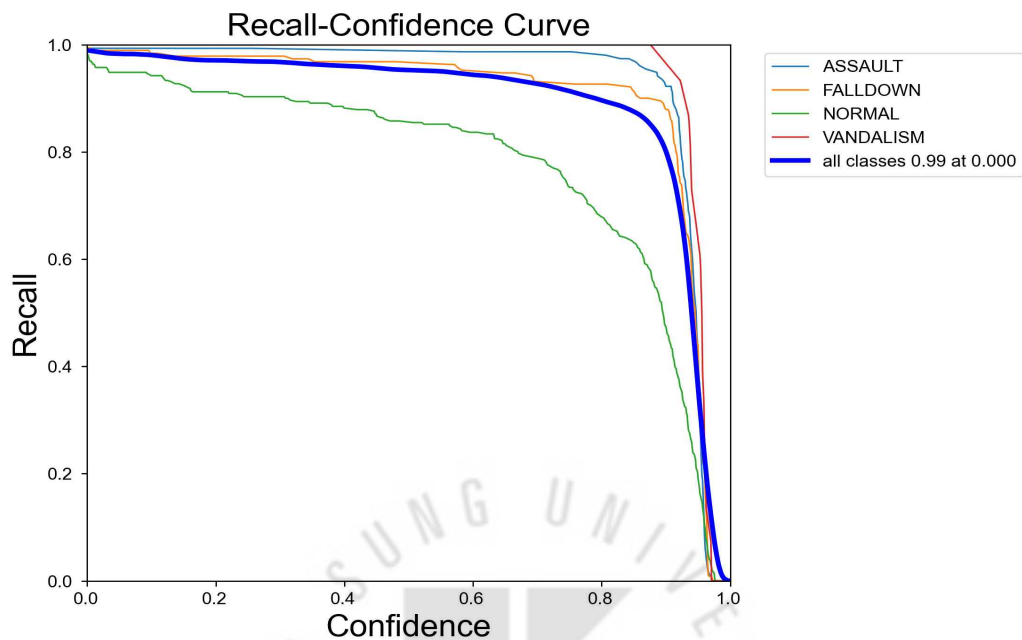


그림 93. 팽창 컨벌루션 YOLOv5 s버전 학습결과 Recall-Confidence 그래프

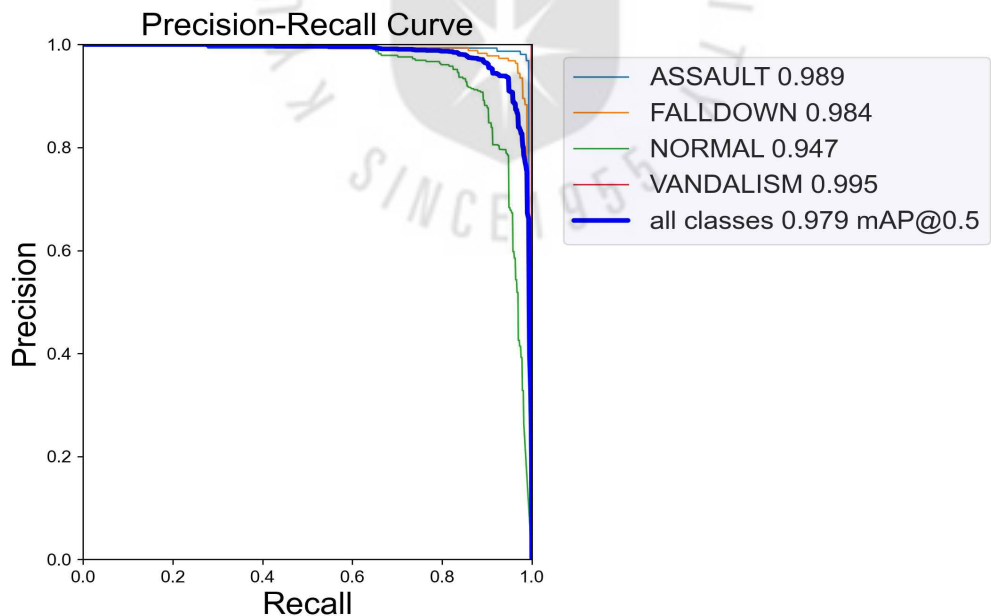


그림 94. 팽창 컨벌루션 YOLOv5 s버전 학습결과 Precision-Recall Confidence
그래프

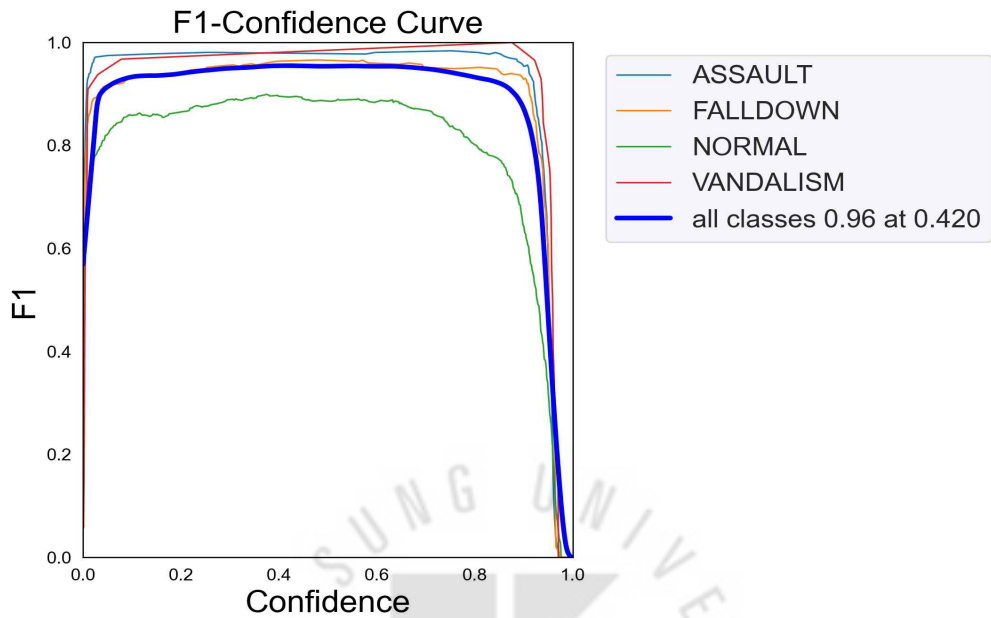


그림 95. 팽창 컨벌루션 YOLOv5 s버전 학습결과 F1-Confidence 그래프

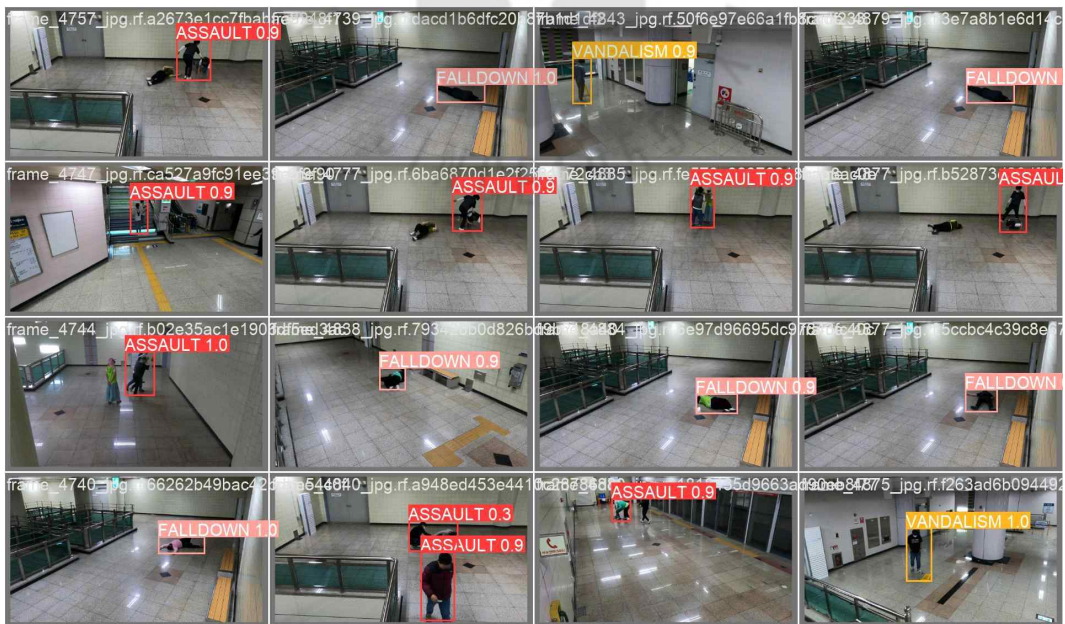


그림 96. 팽창 컨벌루션 YOLOv5 s버전 학습결과

3.8 YOLOv5 m버전 팽창(dilated) 컨벌루션 4 클래스 분류 학습 결과

그림 97, 98, 99 그리고 100은 5x5 팽창 컨벌루션을 적용한 YOLOv5 m버전의 정밀도, 재현률, 정밀도-재현률 그리고 F1-Score 그래프이다. 정밀도는 신뢰 수준 98.4%에서 모든 클래스에 대해서는 100%이며, 재현률은 신뢰 수준 0%에서 99%이다. 정밀도-재현률 그래프는 mAP50으로 98.1%를 나타낸다. F1-Score은 신뢰 수준 46.9%에서 96%로 나타낸다. 그림 101은 5x5 팽창 컨벌루션 YOLOv5 m버전의 mAP50에서의 분류 정확도를 나타낸 학습 결과이다.

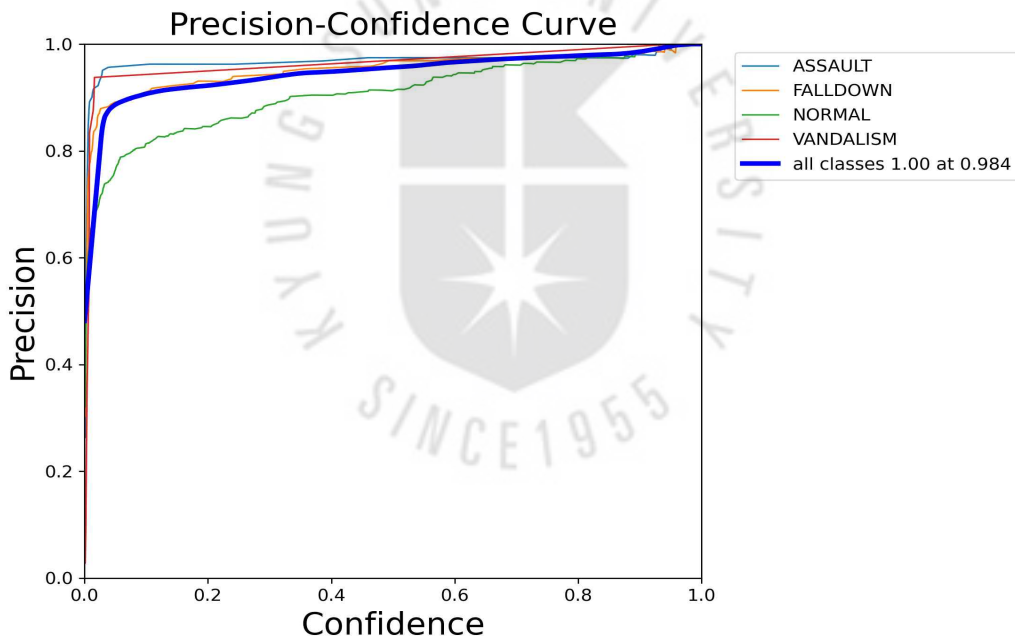


그림 97. 팽창 컨벌루션 YOLOv5 m버전 학습결과 Precision-Confidence 그래프

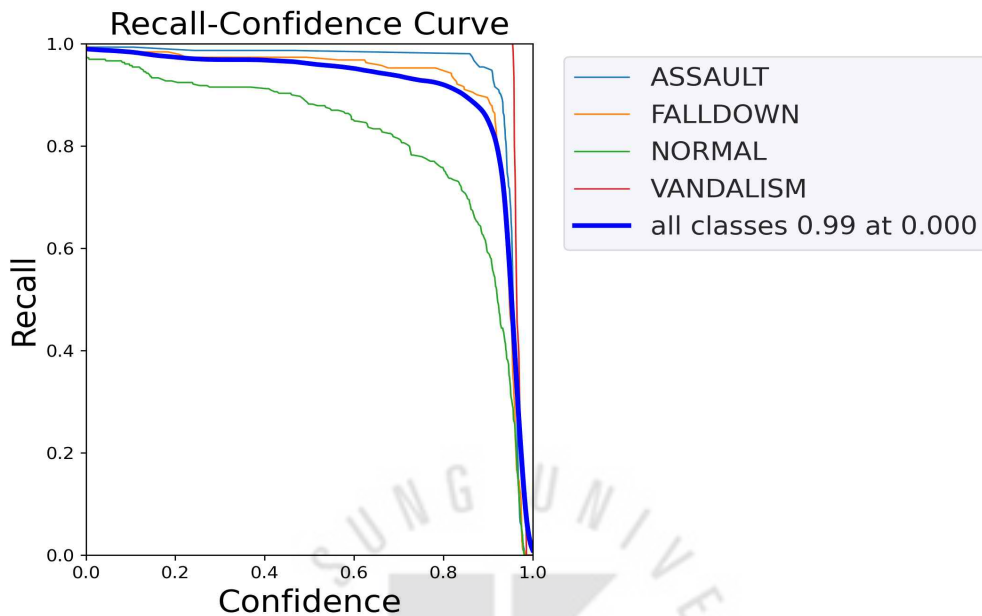


그림 98. 팽창 컨벌루션 YOLOv5 m버전 학습결과 Recall-Confidence 그래프

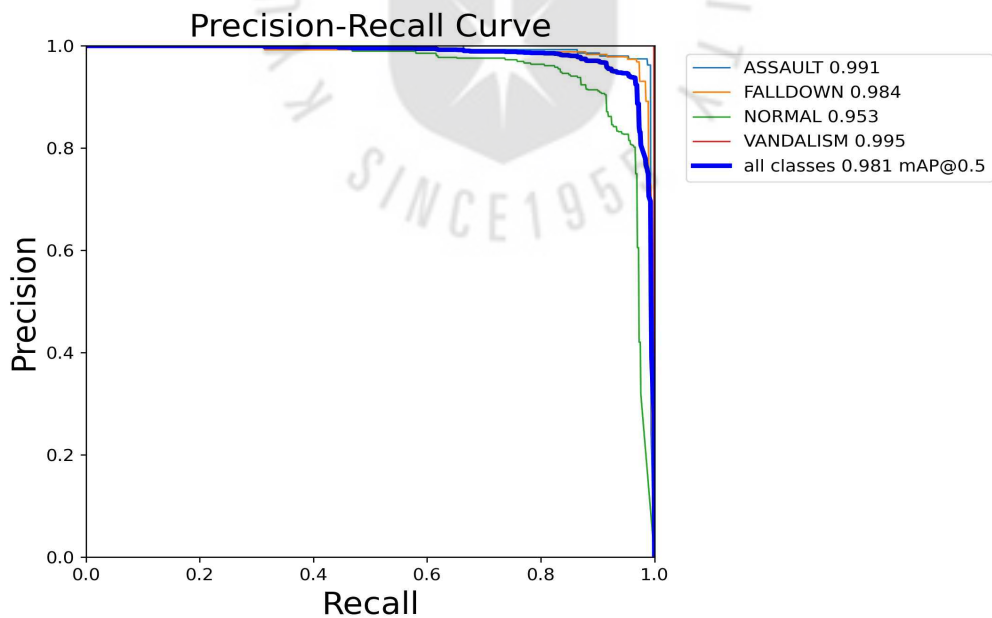


그림 99. 팽창 컨벌루션 YOLOv5 m버전 학습결과 Precision-Recall Confidence
그래프

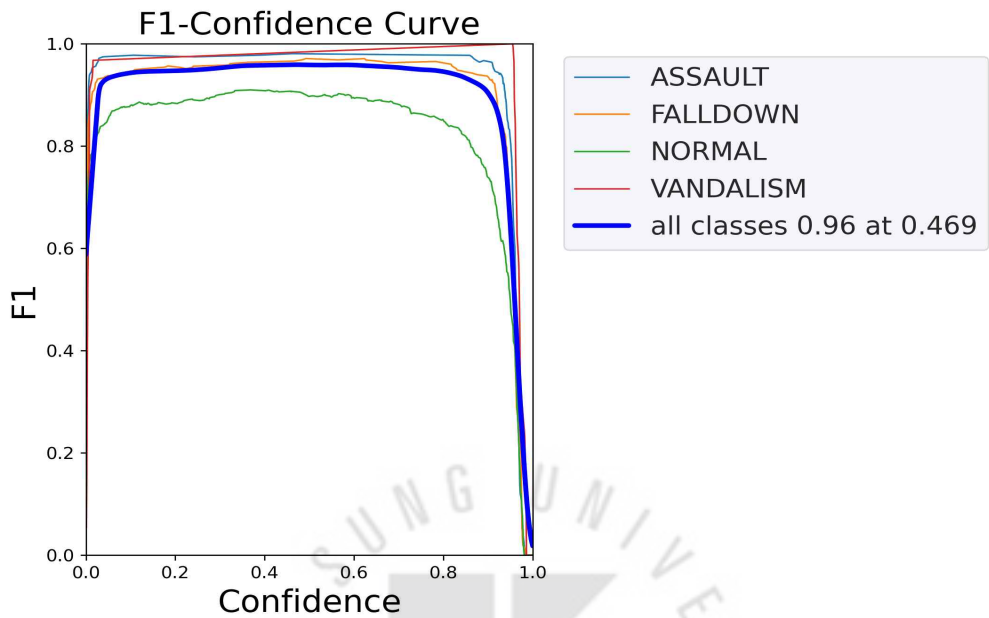


그림 100. 팽창 컨벌루션 YOLOv5 m버전 학습결과 F1-Confidence 그래프

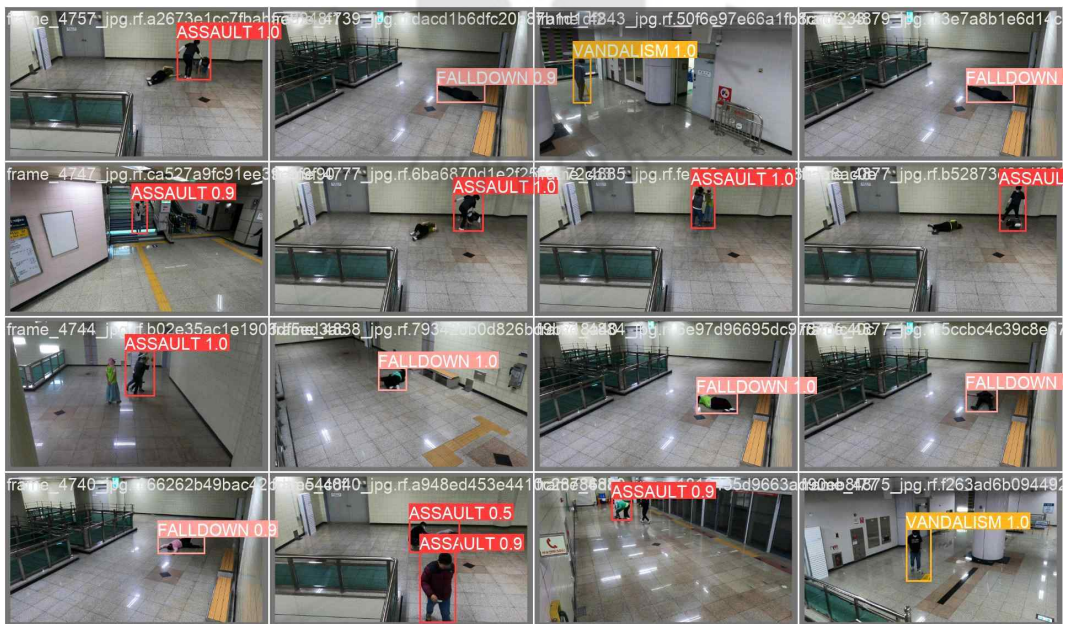


그림 101. 팽창 컨벌루션 YOLOv5 m버전 학습결과

3.9 YOLOv5 1버전 팽창(dilated) 컨벌루션 4 클래스 분류 학습 결과

그림 102, 103, 104 그리고 105는 5x5 팽창 컨벌루션을 적용한 YOLOv5 1버전의 정밀도, 재현률, 정밀도-재현률 그리고 F1-Score 그래프이다. 정밀도는 신뢰 수준 97.2%에서 모든 클래스에 대해서는 100%이며, 재현률은 신뢰 수준 0%에서 99%이다. 정밀도-재현률 그래프는 mAP50으로 98.3%를 나타낸다. F1-Score는 신뢰 수준 43.1%에서 96%로 나타난다. 그림 106은 5x5 팽창 컨벌루션 YOLOv5 1버전의 mAP50에서의 분류 정확도를 나타낸 학습 결과이다.

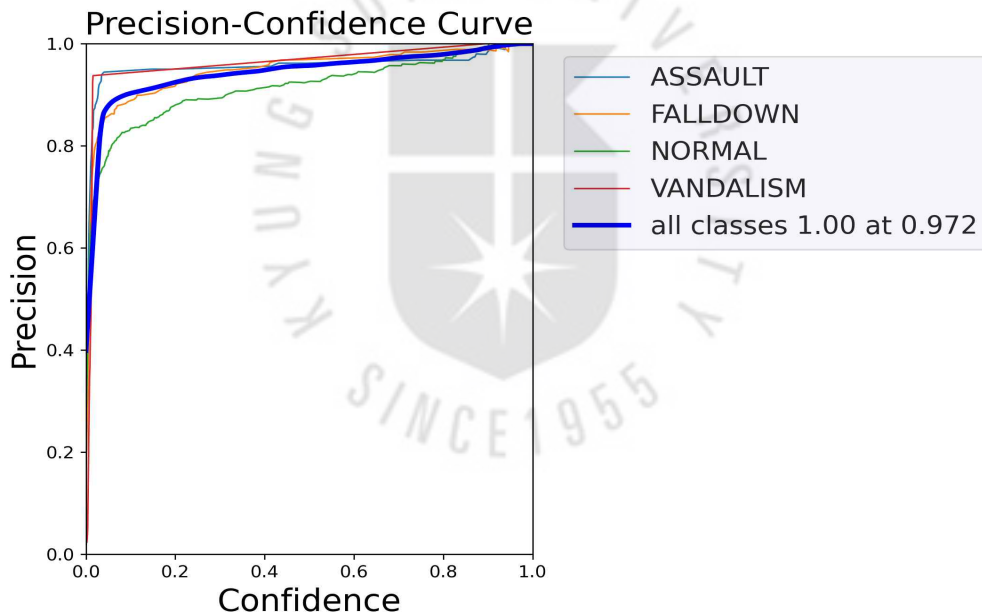


그림 102 팽창 컨벌루션 YOLOv5 1버전 학습결과 Precision-Confidence 그래프

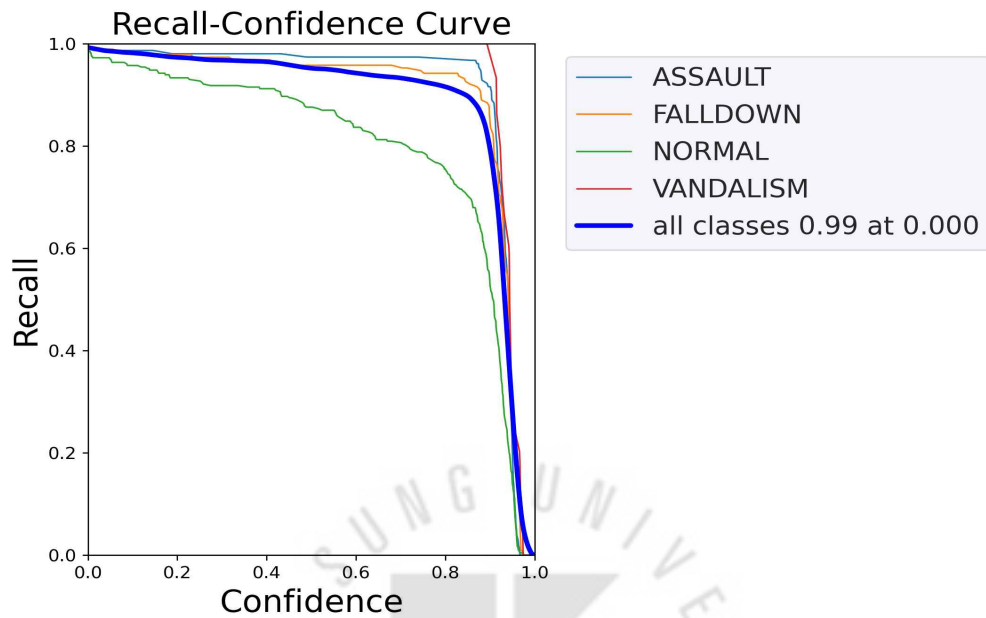


그림 103 팽창 컨벌루션 YOLOv5 1버전 학습결과 Recall-Confidence 그래프

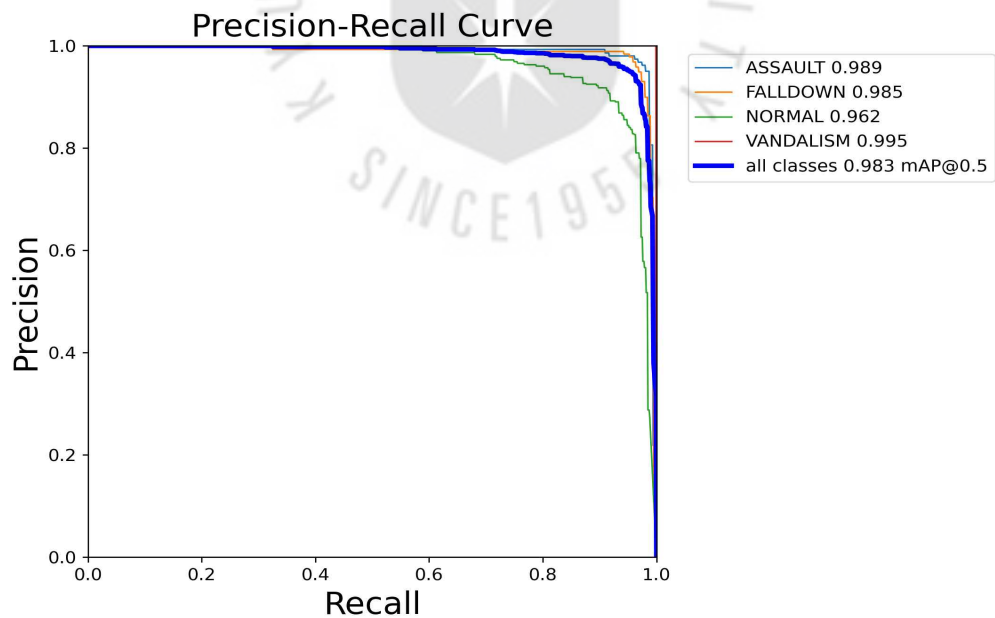


그림 104 팽창 컨벌루션 YOLOv5 1버전 학습결과 Precision-Recall Confidence
그래프

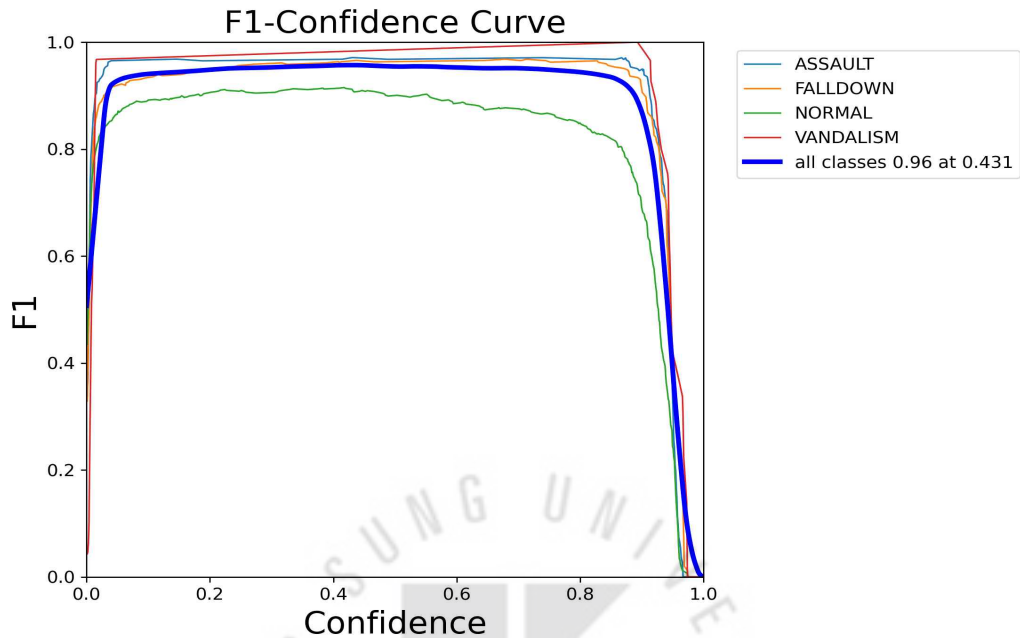


그림 105 팽창 컨벌루션 YOLOv5 1버전 학습결과 F1-Confidence 그래프

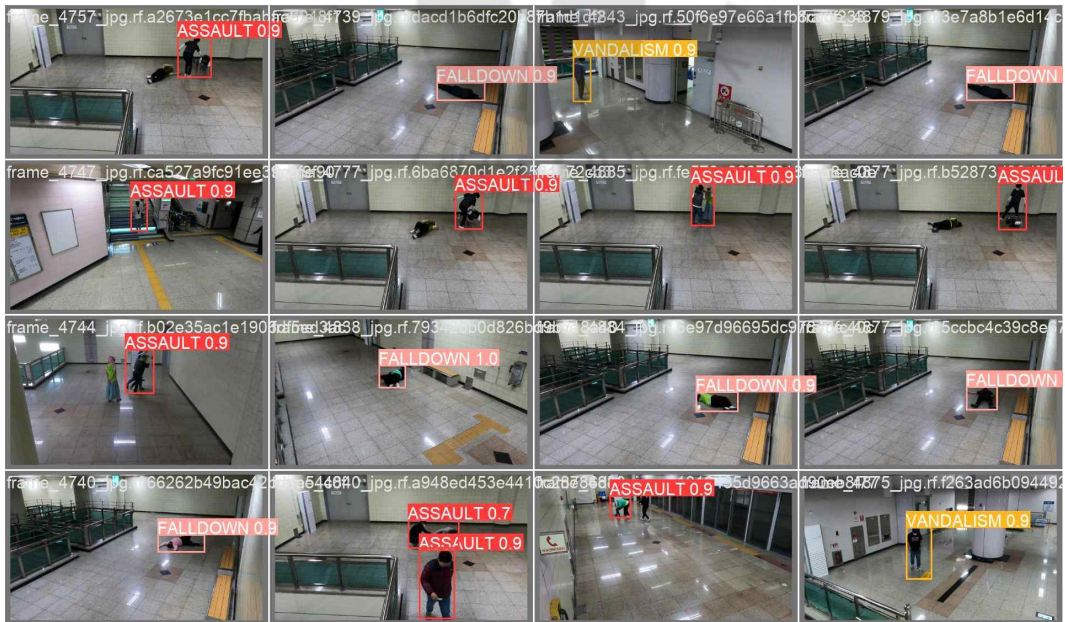


그림 106 팽창 컨벌루션 YOLOv5 1버전 학습결과

3.10 YOLOv5 x버전 팽창(dilated) 컨벌루션 4 클래스 분류 학습 결과

그림 107, 108, 109 그리고 110은 5x5 팽창 컨벌루션을 적용한 YOLOv5 x버전의 정밀도, 재현률, 정밀도-재현률 그리고 F1-Score 그래프이다. 정밀도는 신뢰 수준 99.9%에서 모든 클래스에 대해서는 100% 이며, 재현률은 신뢰 수준 0%에서 99% 이다. 정밀도-재현률 그래프는 mAP50으로 98.3%를 나타낸다. F1-Score는 신뢰 수준 52.1%에서 97%로 나타난다. 그림 111은 5x5 팽창 컨벌루션 YOLOv5 x버전의 mAP50에서의 분류 정확도를 나타낸 학습 결과이다.

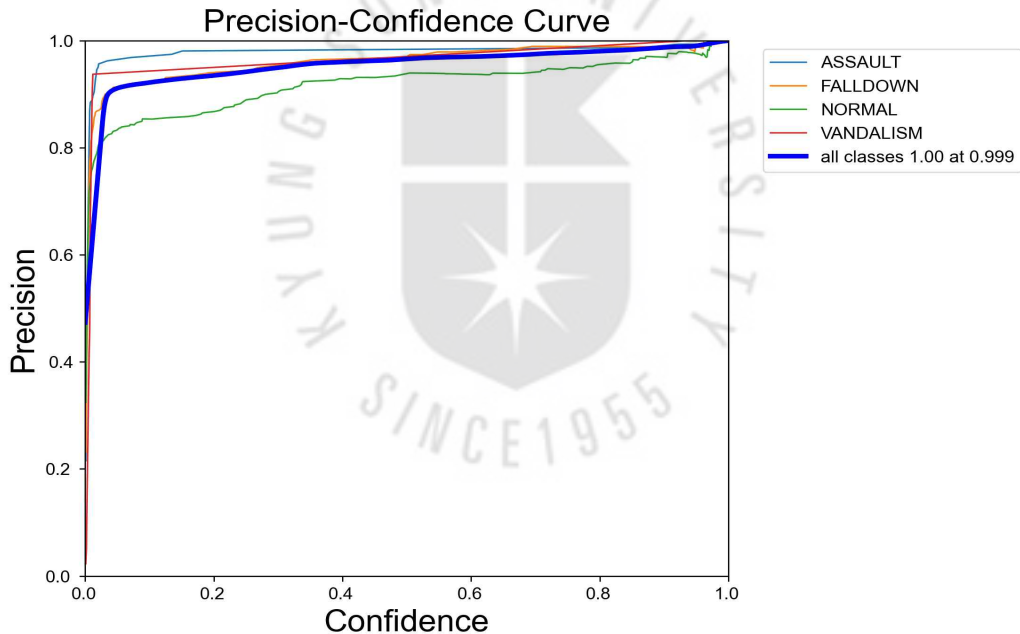


그림 107 팽창 컨벌루션 YOLOv5 x버전 학습결과 Precision-Confidence 그래프

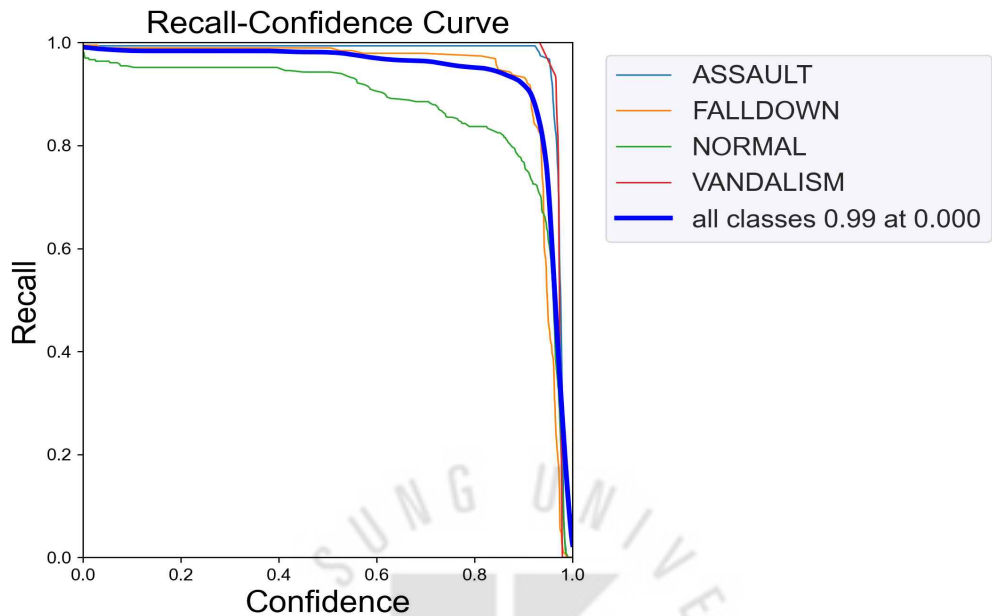


그림 108 팽창 컨벌루션 YOLOv5 x버전 학습결과 Recall-Confidence 그래프

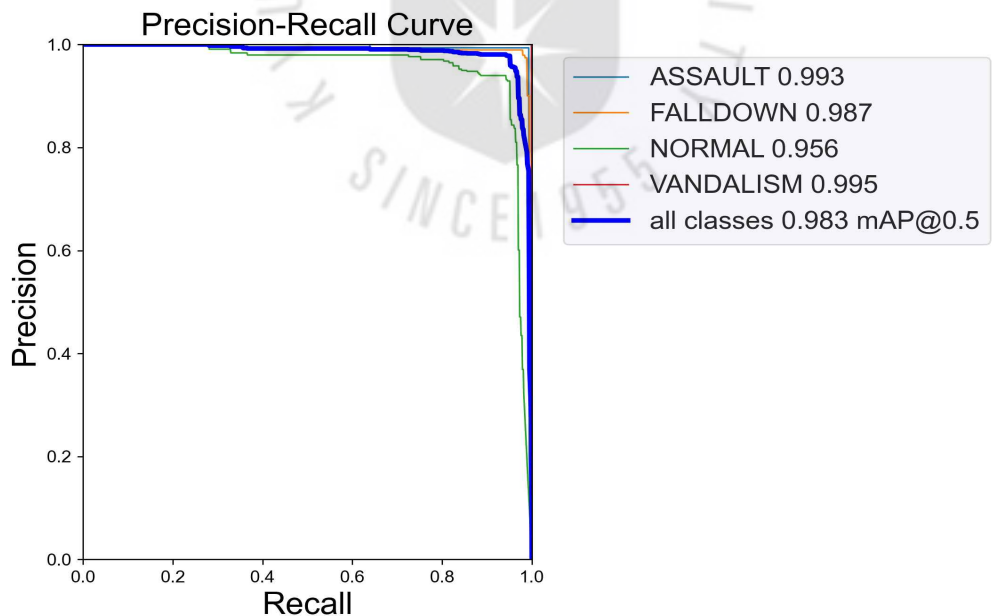


그림 109 팽창 컨벌루션 YOLOv5 x버전 학습결과 Precision-Recall Confidence
그래프

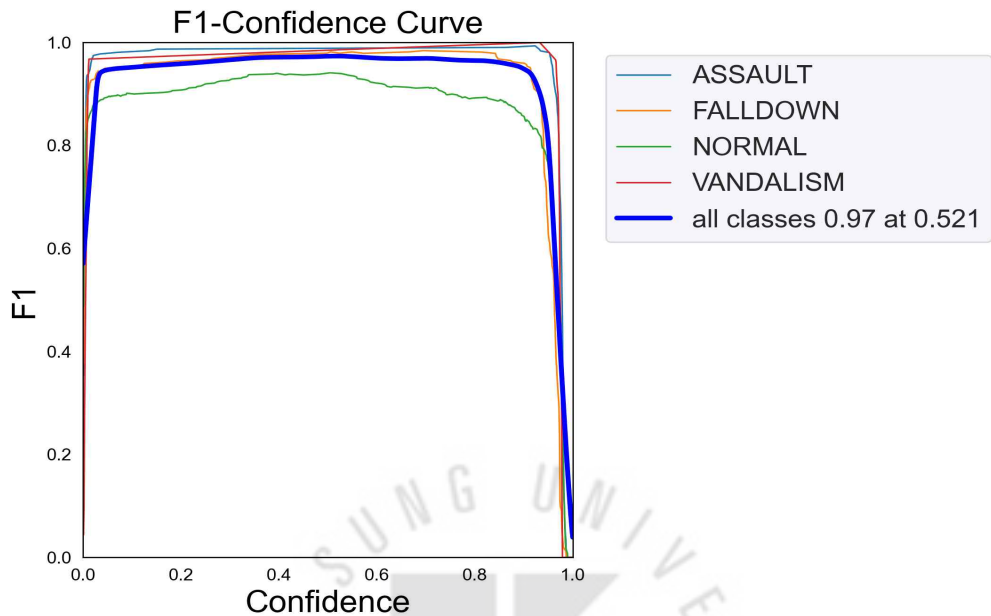


그림 110 팽창 컨벌루션 YOLOv5 x버전 학습결과 F1-Confidence 그래프

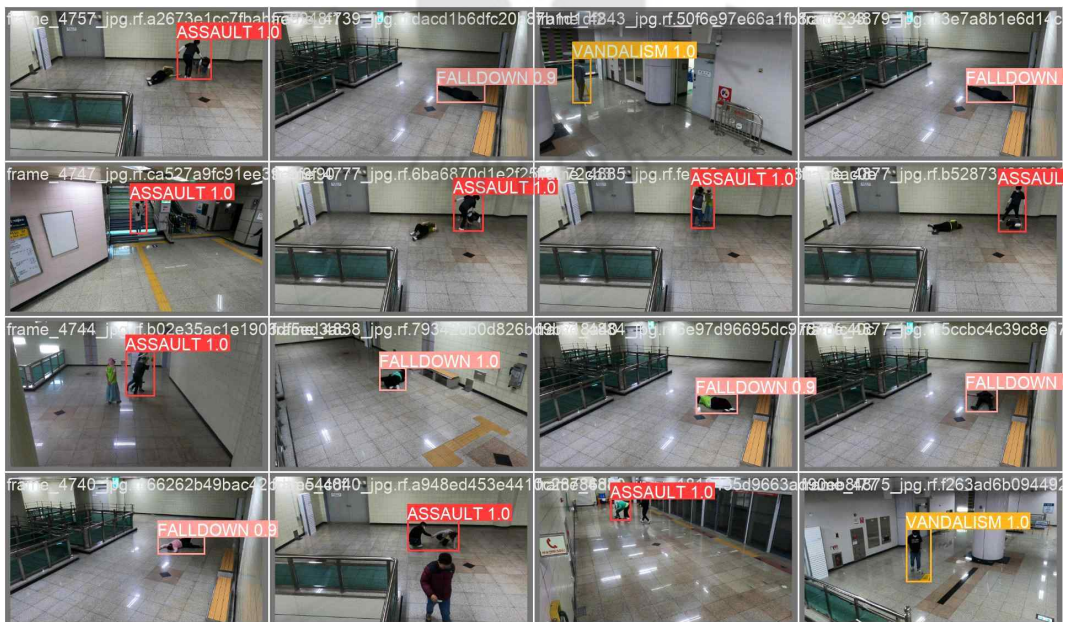


그림 111 팽창 컨벌루션 YOLOv5 x버전 학습결과

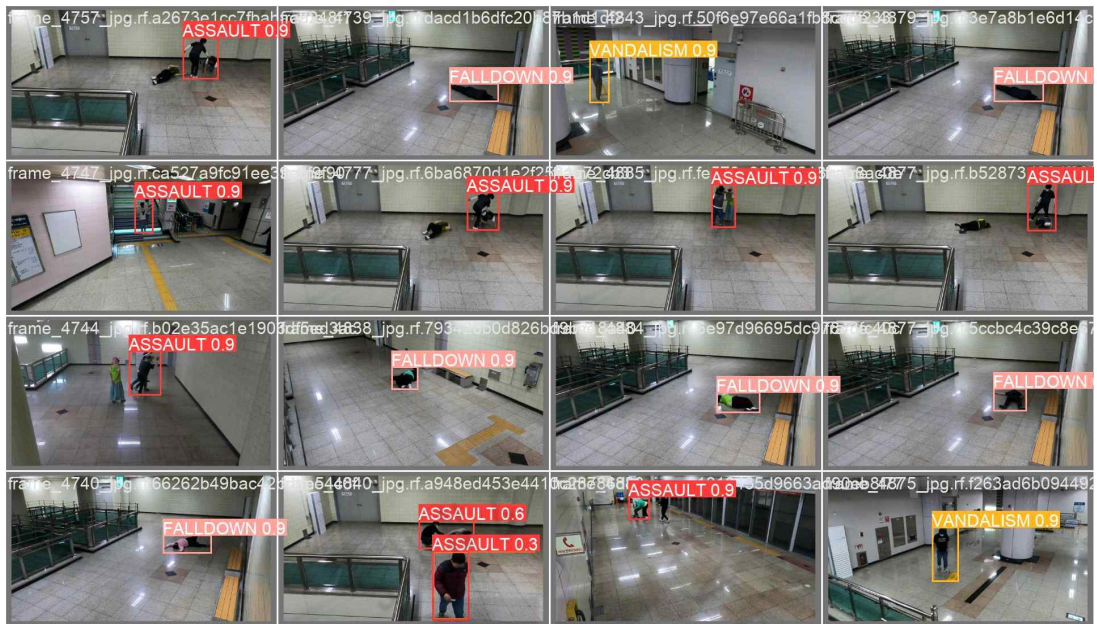


그림 112. YOLOv5 n버전 학습결과



그림 113. 팽창 컨벌루션 YOLOv5 n 버전 학습결과

그림 112는 3x3 컨벌루션 YOLOv5 각 버전들과 그림 113은 5x5 컨벌루션 YOLOv5 n 버전의 mAP50에서의 분류 정확도를 비교한 것으로 팽창 컨벌루션을 적용하면 3x3 컨벌루션 YOLOv5 n 버전에 미세하게 개선됨을 확인할 수 있다.

그림 114는 3x3 컨벌루션 결과와 팽창 컨벌루션 n 버전을 비교한 것이다.

mAP50을 비교하면 YOLOv5n 은 97.8% YOLOv5n의 팽창컨벌루션 적용하였을 때 97.9%로 미세한 차이로 개선됨을 확인하였고, YOLOv5 n, s, m, l, x 모델을 비교하였을 때 mAP50의 큰 차이는 나지 않는 것으로 보아 레이어 수가 적은 n 버전 모델을 활용하는 것이 계산량을 줄 일 수 있으며, 팽창 컨벌루션 n 버전과의 결과도 큰 차이는 없으나 도시철도 역사 내 CCTV 설치 위치를 고려해 볼 때 확장율 2를 사용하여 커널을 사용하게 됨으로 3x3 컨벌루션 결과의 파라미터를 사용할 때와 동일한 뷰를 가지게 효과를 볼 수 있는 5x5 컨벌루션 YOLOv5 n 버전 모델을 적용하는 것이 더 효과적이며 처리 속도 측면에서도 효율적이라 판단한다.

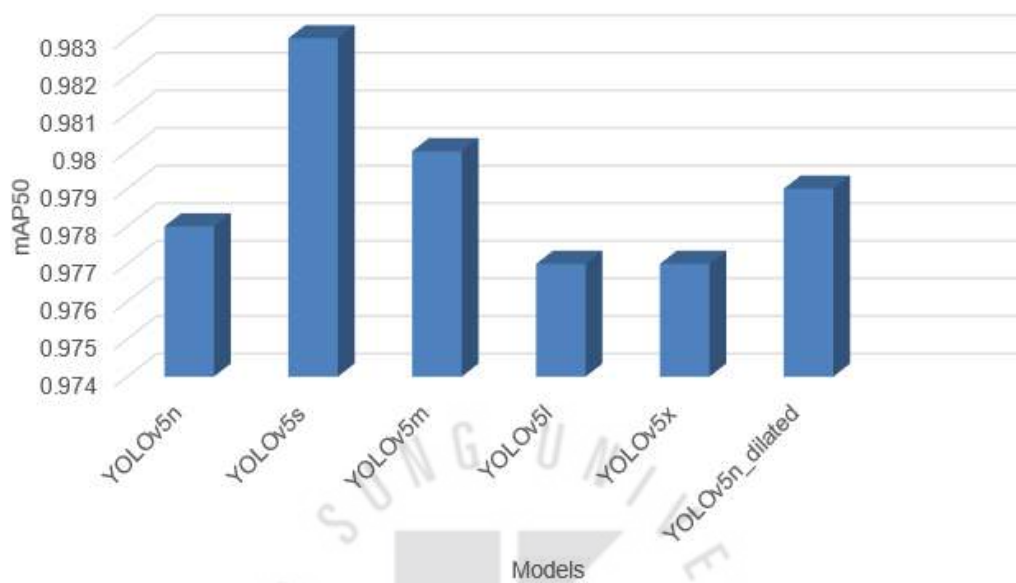


그림 114. 3x3 컨벌루션 결과와 팽창 컨벌루션 n버전 비교

4. 안드로이드 앱 구현 결과

본 논문에서는 임베디드 에지 컴퓨팅 시스템에 적합한 이상행동 감지 모델을 선정하고 팽창 컨벌루션을 활용하여 성능을 개선하였다. 팽창 컨벌루션을 적용한 YOLOv5 n 버전을 앱으로 구현하여 실시간 감지를 확인한다.

그림 115은 YOLOv5 n 버전의 Pytorch 계수 파일(weight file), 즉 pt 파일(pytorch weight file)을 안드로이드 앱을 위한 경량화 계수 파일인 ptl 파일(pytorch weight lite) 변환하는 과정이다. Pytorch의 경량화를 지원하는 프로그램으로 경량화를 수행한다.

그림 116는 변환된 YOLOv5의 경량화 계수를 활용하여 안드로이드 앱으로 구현한 결과이다. 안드로이드 앱은 도시철도 이상행동 정지영상을 6건을 저장하여 쓰러짐 이벤트를 감지하는 것을 시연할 수 있으며, 안드로이드 스마트폰의 카메라 영상에 대하여 실시간 이상행동을 감지할 수 있는 기능을 포함하고 있다.

데모 영상에 대하여 정상상태와 쓰러짐을 분리하여 검출하는 것을 확인할 수 있으며, 유리면에 반사된 사람이 검출되는 것도 확인된다.

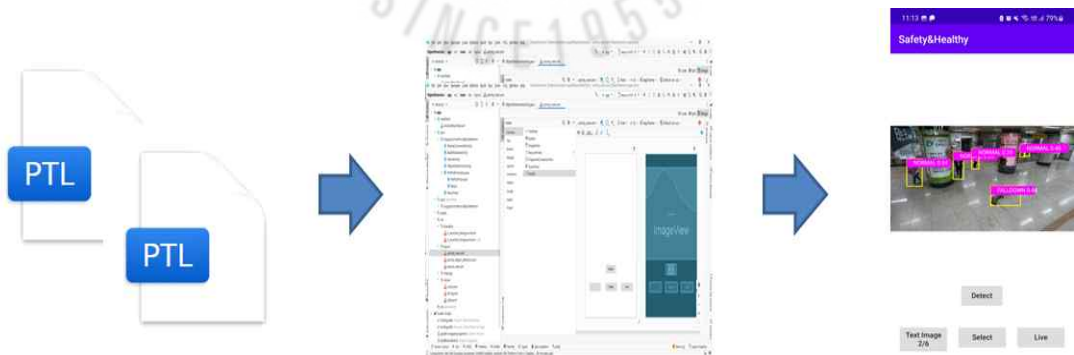


그림 115. 안드로이드 앱 구현 과정

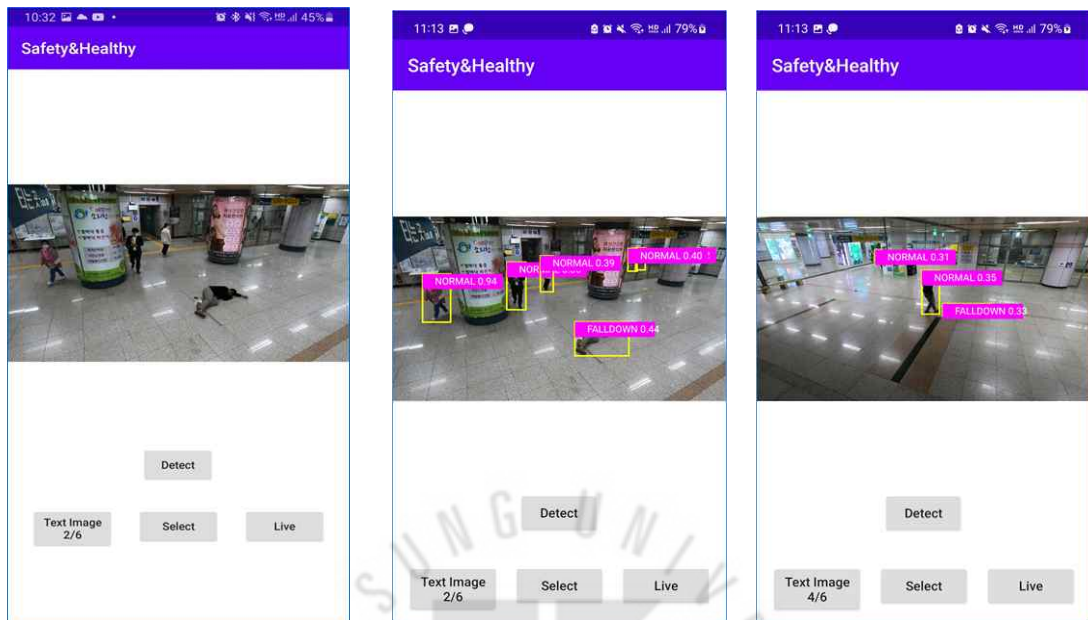


그림 116. 안드로이드 앱 구현 결과

V. 결론

도시철도는 많은 시민들이 활용하고 있기 때문에 안전한 관리가 필요하지만, 최근에는 역사 관리를 위한 인원을 감소하고 있어 효율적인 역사 안전 관리를 위해서는 지능형 CCTV 시스템 도입이 필요하다.

본 논문에서는 실시간으로 도시철도 역사내 이상행동을 감지하기 위하여 객체 검출 딥러닝 모델의 감지 성능을 비교 분석하고, 경량화를 통한 임베디드 에지 컴퓨팅에 적합한 모델을 제안한다. 제안하는 모델은 YOLOv5 n 버전을 팽창 컨벌루션을 적용하여 성능을 개선한 것이다.

YOLOv5의 5가지 버전에 대하여 이상행동 학습을 통하여 성능을 비교 분석한 결과 레이어가 155개인 n 버전과 322개인 x 버전이 0.6%로 크지 않음을 확인하였다. YOLOv5 n 버전에 대하여 팽창 컨벌루션을 적용하여 배경에 대한 오감지가 다소 있는 정상상태 검출 성능을 3% 개선된다. 팽창 컨벌루션은 계산량의 증가 없이 밀집도가 낮은 영상(sparse image)에 대한 성능 개선을 할 수 있다.

3x3 컨벌루션의 결과는 쓰러짐과 정상상태 검출률은 각각 97%, 88%이며, 배경을 정상상태로 오감지하는 것이 9%이다. 5x5 팽창 컨벌루션을 적용한 결과 쓰러짐과 정상상태 검출은 각각 97%, 91%로 정상상태 검출에 대한 성능이 3% 개선된다. 정상상태에 대한 배경으로의 오감지도 6%로 3% 정도 개선되는 것을 확인할 수 있다.

4 클래스에서의 일반 컨벌루션과 팽창 컨벌루션 결과에 대해 비교 분석한 결과 3x3 컨벌루션 YOLOv5 각 버전들과 5x5 컨벌루션 YOLOv5 n 버전의 mAP50에서의 분류 정확도를 비교한 것으로 팽창 컨벌루션을 적용하면 3x3 컨벌루션 YOLOv5 n버전에 미세하게 개선됨을 확인할 수 있다.

mAP50을 비교하면 YOLOv5n 은 97.8% YOLOv5n의 팽창컨벌루션 적용하였을 때 97.9%로 미세한 차이로 개선됨을 확인하였고, YOLOv5 n, s, m, l, x 모델을 비교하였을 때 mAP50의 큰 차이는 나지 않는 것으로 보아 레이어 수가 적은

n 버전 모델을 활용하는 것이 계산량을 줄일 수 있으며, 팡창 컨벌루션 n 버전과의 결과도 큰 차이는 없으나 도시철도 역사 내 CCTV 설치 위치를 고려해 볼 때 확장을 2를 사용하여 커널을 사용하게 됨으로 3x3 컨벌루션 결과의 파라미터를 사용할 때와 동일한 뷰를 가지게 효과를 볼 수 있는 5x5 컨벌루션 YOLOv5 n 버전 모델을 적용하는 것이 더 효과적이며 처리 속도 측면에서도 효율적이라 판단한다.

팡창 컨벌루션을 적용한 YOLOv5 n 버전을 안드로이드 앱으로 구현하여 실시간 이상행동 감지가 가능함을 보인다. Pytorch 플랫폼으로 학습한 계수 파일을 Pytorch 경량화 프로그램으로 경량화 처리를 하고 안드로이드 앱에 적용하여 실시간 이상행동 감지 앱을 구현하여 에지컴퓨팅에 적용할 수 가능성을 확인한다.

이상행동 감지 정확도를 개선하기 위하여 학습데이터를 확대하고 도시철도 역사 내에서 발생할 수 있는 이상행동 이벤트를 추가하여 개발한 이상행동 감지 시스템을 개선하고자 한다.

참고문헌

- [1] 부산교통공사, “도시철도 운영시스템” 제1권, 2016, pp. 144-145.
- [2] 부산교통공사, “2020년 안전사고 통계 내부 보고서” 2022, pp. 2-3.
- [3] 김태운, “대구지하철 참사 사례연구” 한국정책학회 회보, 2005, pp. 275-304
- [4] www.aihub.or.kr AI Hub
- [5] www.aihub.or.kr AI Hub, 지하철 역사 내 CCTV 이상행동 데이터셋 사용설명서
- [6] Ju Han and Bir Bhanu, “*Human Activity Recognition in Thermal Infrared Imagery*,” IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 2005
- [7] 지능형 보안 알고리즘을 이용한 야간 영상 기반의 이상 행동 감지 기술 개발, 교 육과학기술부 연구보고서, 2013
- [8] 박홍진, “인공지능, 기계학습, 딥러닝 분야의 국내 논문 동향 분석,” 한국정보전 자통신기술학회 논문지, 제13권 제4호, 2020, pp 283-292
- [9] N.Munir, H.J.Kim, S.J.Song, S.S.Kang, “*Investigation of deep neural network with drop out for ultrasonic flaw classification in weldments*,” *Journal of Mechanical Science and Technology*, vol. 32, no. 7, pp.3073-3080
- [10] N.Munir, J.Park, H.J.Kim, S.J.Song, S.S.Kang, “*Performance enhancement of convolutional neural network for ultrasonic flaw classification by adopting autoencoder*,” *NDT & E International*, vol. 111, pp.1-13
- [11] 개인 블로그, Yolo 논문 정리 및 Pytorch 코드 구현, 분석 01 (You Only Look Once: Unified, Real-Time Object Detection), 2022. 2. 10., 검색 <https://wingnim.tistory.com/56>
- [12] J. Redmon, S. Divvala, R. Girshick, A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” 2016.

- [13] 하지석, 최철호, 문병인, “YOLOv2 구조 기반 얼굴 검출기의 성능 향상을 위한 웨이블릿 전처리 방법,” 한국정보기술학회 2021년도 하계종합학술대회 및 대학생논문경진대회, pp. 151-153
- [14] 개인 블로그, YOLO, YOLOv2 및 YOLOv3: 알고 싶은 모든 것, 2022. 1. 20., 검색 <https://amrokamal-47691.medium.com/>
- [15] J. Redmon, A. Farhadi, “YOLO9000: Better, faster, stronger,” In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Honolulu, HI, USA, 21-26 July 2017; pp. 7263-7271.
- [16] 개인 블로그, YOLO v2 논문(YOLO9000: Better, Faster, Stronger) 리뷰, 2022. 1. 25., 검색 <https://herbwood.tistory.com/17>
- [17] W. Chen, H. hongbo, S. Peng, C. Zhou, C. Zhang, “YOLO-face: a real-time face detector,” Vis. Computer, vol. 37, pp. 805-813, 2021
- [18] J. Redmon et al., “YOLOv3: An Incremental Improvement,” Computer Vision and Pattern Recognition, arXiv:1804.02767, 2018.
- [19] S. Ren, K. He, R. Girshick, and J. Sun. “Faster r-cnn: Towards real-time object detection with region proposal networks,” arXiv preprint arXiv:1506.01497, 2015.
- [20] 개인 블로그, YOLO v3 논문(YOLOv3: An Incremental Improvement) 리뷰, 2022. 3. 1., 검색 <https://herbwood.tistory.com/21>
- [21] P. Zhang, Y. Zhong, & X. Li, “SlimYOLOv3: Narrower, faster and better for real-time UAV applications,” In Proceedings of the IEEE/CVF International Conference on Computer Vision Workshop, 2019.
- [22] P. Adarsh, P. Rathi, , & M. Kumar, “YOLO v3-Tiny: Object Detection and Recognition using one stage improved model,” In 2020 6th International Conference on Advanced Computing and Communication Systems (ICACCS), (pp. 687-694), IEEE, (2020, March).

- [23] 이용환, 김영섭, 객체 검출을 위한 CNN과 YOLO 성능 비교 실험, 반도체디스플레이학회지 제19권 제1호, 2020
- [24] A. Bochkovskiy, C. Y. Wang, and H. Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” Computer Science, arXiv:2004.10934, Apr, 2020.
- [25] 개인 블로그, YOLO v4 논문(YOLOv4: Optimal Speed and Accuracy of Object Detection) 리뷰, 2022. 3. 20., 검색
<https://herbwood.tistory.com/24>
- [26] 김용현. “YOLOv4와 opticalflow를 활용한 보행자 행동검출,” 국내석사학위논문 성균관대학교 일반대학원, 2021.
- [27] 위성민. “실시간 에지 컴퓨팅을 위한 DNN 기반 YOLOv4 물체 검출기 간소화 및 고속화,” 국내석사학위논문 세종대학교 대학원, 2022.
- [28] Glenn Jocher, YOLOv5 Classification Models, Apple ML, Reproducibility, ClearML and Deci.ai integrations. 2022. Available online:
<https://github.com/ultralytics/yolov5/releases>.
- [29] 개인 블로그, 논문리뷰 You Only Look Once v5 - blog, 2022. 5. 20., 검색
<https://dkssud8150.github.io/posts/YoloV5/>
- [30] 개인 블로그, Yolov5 Yolo를 이용한 Object Detection, 2022. 5. 23., 검색
<https://ssong915.tistory.com/43/>
- [31] W.S. Mseddi, M.A. Sedrine, R. Attia, “YOLOv5 Based Visual Localization for Autonomous Vehicles,” In Proceedings of the 2021 29th European Signal Processing Conference (EUSIPCO), Dublin, Ireland, 23–27 August 2021; IEEE: Piscataway, NJ, USA; pp. 746–750.ine:
<https://github.com/ultralytics/yolov5/discussions/3181m1> (accessed on 20 October 2021).

- [32] C. Y. Wang, A. Bochkovskiy, M. H. Liao, “YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors “.
- [33] L. Wei, A. D. Anguelov, D. Erhan, C. Szegedy, Scott Reed, C. Y. Fu, A. C. Berg, “SSD: Single Shot Multibox Detector, “ Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops. 2016.
- [34] J. A. Kim, J. Y. Sung, S. H. Park, “Comparison of Faster-RCNN, YOLO, and SSD for real-time vehicle type recognition,” In Proceedings of the 2020 IEEE International Conference on Consumer Electronics-Asia (ICCE-Asia), Seoul, Korea, 1-3 November2020; IEEE: Piscataway, NJ, USA, 2020; pp. 1-4.
- [35] M. Li, Z. Zhang, L. Lei, X. Wang, X. Guo, “Agricultural greenhouses detection in high-resolution satellite images based on convolutional neural networks: Comparison of Faster R-CNN, YOLOv3 and SSD,” Sensors 2020, 20, 4938.
- [36] M. Tan, R. Pang, Q. V. Le, “EfficientDet: Scalable and Efficient Object Detection,” Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops. 2019.
- [37] S. Hochreiter, J. Schmidhuber “Long Short-term Memory,” Neural Computation, vol. 9, no. 8, pp.1735-1780, 1997.
- [38] J. Chung, C. Gulcehre, K. Cho, Y. Bengio, “Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling,” NIPS 2014, Deep Learning and Representation Learning Workshop, 2014.
- [39] 임승진, Nustrat Jahan Tahira, 손정락, 박장식, “도시철도 안전관리를 위한 역사내 LSTM 기반 이상행동 감지,” 한국철도학회 2022년도 한국철도학회 추계학술대회 논문집
- [40] 과학기술정보통신부, NIA 한국정보화진흥원, 한국정보통신기술협회, “인공지능 데이터셋 구축 가이드북” 책자, 2019
- [41] NIA 한국정보화진흥원, “신뢰 가능 AI 구현을 위한 정책 방향, OECD AI 권고안을 중심

으로” 제2호, 2019

- [42] NIA 한국정보화진흥원, “AI 학습용 데이터 사업의 실효성 향상을 위한 정책 방향” 제7호, 2020
- [43] L. Chen, G. Papandreou, I. Kokkinos, K. Murphy and A. L. Yuille, “*DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs,*” ArXiv, Computer Vision and Pattern Recognition, 2017.



Doctoral Dissertation

For the safety management of urban railways
Deep learning-based anomaly detection

Lim, Seung-Jin

Department of Railway System Engineering

Graduate School

Kyungsung University

Advised by Professor Park, Jang-Sik

Abstract

In the 1970s, Korea began to build urban railroads to cope with increased traffic demand from the development of industries. On August 15, 1974, the first Seoul Metro Line 1 was opened, and Busan opened Line 1 on July 19, 1985. Busan Metro is currently operating a total of four lines, and transporting an average of 900,000 people a day. While the number of transportation personnel has exploded in the past due to rain, the number of employees working in history is decreasing due to automation

systems in areas such as ticket sales and gate management, it is therefore necessary to introduce an intelligent CCTV system for absolute safety management of railway stations.

In this thesis, I compare and analyze the detection performance of object detection deep learning models to detect abnormal behavior in urban railway stations in real time, and I also propose a model suitable for embedded edge computing through lightening. The proposed model improves performance by applying dilated convolution to the YOLOv5n version.

As a result of comparing and analyzing the performance of the five versions of YOLOv5 through abnormal behavior learning, it was confirmed that the performance for YOLOv5n version with 155 layers and the x version with 322 layers were not large at 0.6%. Applying the dilated convolution to the YOLOv5n version improves the steady-state detection performance by 3% with some. The dilated convolution solution improves performance for sparse images without increasing the computation cost.

In this thesis, I show that real-time abnormal behavior detection is possible by implementing the YOLOv5n version applied with the dilated convolution as on an Android app. The coefficient file learned with the Pythorch platform is lightweight with the Pythorch lightweight program and applied to the Android app to implement a real-time anomaly behavior detection app to confirm the possibility of applying it to edge computing.

In order to improve the accuracy of abnormal behavior detection, we intended to improve the abnormal behavior detection system developed by expanding the learning data and adding abnormal behavior events that may occur within urban railway stations.